

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



SEMINAR
SEMESTER 252 ACADEMIC YEAR 2025-2026

SOFTWARE ARCHITECTURES
FOR IOT SYSTEMS

COURSE: SOFTWARE ARCHITECTURE (CO3017)

ADVISOR: PhD. Trương Thị Thái Minh

GROUP: CC01 - 03

STUDENT 1: Nguyễn Háo Hồng Dũ – 2252129

STUDENT 2: Hà Kiến Hoa – 2252225

STUDENT 3: Nguyễn Tiến Khang – 2252304

STUDENT 4: Nguyễn Đức Quốc Kỳ – 2252415

STUDENT 5: Nguyễn Phúc Điền – 2352258

STUDENT 6: Hồ Quốc Huy – 2352379

STUDENT 7: Nguyễn Minh Triết – 2353214

HO CHI MINH CITY, MARCH 2026

Contents

1	Abstract	1
2	Introduction	3
2.1	IOT Systems	4
2.1.1	Definitions	4
2.1.2	Reference Architecture	7
2.1.3	Common Issues of IoT Systems	10
2.2	Software Architectures & Design Patterns	12
2.2.1	Software Architectures	12
2.2.2	Design Patterns	14
3	Essential Software Architectures	16
3.1	N-tier Architectures	17
3.2	Service-oriented Architectures	21
3.3	Microservice Architectures	23
3.4	Event-driven Architectures	29
4	Analysis of Architectures	34
4.1	Service-oriented Architectures for IoT	34
4.1.1	Service Granularity	34
4.1.2	Communications	36
4.1.3	Service Composition and Orchestration	37
4.1.4	Scalability and Integration	39
4.2	Microservice Architectures	42
4.3	Event-Driven Architecture: MQTT Protocol	46

4.3.1	Definition	46
4.3.2	MQTT Architecture	47
4.3.3	Pub/Sub Communications	52
4.3.4	MQTT Features	52
4.3.5	Security in MQTT	55
4.3.6	Performance Analysis	58
4.3.7	Performance and Scalability Analysis	58
4.3.8	Conclusion	59
4.4	Advanced Protocols: Beyond MQTT	59
4.4.1	MQTT 5.0	59
4.4.2	CoAP: Constrained Application Protocol	61
4.4.3	DDS: Data Distribution Service	63
5	Industrial Case Studies	66
5.1	Case Study 1: MQTT-Based Cold Chain Monitoring System	66
5.1.1	Industrial Problem	66
5.1.2	MQTT-Based System Architecture	68
5.1.3	Implementation Overview	69
5.1.4	Architectural Evaluation	70
5.1.5	Architectural Limitations	71
5.1.6	Conclusion	71
5.2	Case Study 2: Secure MQTT 5.0-Based OTA Architecture	72
5.2.1	Industrial Problem	72
5.2.2	Background: MQTT 5.0 Enhancements	73
5.2.3	Traditional OTA Architecture	74
5.2.4	Proposed MQTT 5.0 OTA Model	74
5.2.5	Topic Design and Communication Workflow	76
5.2.6	Security Analysis	78
5.2.7	Performance Evaluation	79
5.2.8	Architectural Evaluation	79
5.2.9	Conclusion	80

6	Future Directions	81
6.1	Fog Computing	82
6.2	Serverless Architecture	84
7	Conclusion	87
8	Ethics Report: AI Uses	89
	References	89

List of Figures

2.1	High-level overview of a Smart Home	4
2.2	Conceptual architecture of Iot-Enabled smart warehouse.	6
2.3	Iot Gateway is essential in the orchestration of an IoT System	7
2.4	ISO/IEC 30141 separates an IOT System to IoT Environment, System and Component	8
2.5	7 Layer IoT Architecture	9
2.6	The architecture characteristics define the success criteria of a system .	13
2.7	An Exemplary IoT System Architecture	13
3.1	Alternate partitioning for Iot Systems	19
3.2	Example of Edge-Centric IoT System	19
3.3	Service-Oriented Architecture's Topology	21
3.4	Topology of Microservice	23
3.5	Monolithic vs Microservice	24
3.6	Example Microservice-based System	26
3.7	Example IOT System	28
3.8	High-level Overview of Event-driven Architecture for IOT Systems . .	29
3.9	Broker Topology	30
3.10	Mediator Topology	31
4.1	Overview of a CPS sysem	35
4.2	The Enabler Framework for SOA	40
4.3	Reference Architecture for IoT Stream Processing	44
4.4	Basic MQTT Protocol Overview	46

4.5	An Example of how MQTT works	48
4.6	Working Demonstration of MQTT Protocol	49
4.7	MQTT's Quality of Service	51
4.8	Example Last Will Testament Message	54
4.9	High-level Overview of CoAP	61
4.10	Example CoAP session	62
4.11	MQTT vs CoAP	63
4.12	High-level Overview of DDS Systems	64
4.13	A Reference Architecture for DDS	65
5.1	Reference Hardware Setup	67
5.2	IOT System Architecture	69
5.3	MQTT 5.0	73
5.4	The notations.	75
5.5	The two channels of the OTA model	76
5.6	Topic Tree Design	76
5.7	The message interactions during the End2End channel building	77
5.8	The message flows of the OTA update phase	78
6.1	Fog Architecture	83
6.2	Amazon's Serverless IOT System with ESP32	85

Chapter 1

Abstract

The Internet of Things (IoT) has become a cornerstone in modern computing, enabling the integration of physical devices, communication networks, and intelligent data processing systems. However, the inherent characteristics of IoT environments, including data and hardware heterogeneity, large-scale distribution, real-time requirements, and resource constraints, introduce significant architectural challenges. This report presents a comprehensive study of software architectures for IoT systems.

Chapter 2 introduces the fundamental concepts of IoT systems and software architecture, including common IoT reference models, system characteristics, and key design challenges such as scalability, latency, and interoperability. It further defines software architecture and discusses its role in addressing system-wide concerns.

Chapter 3 explores the basic concepts of various architectures, including N-tier Architecture and Service-Oriented Architecture (SOA). Providing their definitions, basic principles, components and some advantages and disadvantages. IoT system is not touched upon here yet, we will leave it to the next chapter.

Chapter 4 focuses on analysis architectural approaches, including Microservice Architecture, Event-Driven Architecture (EDA) and Service-oriented Architecture. This chapter also examines communication protocols that support these architectures, including MQTT, CoAP, and DDS, highlighting their respective trade-offs in performance, scalability, and real-time capabilities. This serves as the central chapter of the report, outlining the architectures and their roles in IoT Systems.

Chapter 5 presents industrial case studies, including MQTT-based cold chain mon-

itoring systems and secure over-the-air (OTA) update mechanisms. These case studies demonstrate the practical application of architectural patterns and protocols, with particular emphasis on system performance, scalability, and security.

Chapter 6 discusses future directions, focusing on Fog Computing and Serverless Architecture.

Finally, Chapter 7 concludes the report by summarizing the key findings and emphasizing the importance of adopting flexible, scalable, and event-driven architectures for modern IoT systems. The report highlights that hybrid approaches combining multiple architectural paradigms are essential to address the complex requirements of large-scale and dynamic IoT environments.

Chapter 2

Introduction

2.1 IOT Systems

2.1.1 Definitions

Internet of Things (IOT for short) is a network of interconnected physical devices embedded with sensors, software, and a form of connectivity that enables them to collect and exchange data with other devices and systems over the internet¹ or other communication networks. These physical devices utilize a unified protocol stack to execute such data exchanges, transforming raw environmental stimuli into digital representations that facilitate autonomous decision making.²

Examples of IoT systems include, but are not limited to, Smart Home Systems, where an entire household is governed by a comprehensive network of interconnected edge devices that execute complex tasks autonomously with minimal human intervention.³ Within these environments, traditional household infrastructure, ranging from environmental controls to security control is transformed into an intelligent, context-aware ecosystem, where users only remotely control through a mobile app, where the edge devices will orchestrate and execute the tasks.

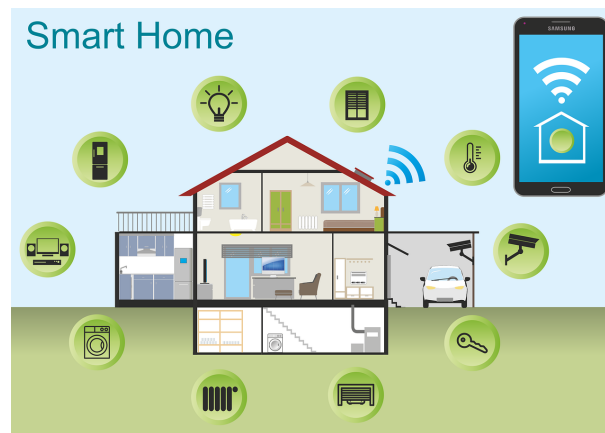


Figure 2.1: High-level overview of a Smart Home

¹*Internet of Things - an overview | ScienceDirect Topics*. URL: https://www.sciencedirect.com/topics/computer-science/internet-of-things?__cf_chl_tk=rZj4oj48zwWVInYRqFpak7u9_NXyAMlOJXc1vhk.I7A-1773386612-1.0.1.1-xSppxPogSd5G0eTkVEMJG8PoI3pKNZrM4XDSr8AQke0 (visited on 02/17/2026).

²ISO/IEC JTC 1/SC 41. *ISO/IEC 30141:2024 internet of things (IoT) — reference architecture*. Tech. rep. International Organization for Standardization, 2024.

³*Manage Your Smart Home With Google Home | Google Home*. en. URL: <https://home.google.com/welcome/> (visited on 03/11/2026); *AWS IoT Connected Home Solutions*. en-US. URL: <https://aws.amazon.com/iot/solutions/connected-home/> (visited on 03/11/2026).

Beyond smart home environments, IoT systems are also widely adopted in industrial and logistics domains, where they enable large-scale automation and intelligent resource management. Modern smart warehouses typically adopt a multi-layered architecture, including a **sensing layer** (RFID tags, environmental sensors, and autonomous mobile robots) that collects real-time data, and an **edge layer** where gateways perform local processing to reduce latency and bandwidth usage. The **network and middleware layer** ensures interoperability through protocols such as MQTT and CoAP, while the **cloud layer** supports large-scale analytics, predictive maintenance, and inventory optimization. Finally, the **application layer** provides dashboards and control systems for real-time monitoring and coordination of warehouse operations. The adoption of IoT technologies significantly improves warehouse performance by increasing inventory accuracy, enhancing operational efficiency, and enabling predictive maintenance. IoT-enabled systems can achieve inventory accuracy exceeding 99.5% while reducing processing time through automation and real-time tracking. Additionally, IoT facilitates demand forecasting, efficient inventory management, and improved customer satisfaction^{4,5}. However, challenges such as cybersecurity risks, interoperability issues, and high implementation costs remain barriers to widespread adoption. Recent studies emphasize the need for scalable, secure, and sustainable IoT architectures, aligning future warehouse systems with the principles of Industry 4.0 and emerging Industry 5.0 paradigms.⁶

To support such complex systems, IoT architectures rely on several fundamental components, particularly edge devices and gateways.

IoT systems are built upon **Edge Devices**, which act as the bridge between the physical world and digital networks by collecting raw data directly from their environment.⁷ This allows them to process and filter data locally rather than sending everything to the cloud, which significantly reduces network traffic and ensures a faster system response and enhance overall system's performance.⁸ Furthermore, due to their local processing nature,

⁴Unknown. "A systematic review of recent advances in IoT-based sensor networks for warehouse management". In: *Journal/Conference TBD* (2025).

⁵Unknown. "Research on impact of IoT on warehouse management". In: *Journal/Conference TBD* (2023).

⁶Unknown, "A systematic review of recent advances in IoT-based sensor networks for warehouse management".

⁷ISO/IEC JTC 1/SC 41, *ISO/IEC 30141:2024 internet of things (IoT) — reference architecture*.

⁸*What is an Edge Device | Glossary*. en-US. URL: <https://www.hpe.com/us/en/what-is/edge-device.html> (visited on 03/11/2026).

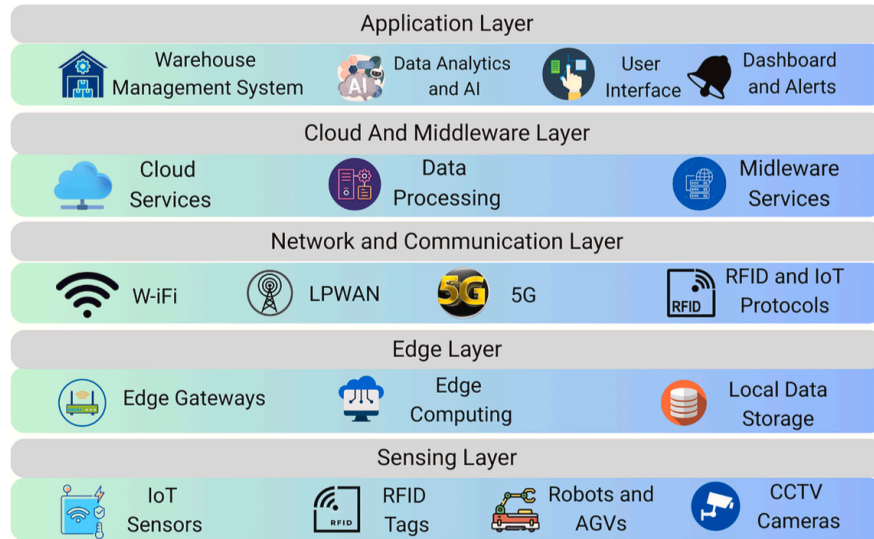


Figure 2.2: Conceptual architecture of Iot-Enabled smart warehouse.

edge devices enable not only efficiency, but also privacy and security of the IoT system, acting as a critical bridge between internal protected networks and external, untrusted networks like the Internet. By applying security policies at this logical boundary, edge devices channel traffic to protect internal services from unfettered access by malicious actors.

In order to orchestrate the workflow and coordination between edge devices and the user, IoT Systems typically employ a form of message broker, in other words a **gateway**. An IoT gateway serves as an intermediary layer between edge devices and cloud-based systems, facilitating communication, data processing, and protocol translation within IoT architectures. Due to the heterogeneity of IoT devices and communication protocols, gateways play a critical role in ensuring interoperability across different system components.⁹ One of the primary functions of an IoT gateway is **protocol translation**. Many edge devices operate using lightweight or specialized communication protocols, while cloud platforms typically rely on standard Internet protocols such as HTTP. The gateway bridges this gap by converting data between protocols, enabling seamless integration.¹⁰ In addition to communication management, IoT gateways often perform **local data processing**, including filtering, aggregation, and preliminary analytics. This re-

⁹ISO/IEC JTC 1/SC 41, *ISO/IEC 30141:2024 internet of things (IoT) — reference architecture*; NIST. *NIST special publication 800-183: Networks of things*. 2016.

¹⁰Cisco. *IoT fundamentals: Networking technologies, protocols, and use cases*. 2017.

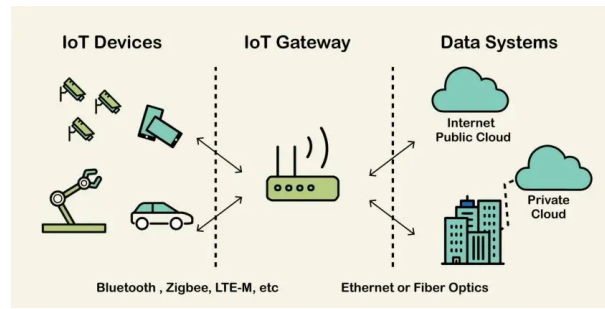


Figure 2.3: Iot Gateway is essential in the orchestration of an IoT System

duces the volume of data transmitted to the cloud, thereby lowering bandwidth usage and improving system efficiency.¹¹ Furthermore, gateways enhance system *security* by acting as a control point between internal device networks and external systems. They can enforce authentication, encryption, and access control policies, protecting IoT infrastructures from unauthorized access and cyber threats.¹² Overall, the IoT gateway is a key architectural component that enables scalability, interoperability, and efficient data management in modern IoT systems.

2.1.2 Reference Architecture

IoT systems are inherently complex due to their heterogeneous devices, distributed nature, and diverse communication requirements. As a result, various architectural models have been proposed to standardize the design and development of IoT systems. Among these, the most widely recognized is the reference architecture defined in ISO/IEC 30141, which provides a comprehensive framework for describing IoT systems.¹³

The ISO/IEC 30141 standard does not enforce a fixed layered structure. Instead, it defines multiple architectural views, including functional, system, and deployment perspectives, allowing flexibility in system design. This abstraction enables architects to model IoT systems according to specific requirements while maintaining interoperability and scalability across different implementations.

In contrast, many practical implementations adopt a layered architectural model to simplify system design and illustrate data flow. A commonly used representation is

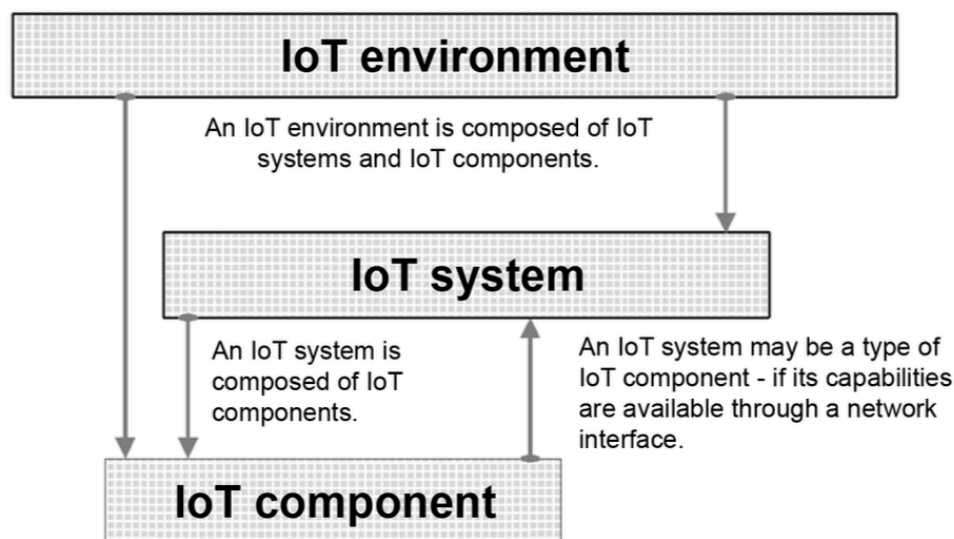
¹¹NIST, *NIST special publication 800-183: Networks of things*.

¹²Microsoft. *Azure IoT gateway documentation*. 2023.

¹³ISO/IEC JTC 1/SC 41, *ISO/IEC 30141:2024 internet of things (IoT) — reference architecture*.

the multi-layer IoT architecture, often described as a seven-layer model. This model organizes IoT systems into layers such as device, connectivity, edge computing, data processing, and application layers, providing a clear and intuitive understanding of how data is generated, transmitted, processed, and utilized. In addition to architectural views, ISO/IEC 30141 defines fundamental **IoT system concepts**, including the IoT environment, IoT system, and IoT component. The **IoT environment** represents the broader context that includes all relevant systems, infrastructure, and external elements. Within this environment, an **IoT system** is defined as a configuration of interacting components designed to provide specific functionalities and value to stakeholders. Each system is composed of multiple **IoT components**, which are the fundamental building blocks responsible for sensing, processing, communication, or actuation.

These concepts are hierarchically related, where IoT components form IoT systems, and multiple IoT systems exist within a broader IoT environment. Notably, ISO/IEC 30141 also allows an IoT system to be considered as a component of a larger system if its functionality is exposed through a network interface. This recursive property supports modularity and scalability in complex IoT architectures.



IEC

Figure 2.4: ISO/IEC 30141 separates an IoT System to IoT Environment, System and Component

Moreover, ISO/IEC 30141 also identifies a set of fundamental **characteristics in-**

herent to IoT systems, which distinguish them from traditional distributed systems,¹⁴ primarily centered on **composability** and **modularity** to ensure discrete components can be integrated, upgraded, or replaced cleanly to meet evolving business goals. These systems are **highly distributed** and **heterogeneous** in nature, often spanning entire cities or the globe while requiring seamless **interoperability** across diverse protocols and cross-domain products. To manage this complexity, a clean separation of functional and management capabilities is essential, especially when devices are geographically dispersed or physically accessible to unauthorized users. Furthermore, the architecture must prioritize **scalability** and network communication **flexibility**, utilizing various proximity and wide-area networks to handle the rapid growth of connected entities and massive data inflows. Efficiency is further enhanced through **shareability**, allowing underutilized components to serve multiple systems, while accuracy remains a critical but cost-sensitive factor, as the precision of sensors and actuators must be carefully balanced against business needs and environmental influences like temperature

While ISO/IEC 30141 provides a formal and flexible reference framework, the layered model offers a more accessible and structured representation. Therefore, the layered approach can be considered a practical interpretation of the principles defined in the ISO standard.

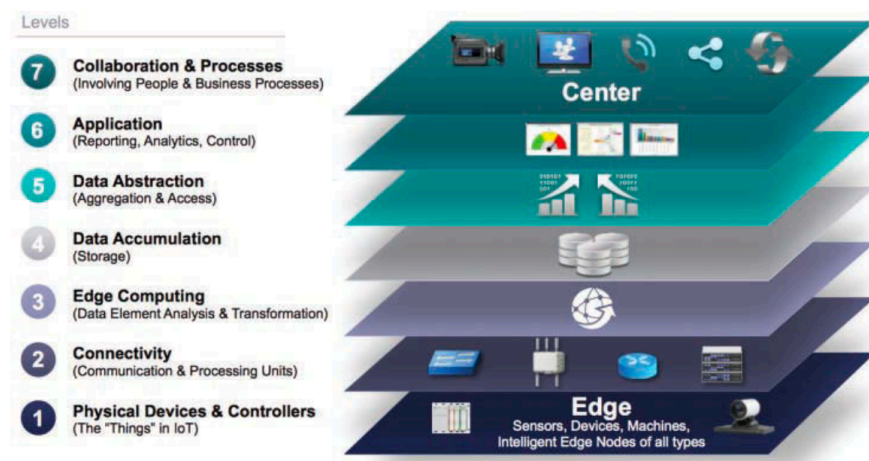


Figure 2.5: 7 Layer IoT Architecture

These reference architectures form the foundation for more advanced software architectural styles, such as N-tier, service-oriented, microservices, and event-driven architec-

¹⁴ISO/IEC JTC 1/SC 41, *ISO/IEC 30141:2024 internet of things (IoT) — reference architecture*.

tures, which are discussed in the following sections. In this architecture, they emphasize the separation of IoT Systems into 7 layers:

- **Physical Devices Layer:** This layer consists of sensors and actuators that interact directly with the physical environment. Sensors collect raw data (e.g., temperature, humidity, motion), while actuators perform actions based on system decisions.
- **Connectivity Layer:** Responsible for transmitting data between devices and other system components through communication technologies such as Wi-Fi, Bluetooth, 5G, and LPWAN. It ensures reliable and secure data transfer.
- **Edge Computing Layer:** Performs local data processing near the source of data generation. This includes filtering, aggregation, and real-time decision-making, reducing latency and bandwidth usage.
- **Data Accumulation Layer:** Handles the storage and aggregation of data collected from edge devices. It converts streaming data into persistent storage formats suitable for further processing.
- **Data Abstraction Layer:** Processes and analyzes stored data, transforming it into meaningful information. This layer includes data modeling, analytics, and integration with higher-level services.
- **Application Layer:** Provides user-facing services and interfaces, such as dashboards, monitoring systems, and control applications. It enables users to interact with and manage the IoT system.
- **Business Layer:** Focuses on business logic, decision-making, and overall system management. It includes analytics, reporting, and strategic operations that align IoT data with organizational goals.

2.1.3 Common Issues of IoT Systems

Despite their advantages, IoT systems introduce several significant challenges due to their scale, heterogeneity, and distributed nature. These challenges stem from the inherent characteristics of IoT environments and must be addressed when designing robust and efficient architectures.

One of the primary challenges is **heterogeneity**. IoT systems consist of diverse devices, communication protocols, and platforms developed by different vendors, making interoperability difficult to achieve.¹⁵ The diversity of hardware capabilities and software environments further complicates standardization and system integration.

Another major issue is **scalability**. IoT deployments often involve a large number of devices generating continuous data streams. Managing such large-scale systems requires architectures capable of handling increasing workloads in data processing, storage, and communication.¹⁶

Latency and real-time processing are also critical concerns. Many IoT applications, such as industrial automation and autonomous systems, require immediate responses to sensor data. Centralized cloud-based solutions may introduce unacceptable delays, making edge computing and distributed processing essential.¹⁷

Security and privacy represent some of the most critical challenges in IoT systems. The rapid growth of interconnected devices and continuous generation of sensitive data create a large attack surface for adversaries.¹⁸ Various attacks such as malware, denial-of-service (DoS), and data breaches have been observed in real-world IoT deployments, often exploiting weak security mechanisms and resource-constrained devices.¹⁹ As highlighted in recent studies, many IoT solutions prioritize functionality over security, making them vulnerable to emerging cyber threats.²⁰

In addition, IoT devices typically operate under **resource constraints**, including limited processing power, memory, and energy supply. These limitations restrict the complexity of algorithms and communication protocols that can be implemented, requiring lightweight and efficient solutions.²¹

Data management is another major challenge, as IoT systems generate massive vol-

¹⁵Jiafu Wan et al. "A comprehensive study of internet of things". In: *IEEE access : practical innovations, open solutions* 4 (2016), pp. 363–378. DOI: 10.1109/ACCESS.2016.2523501.

¹⁶Andrea Zanella et al. "Internet of things for smart cities". In: *IEEE Internet of Things Journal* 1.1 (2014), pp. 22–32. DOI: 10.1109/JIOT.2014.2306328.

¹⁷Weisong et al. Shi. "Edge computing: Vision and challenges". In: *IEEE Internet of Things Journal* (2016).

¹⁸Rodrigo Roman, Javier Lopez, and Masahiro Mambo. "Mobile edge computing, fog et al.: a survey and analysis of security threats and challenges". In: *Future Generation Computer Systems* 78 (2018), pp. 680–698. DOI: 10.1016/j.future.2016.11.009.

¹⁹Parushi Malhotra et al. "Internet of things: Evolution, concerns and security challenges". In: *Sensors* 21.5 (2021), p. 1809. DOI: 10.3390/s21051809.

²⁰Malhotra et al., "Internet of things: Evolution, concerns and security challenges".

²¹Shi, "Edge computing: Vision and challenges".

umes of data. Efficient data collection, storage, and analysis are necessary to extract meaningful insights while avoiding system bottlenecks.²²

Finally, **reliability and fault tolerance** are crucial in distributed IoT environments. Devices and network connections may fail unpredictably, and maintaining system stability under such conditions remains a significant challenge.²³

These challenges highlight the need for scalable, flexible, and resilient software architectures, motivating the use of distributed and event-driven design approaches discussed in the following sections.

2.2 Software Architectures & Design Patterns

2.2.1 Software Architectures

Software architectures can be defined as "the fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution."²⁴ In brief, Software architecture refers to the fundamental structure of a software system, encompassing its components, the relationships between them, and the principles governing their design and evolution.²⁵ It provides a high-level abstraction that defines how a system is organized and how its parts interact to fulfill both functional and non-functional requirements.

Beyond structural organization, software architecture also involves a set of critical design decisions that shape the system. These include choices related to communication mechanisms, data management strategies, technology stacks, and deployment models. Such decisions have a direct impact on key **quality attributes**, including scalability, performance, security, maintainability, and reliability.

Unlike lower-level software design, which focuses on individual components and implementation details, software architecture addresses system-wide concerns and con-

²²[gubbi2013iot](#).

²³Wan et al., "A comprehensive study of internet of things".

²⁴"IEEE Recommended Practice for Architectural Description for Software-Intensive Systems". In: *IEEE Std 1471-2000* (Oct. 2000), pp. 1–30. DOI: 10.1109/IEEESTD.2000.91944. URL: <https://ieeexplore.ieee.org/document/875998> (visited on 02/14/2026).

²⁵Mark Richards and Neal Ford. *Fundamentals of software architecture: An engineering approach*. O'Reilly Media, 2020. ISBN: 978-1-4920-4345-4.

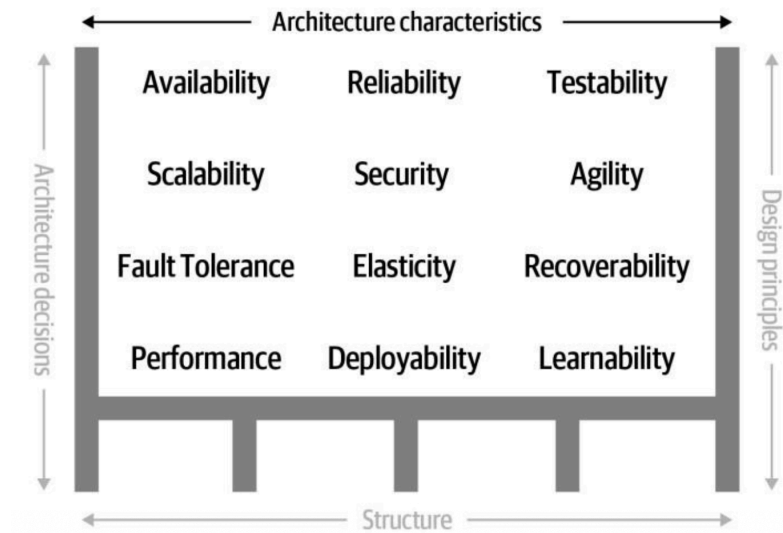


Figure 2.6: The architecture characteristics define the success criteria of a system

straints. It defines how different parts of the system collaborate, how responsibilities are distributed, and how trade-offs are managed. For instance, improving scalability may increase system complexity, while enhancing performance may reduce flexibility. In IoT Systems, where scalability and modularity are essential, deciding on a suitable software architecture is vital.

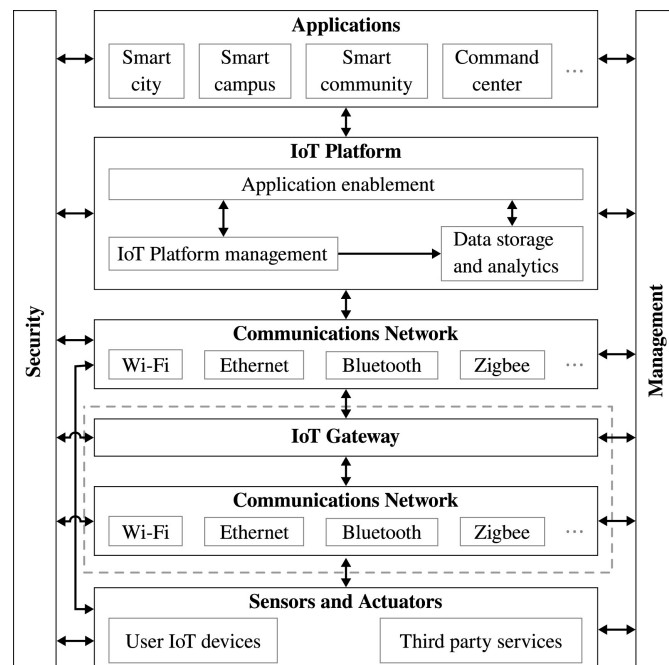


Figure 2.7: An Exemplary IoT System Architecture

In the context of IoT systems, architectural patterns are particularly important due to the distributed and heterogeneous nature of devices and services. Common architec-

tural patterns include **N-tier architecture**²⁶, which separates concerns into layers such as presentation, logic, and data; **Service-oriented architecture (SOA)**^{27,28}, which organizes functionality as loosely coupled services; **microservices architecture**, which decomposes applications into small, independently deployable services²⁹; and **Event-driven Architecture**³⁰, where system components communicate through asynchronous events based on a pre-defined protocol^{31,32}. These patterns enable IoT systems to handle scalability, real-time processing, and interoperability challenges.

2.2.2 Design Patterns

Design patterns, in contrast, address recurring problems at a lower level of abstraction, typically within individual components or modules. Another interpretation is "design patterns are descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context"³³. They provide reusable solutions to common software design problems, improving code maintainability, flexibility, and reusability.

Common design patterns relevant to IoT systems include the **publish-subscribe pattern**, widely used in messaging protocols such as MQTT for decoupled communication

²⁶Tiered versus tierless IoT stacks | *Proceedings of the 10th International Conference on the Internet of Things*. en. URL: <https://dl.acm.org/doi/10.1145/3410992.3411002> (visited on 02/14/2026).

²⁷Naghmeh Niknejad et al. "Understanding Service-Oriented Architecture (SOA): A systematic literature review and directions for further investigation". In: *Information Systems* 91 (July 2020), p. 101491. ISSN: 0306-4379. DOI: 10.1016/j.is.2020.101491. URL: <https://www.sciencedirect.com/science/article/pii/S0306437920300028> (visited on 02/14/2026).

²⁸Sugyan Kumar Mishra and Anirban Sarkar. "Service-oriented architecture for Internet of Things: A semantic approach". In: *Journal of King Saud University - Computer and Information Sciences* 34.10, Part A (Nov. 2022), pp. 8765–8776. ISSN: 1319-1578. DOI: 10.1016/j.jksuci.2021.09.024. URL: <https://www.sciencedirect.com/science/article/pii/S1319157821002755> (visited on 02/14/2026).

²⁹Ruiqi Ouyang et al. "A Microservice and Serverless Architecture for Secure IoT System". In: *Sensors (Basel, Switzerland)* 23.10 (May 2023), p. 4868. ISSN: 1424-8220. DOI: 10.3390/s23104868. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10220873/> (visited on 02/14/2026).

³⁰Use an event driven architecture in your IoT devices - *Internet of Things (IoT) Lens*. URL: <https://docs.aws.amazon.com/wellarchitected/latest/iot-lens/use-an-event-driven-architecture-in-your-iot-devices.html> (visited on 02/14/2026).

³¹Silvio Quincozes, Tubino Emilio, and Juliano Kazienko. "MQTT Protocol: Fundamentals, Tools and Future Directions". In: *IEEE Latin America Transactions* 17.09 (Sept. 2019), pp. 1439–1448. ISSN: 1548-0992. DOI: 10.1109/TLA.2019.8931137. URL: <https://ieeexplore.ieee.org/abstract/document/8931137> (visited on 02/14/2026).

³²OASIS. *MQTT version 3.1.1*. 2014.

³³Erich Gamma, ed. *Design patterns: elements of reusable object-oriented software*. Addison-Wesley professional computing series. Reading, Mass: Addison-Wesley, 1995. ISBN: 978-0-201-63361-0.

between devices and services;³⁴ and Input-Output” (I/O) design patterns, which categorize the data flow semantics between sensors, actuators, and the cloud to solve recurring problems in industrial process and business optimization.³⁵

Together, architectural and design patterns provide complementary approaches to managing the complexity of IoT systems, with architectural patterns addressing system-level organization and design patterns refining component-level behavior.

³⁴Apostolos Lazidis, Konstantinos Tsakos, and Euripides G. M. Petrakis. “Publish–Subscribe approaches for the IoT and the cloud: Functional and performance evaluation of open-source systems”. In: *Internet of Things* 19 (Aug. 2022), p. 100538. ISSN: 2542-6605. DOI: 10.1016/j.iot.2022.100538. URL: <https://www.sciencedirect.com/science/article/pii/S2542660522000403> (visited on 02/14/2026).

³⁵Gedare Bloom et al. “Design patterns for the industrial Internet of Things”. en. In: *2018 14th IEEE International Workshop on Factory Communication Systems (WFCS)*. Imperia, Italy: IEEE, June 2018, pp. 1–10. ISBN: 978-1-5386-1066-4. DOI: 10.1109/WFCS.2018.8402353. URL: <https://ieeexplore.ieee.org/document/8402353/> (visited on 02/14/2026).

Chapter 3

Essential Software Architectures

3.1 N-tier Architectures

N-tier architecture is one of the most fundamental architectural styles, in which a system is organized into multiple layers (tiers), each responsible for a specific functionality such as presentation, business logic, and data management.

N-tier architecture closely resembles layered architecture, as described in,¹ but extends it by considering not only logical separation but also physical deployment across multiple tiers.

Topology N-tier architectures follow a hierarchical topology in which components are organized into a sequence of layers, each interacting primarily with its adjacent layers. Requests typically propagate from upper layers, such as application services, down to lower layers responsible for data access or device interaction, and results are returned along the same path. This linear communication model simplifies system structure but can introduce inefficiencies in highly distributed environments.

Separation of Concerns A defining characteristic of N-tier architecture is the clear separation of concerns across layers. Each tier is responsible for a specific aspect of system functionality, such as data acquisition, communication, processing, or application logic. In IoT systems, this separation enables modular development and improves maintainability by isolating changes within individual layers.

Layers of Isolation Layers in an N-tier architecture act as boundaries that isolate system components from one another. This isolation reduces direct dependencies between different parts of the system, allowing changes within one layer to have minimal impact on others. However, this strict layering can also introduce rigidity, as interactions must follow predefined paths, limiting flexibility and potentially increasing latency in complex IoT scenarios.

These characteristics highlight that N-tier architecture emphasizes structural clarity and modularity, but at the cost of flexibility and efficiency in highly distributed systems such as IoT.

¹Richards and Ford, *Fundamentals of software architecture: An engineering approach*.

Moreover, this layered separation enables a clear structuring of system responsibilities and has been widely adopted in traditional software systems, even in more heterogeneous system. For example, in a manufacturing system, where the partitioning is based on the separation and modularization of logic mechanisms, in other words the business logic:²

- Tier 1 (Data): Represents the source of manufacturing data, specifically the physical devices
- Tier 2 (Aggregation): Contains business logic mechanisms to aggregate and contextualize different data streams relative to specific activities
- Tier 3 (Delivery): Contains presentation logic mechanisms where data is processed and cached for efficient delivery to clients.
- Tier 4 (Client): Represents the user interface providing access to volatile data streams, queryable repositories, and data packages.

In the context of IoT, similar layered thinking can be observed in reference models such as the IoT World Forum's 7-layer architecture, which divides IoT systems into tiers ranging from physical devices to data processing and application layers.³ While not strictly an N-tier implementation, this model reflects the same principle of hierarchical decomposition based on functional responsibilities.

Moreover, although reference models such as the IoT World Forum define a seven-layer architecture, in practice these layers are not rigid. System designers often adapt or merge layers depending on system requirements, resulting in different forms of tier partitioning.

As shown in Figure 3.1, some IoT systems adopt a simplified structure consisting of perception, connectivity, processing, service, and application layers.

²Moneer Helu, Thomas Hedberg, and Allison Barnard Feeney. "Reference architecture to integrate heterogeneous manufacturing systems for the digital thread". In: *CIRP journal of manufacturing science and technology* 19 (2017), 10.1016/j.cirpj.2017.04.002. ISSN: 1755-5817. DOI: 10.1016/j.cirpj.2017.04.002. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC6459417/> (visited on 02/14/2026).

³Stuart Corner. *Cisco IoT World Forum Reference Model for interoperability*. en-AU. July 2015. URL: <https://www.iotaustralia.org.au/2015/07/22/iotnewsglobal/cisco-iot-world-forum-reference-model-promises-interoperability/> (visited on 02/14/2026).

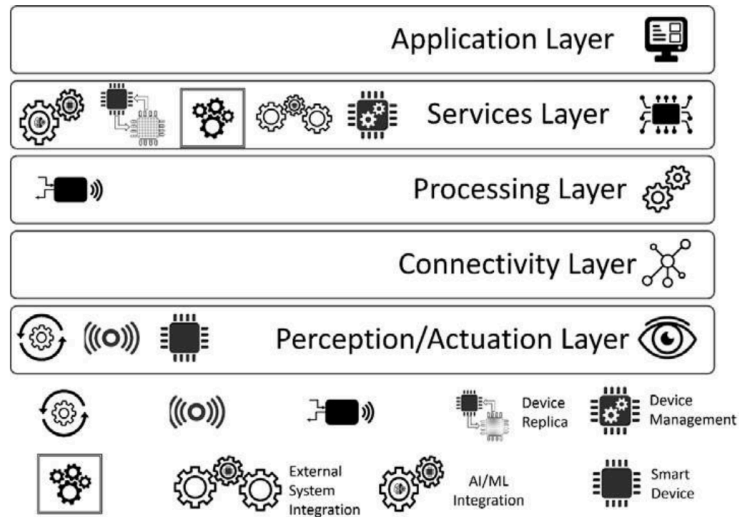


Figure 3.1: Alternate partitioning for Iot Systems

Another example, in edge-centric IoT systems, data processing may be integrated closer to the device layer to reduce latency, while in cloud-centric architectures, multiple layers may be consolidated into centralized backend services.

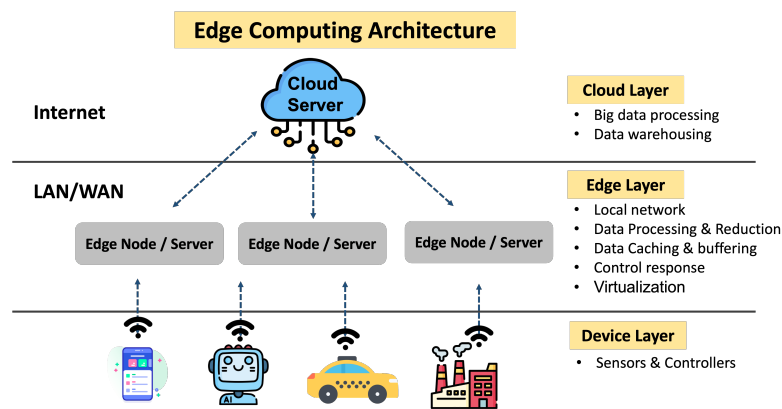


Figure : Edge computing architecture overview
Source : The research team

Figure 3.2: Example of Edge-Centric IoT System

Consequently, N-tier architecture should be understood as a flexible conceptual model for separating concerns, rather than a strict prescription of a fixed number of layers.

Advantages

N-tier architectures offer several advantages, primarily stemming from their clear structural organization. The separation of concerns across layers improves maintainability, as

changes within one layer can often be made with minimal impact on others. This modularity also facilitates team development, allowing different teams to work on distinct layers independently.

Additionally, the layered structure promotes reusability, as components within a given tier can be reused across multiple applications or system contexts. The architecture is also relatively simple and well-understood, making it easier to design, implement, and manage compared to more complex architectural styles.⁴

Finally, N-tier architectures provide a degree of scalability by enabling individual tiers to be deployed on separate infrastructure.⁵ However, this scalability is typically coarse-grained and does not match the flexibility required by highly distributed IoT systems.

Disadvantages

Despite its structural clarity, N-tier architecture exhibits several limitations when applied to IoT systems. One major drawback is its limited scalability. Scaling is typically performed at the level of entire tiers rather than individual functionalities, leading to inefficient resource utilization in large-scale deployments.

Another key issue is the **tight coupling** between layers. Although responsibilities are separated, layers still depend on one another through predefined interfaces, meaning that changes in one layer can propagate and affect others. This reduces flexibility and slows down system evolution.

Furthermore, the centralized nature of N-tier architectures introduces performance bottlenecks. In IoT environments, where large numbers of devices continuously generate data, routing all interactions through centralized processing layers can lead to increased latency and reduced system responsiveness.

Finally, the strictly hierarchical communication model restricts direct interaction between non-adjacent components. This can result in unnecessary data traversal across multiple layers, making the system less efficient compared to more flexible architectures such as microservices or event-driven models.

⁴claytonsiemens77. *N-tier Architecture Style - Azure Architecture Center*. en-us. URL: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier> (visited on 03/09/2026).

⁵Richards and Ford, *Fundamentals of software architecture: An engineering approach*.

These limitations motivate the adoption of **service-oriented architectures**, which decompose systems based on business capabilities rather than rigid layered structures.

3.2 Service-oriented Architectures

Service-Oriented Architecture is one of the most integral and popular architectural patterns for IoT systems. Instead of technical partitioning of system components into rigid layers, SOA decomposes the system into loosely coupled services aligned with business functionalities.

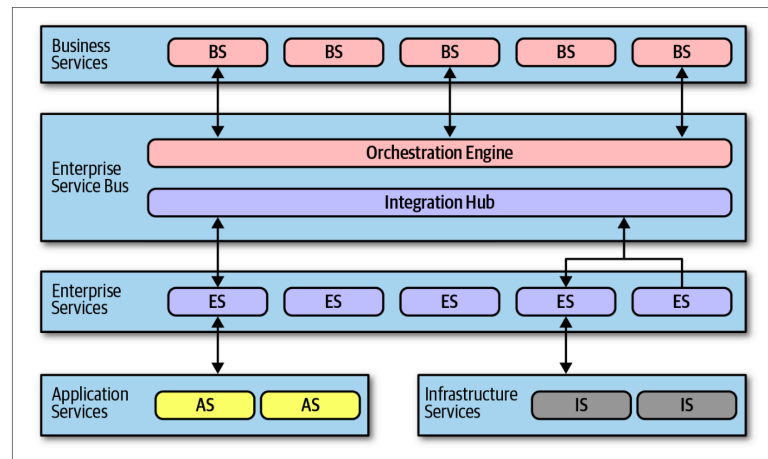


Figure 3.3: Service-Oriented Architecture's Topology

Unlike N-tier architectures, which organize systems into hierarchical layers based on technical responsibilities, Service-Oriented Architecture (SOA) partitions systems into a collection of loosely coupled services aligned with business and functional capabilities.⁶

As illustrated in 3.3, SOA introduces multiple categories of services, including application services, enterprise services, and business services.

Application services represent entry points into the system and are typically tailored to specific applications or user interactions. They handle tasks such as request validation, data formatting, and coordination with underlying services, but generally do not contain complex business logic.

Enterprise services provide reusable, shared functionalities that can be leveraged

⁶Richards and Ford, *Fundamentals of software architecture: An engineering approach*.

across multiple applications. These services encapsulate common capabilities such as data access, authentication, or device management in IoT systems, promoting consistency and reducing duplication across the system.

Business services operate at a higher level of abstraction and implement core business logic. They often compose multiple enterprise services to realize complete business processes, making them central to achieving functional requirements and enabling reuse across different workflows.

These services interact through an integration layer, commonly implemented as an Enterprise Service Bus (ESB). The ESB acts as a middleware backbone that decouples service providers and consumers by handling communication concerns such as message routing, protocol translation, and data transformation. This enables interoperability across heterogeneous systems and allows services to evolve independently.

A key component in this architecture is the orchestration engine, which coordinates multiple services to implement complex workflows. It defines the sequence and logic of service interactions, enabling the composition of higher-level functionalities from smaller, reusable services.

Rather than enforcing strict hierarchical communication as in N-tier architectures, SOA allows more flexible interaction patterns, where services can be composed and reused across different business processes. This form of partitioning shifts the focus from technical layering to service-based decomposition, improving modularity and interoperability. However, it also introduces additional complexity through middleware components such as the ESB, which may become a central point of control and potential bottleneck.

Advantages Service-Oriented Architecture provides strong modularity by decomposing systems into loosely coupled services aligned with business capabilities⁷. This enables high reusability, as services can be shared across multiple applications and workflows. Additionally, SOA improves interoperability through standardized communication and middleware support, making it well-suited for integrating heterogeneous components in IoT systems.

⁷What is Service-Oriented Architecture (SOA)? | IBM. en. Oct. 2021. URL: <https://www.ibm.com/think/topics/soa> (visited on 03/09/2026).

Disadvantages Despite its benefits, SOA introduces significant complexity due to the reliance on middleware such as the Enterprise Service Bus. The ESB can become a central bottleneck and single point of failure, limiting scalability and performance. Furthermore, increased network communication between services may lead to higher latency, which can be problematic in real-time IoT applications.

To better address these challenges, more advanced variations of service-oriented architectures tailored for IoT systems will be analyzed in the following chapter.

3.3 Microservice Architectures

Microservice Architecture is an architectural style that structures an application as a collection of small, autonomous services, each responsible for a specific business capability. These services operate independently, communicate through lightweight interfaces, and can be developed, deployed, and scaled without impacting other components of the system. Unlike monolithic architectures, where all functionalities are tightly integrated into a single unit, microservices promote modularity, flexibility, and continuous evolution of the system.

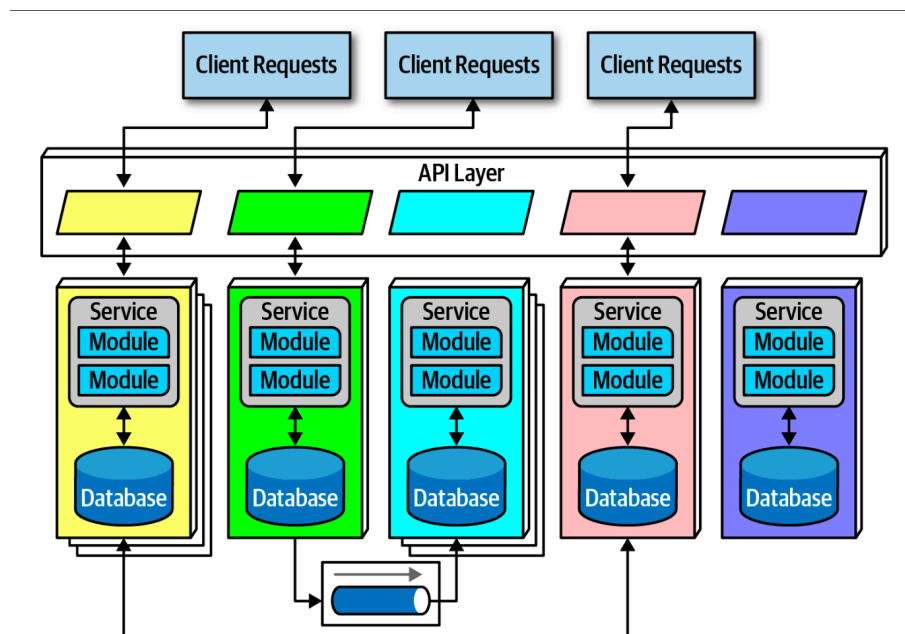


Figure 3.4: Topology of Microservice

As illustrated in figure 3.4, due to its single-purpose nature, the service size in mi-

crosservices is much smaller than other distributed architectures, such as the orchestration-driven service-oriented architecture. Architects expect each service to include all necessary parts to operate independently, including databases and other dependent components. The different characteristics appear in the following sections.⁸

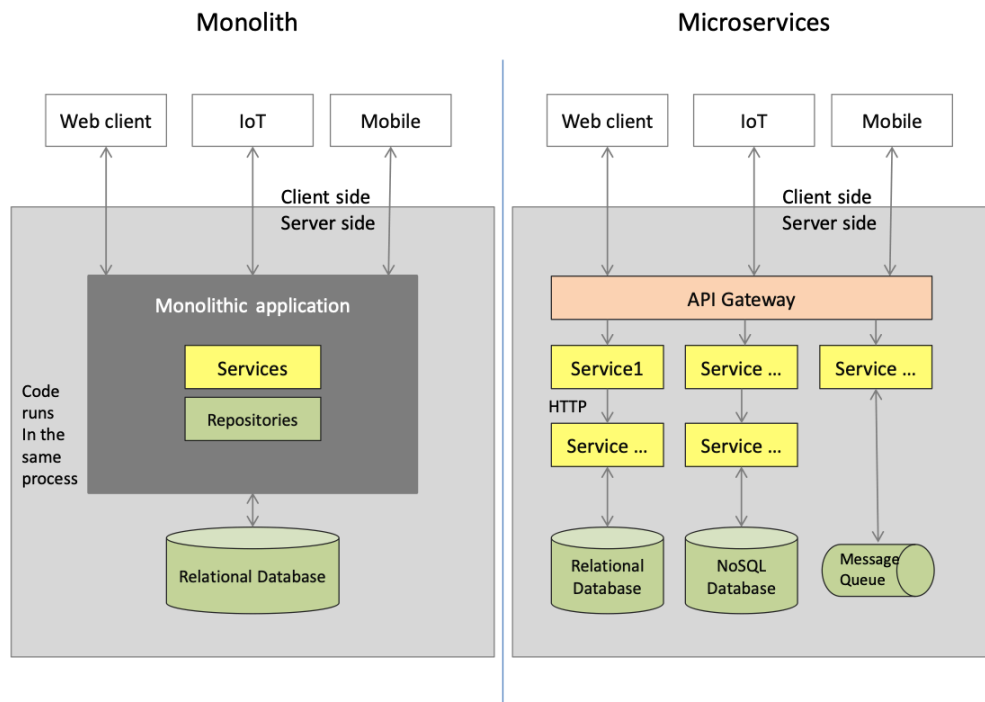


Figure 3.5: Monolithic vs Microservice

There are various aspects to Microservice that Architects should take notes

Distributed

Microservices represent a **distributed architectural style** in which each service operates within its own process. While this originally implied deployment on separate physical machines, it has evolved with the adoption of virtual machines and container technologies⁹ (Docker, Kubernetes). This level of decoupling addresses common challenges found in traditional multitenant infrastructures, where multiple applications share the same resources such as memory, storage, and network bandwidth. Although resource

⁸Richards and Ford, *Fundamentals of software architecture: An engineering approach*.

⁹Prakhar Srivastav. *A Docker Tutorial for Beginners*. en-us. URL: <https://docker-curriculum.com/> (visited on 03/11/2026).

sharing can improve efficiency, it often leads to contention as applications scale, as well as insufficient isolation between them.

By assigning each service to its own process, microservice architectures effectively eliminate many of the issues associated with shared environments.¹⁰ Historically, providing dedicated infrastructure for each application domain was impractical due to cost and operational complexity. However, advancements in cloud computing and automated provisioning have made it feasible to achieve strong isolation at both the domain and operational levels.

Despite these advantages, the distributed nature of microservices introduces performance considerations. Communication between services relies on network calls, which are significantly slower than in-process method invocations. Additionally, enforcing security at each service boundary adds further overhead. As a result, system architects must carefully balance service granularity to avoid excessive communication costs.

Furthermore, due to the complexity of coordinating distributed components, the use of transactions across service boundaries is generally discouraged. This makes defining appropriate service boundaries and granularity a critical factor in the successful design of microservice-based systems.

Bounded Context

A fundamental principle of microservice architecture is the concept of **bounded context**, originating from domain-driven design (DDD). A bounded context defines a clear boundary within which a specific domain model is applicable and consistent. In microservices, each service is designed to represent a distinct domain or subdomain, encapsulating all the necessary components required for its operation, including business logic, data models, and database schemas.

This approach encourages strong autonomy and minimizes dependencies between services. Unlike monolithic architectures, where shared components such as common data classes are reused across multiple modules, microservices favor loose coupling even at the cost of duplication. For instance, similar data structures (e.g., address information)

¹⁰Chris Richardson. *Microservices patterns: with examples in Java*. Shelter Island, New York: Manning Publications, 2019. ISBN: 978-1-61729-454-9.

may be defined independently within different services to avoid tight interdependencies.

By strictly adhering to bounded contexts, microservices extend the idea of domain partitioning to a physical level. Each service becomes an independent unit that directly reflects a specific business capability, effectively translating the conceptual boundaries defined in domain-driven design into deployable system components. This alignment enhances modularity, maintainability, and scalability in complex distributed systems such as IoT applications.

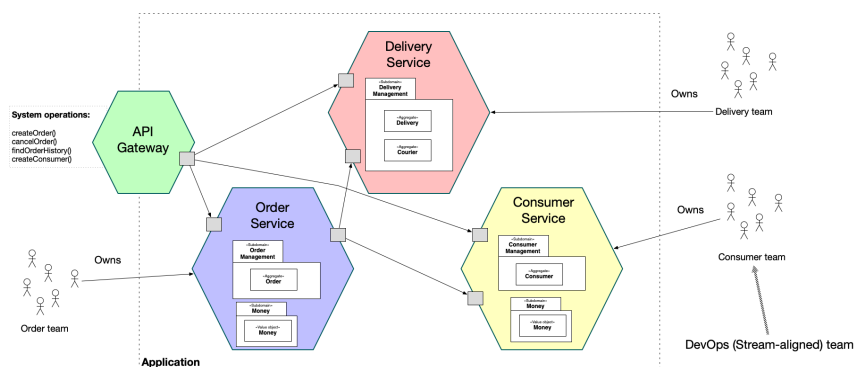


Figure 3.6: Example Microservice-based System

Service Granularity

Determining the appropriate granularity of services is a key challenge in microservice architecture. Services that are too fine-grained can lead to excessive communication overhead, while overly coarse-grained services reduce flexibility. Importantly, the term “microservices” is descriptive rather than prescriptive, and does not imply that services must be extremely small.

Service boundaries should align with business domains or workflows, ensuring high functional cohesion. Each service should encapsulate a meaningful capability within the system, based on natural domain boundaries.

Several guidelines help in defining appropriate granularity. First, services should have a clear purpose and represent a cohesive function. Second, transactional requirements can indicate suitable boundaries, as minimizing cross-service transactions improves system reliability. Third, if services require excessive communication to perform tasks, they may be too fine-grained and should be consolidated.¹¹

¹¹Richards and Ford, *Fundamentals of software architecture: An engineering approach*.

Ultimately, service granularity is refined through iteration, as architects adjust boundaries based on system behavior and communication patterns.

Data Isolation

Data isolation is a key principle of microservice architecture, where each service manages its own data and database schema to avoid tight coupling. Unlike traditional systems with a shared database, this approach enforces clear boundaries between services.

Architects must ensure that services are defined by business capabilities rather than individual database entities, avoiding overly fine-grained designs. Since data is distributed, maintaining consistency requires strategies such as defining a single source of truth or using replication and caching.

While data isolation increases complexity, it provides flexibility, allowing each service to choose and evolve its data storage independently.

In the context of Internet of Things (IoT) systems, microservice architecture provides a suitable foundation for handling large-scale, distributed, and heterogeneous environments. IoT systems typically involve numerous devices, diverse communication protocols, and continuous data streams, all of which require scalable and adaptable system design. By decomposing the system into loosely coupled services, microservices enable efficient management of complexity while supporting independent updates, fault isolation, and rapid integration of new functionalities.

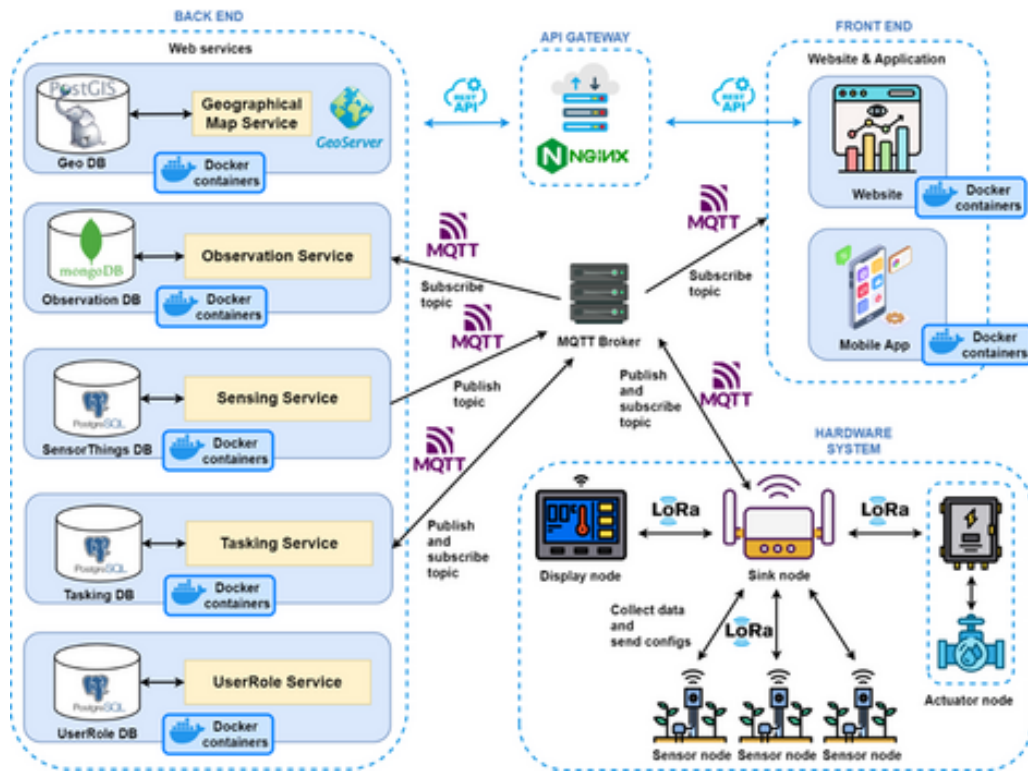


Figure 3.7: Example IOT System

Advantages

Microservice architecture offers several benefits for complex and distributed systems such as IoT. Its modular structure enables independent development, deployment, and scaling of services, improving flexibility and efficient resource utilization.¹² Fault isolation enhances system resilience, as failures in one service do not necessarily affect others. Additionally, microservices support technological heterogeneity, allowing each service to use the most suitable tools and frameworks.

Disadvantages

Despite its advantages, microservice architecture introduces significant challenges. The distributed nature of the system increases complexity, requiring mechanisms for service discovery, communication, and monitoring. Network-based communication can introduce latency compared to in-process calls, and maintaining data consistency across services becomes more difficult due to data isolation. Furthermore, managing security and

¹²Challenges and benefits of the microservice architectural style. en. URL: <https://developer.ibm.com/articles/challenges-and-benefits-of-the-microservice-architectural-style-part-1/> (visited on 03/11/2026).

coordinating multiple services can lead to higher operational overhead.

3.4 Event-driven Architectures

Event-Driven Architecture (EDA) is a "event-based"¹³ software architectural pattern in which system components communicate through the production, detection, and reaction to **events**.¹⁴ An event represents a significant change in **state**, such as a sensor reading exceeding a threshold, a device going offline, or a user issuing a command. Instead of relying on direct, synchronous communication between components, EDA enables **asynchronous** interactions through event streams.

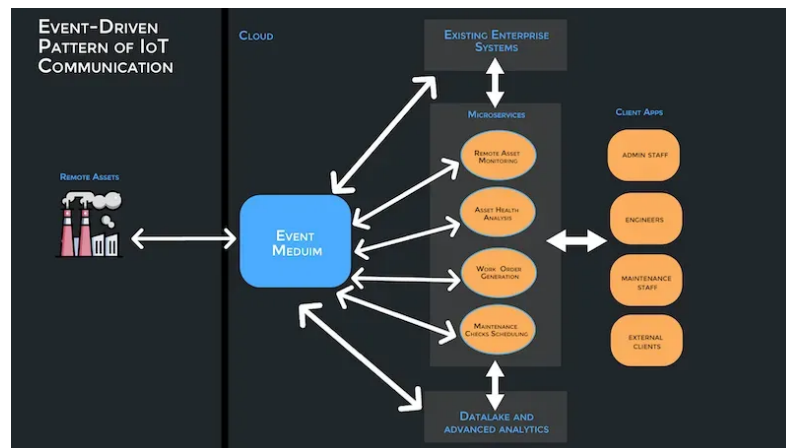


Figure 3.8: High-level Overview of Event-driven Architecture for IOT Systems

In IoT systems, EDA is particularly well-suited due to the inherently distributed and dynamic nature of connected devices. Sensors and devices continuously generate streams of data, which can be modeled as events and transmitted to other components such as processing services, storage systems, or user applications.¹⁵ This decouples producers (devices) from consumers (services), allowing each component to operate independently and scale efficiently.

Event-Driven Architecture can be implemented using two primary topologies: *broker*

¹³Richards and Ford, *Fundamentals of software architecture: An engineering approach*.

¹⁴*What Is Event-Driven Architecture?* | IBM. en. Oct. 2021. URL: <https://www.ibm.com/think/topics/event-driven-architecture> (visited on 03/09/2026).

¹⁵*Building event-driven architectures with IoT sensor data* | AWS Architecture Blog. Section: Amazon Kinesis. Nov. 2022. URL: <https://aws.amazon.com/blogs/architecture/building-event-driven-architectures-with-iot-sensor-data/> (visited on 03/02/2026).

topology and *mediator topology*. These topologies differ in how events are routed and how control flow is managed across the system.

Broker Topology In the broker topology, there is no central coordinator controlling the event flow.¹⁶ Instead, event producers publish events to an event broker (or message broker), and multiple consumers subscribe to the events they are interested in.¹⁷ Each consumer independently reacts to events and may produce new events as a result.

This topology promotes a high degree of decoupling between components, as producers are unaware of who consumes their events, and consumers do not need to know the source of events. The broker (e.g., MQTT broker or Apache Kafka) is responsible only for routing events based on topics or subscriptions.

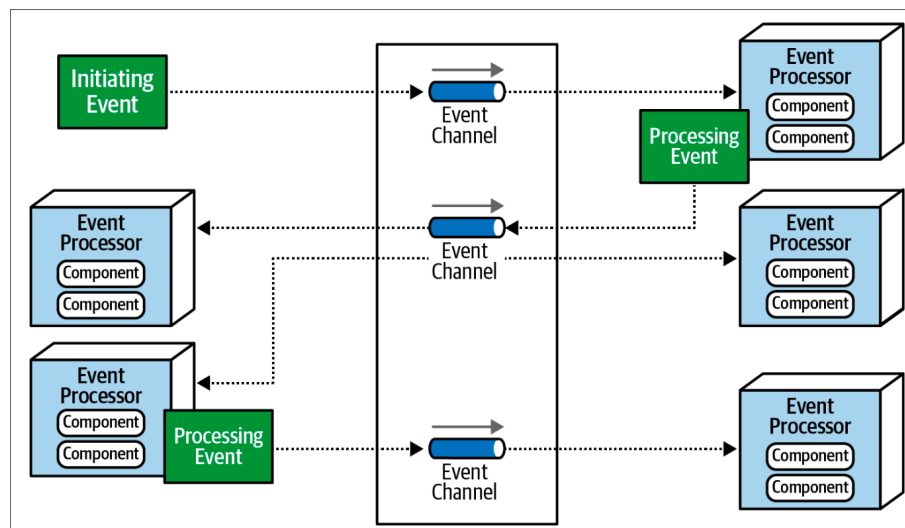


Figure 3.9: Broker Topology

In IoT systems, broker topology is widely used for real-time data streaming. For example, a temperature sensor publishes readings to a broker, and multiple services—such as monitoring dashboards, alerting systems, and data storage—can independently process the same event stream.

The main advantages of broker topology include **scalability**, **flexibility**, and **simplicity** in adding new consumers. However, it can be harder to enforce complex workflows

¹⁶EVENT-DRIVEN ARCHITECTURE BROKER TOPOLOGY | GUARDIAN - An Opinionated Inductive Guidance Framework for Digital Transformation. URL: <https://guardian-framework.github.io/framework/event-driven-architecture-broker-topology.html> (visited on 03/02/2026).

¹⁷Richards and Ford, *Fundamentals of software architecture: An engineering approach*.

or coordination between multiple processing steps, since there is no centralized control.

Mediator Topology In contrast, the mediator topology introduces a central component known as the mediator, which orchestrates the processing of events. Instead of multiple independent consumers reacting freely, the mediator receives events and determines the sequence of actions or services that should be invoked.

The mediator acts as a controller that manages workflow logic, making it suitable for scenarios where event processing requires coordination, decision-making, or multi-step transactions. Components communicate through the mediator rather than directly with each other.

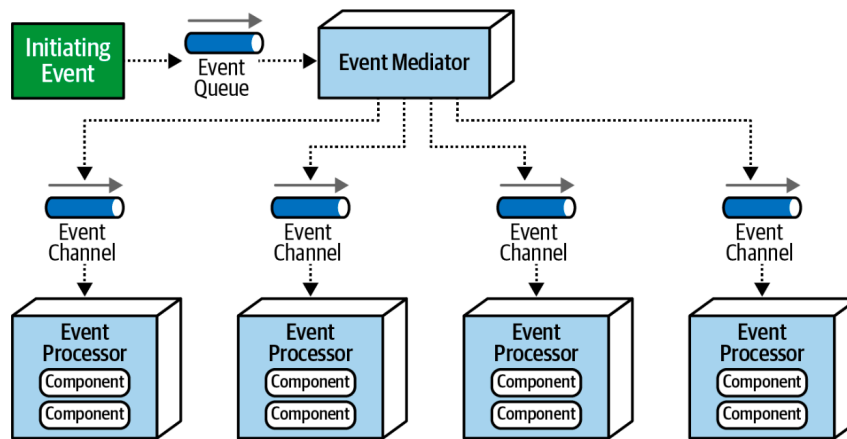


Figure 3.10: Mediator Topology

In IoT systems, mediator topology can be used in applications that require structured workflows. For example, in an industrial IoT system, an event indicating a machine anomaly may trigger a sequence of actions: logging the event, notifying operators, and initiating automated maintenance procedures. The mediator ensures that these steps occur in the correct order.

The advantage of this topology is improved control and manageability of complex processes. However, it introduces tighter coupling to the mediator and may become a bottleneck or single point of failure if not designed carefully.

Comparison In summary, broker topology emphasizes decentralized event processing and loose coupling, making it ideal for scalable and real-time IoT applications. Mediator topology, on the other hand, provides centralized control and is better suited for orches-

trating complex workflows. In practice, many IoT systems adopt a hybrid approach, combining both topologies to balance flexibility and control.

Core Components of Event-Driven Architecture

An Event-Driven Architecture is composed of several key components that enable the production, transmission, and processing of events.

Event Producers: These are sources of events, typically IoT devices such as sensors and actuators. They generate events when a change in state occurs, such as detecting motion or measuring temperature.

Event Broker: The broker is responsible for receiving events from producers and distributing them to interested consumers. It supports asynchronous communication and often implements publish-subscribe mechanisms. Common technologies include MQTT brokers and Apache Kafka.

Event Consumers: Consumers are services or applications that subscribe to events and react accordingly. These may include data processing systems, storage services, or user-facing applications such as dashboards.

Event Channels: These define how events are categorized and transmitted, often using topics or streams. They enable selective consumption of events by different components.

Event Processing: This includes filtering, aggregation, and transformation of events, often performed in real time to derive meaningful insights from raw IoT data.

Advantages and Challenges

Advantages:¹⁸

- *Scalability:* Components can scale independently to handle large volumes of IoT data.
- *Loose Coupling:* Producers and consumers operate independently, improving flexibility.
- *Real-Time Processing:* Enables immediate reaction to events.

¹⁸Richards and Ford, *Fundamentals of software architecture: An engineering approach*.

- *Resilience*: Failures in one component do not necessarily affect others.

Challenges:

- *Complexity*: Distributed event flows can be difficult to design and debug.
- *Event Management*: Requires careful design of event schemas and versioning.
- *Consistency*: Ensuring data consistency across distributed components is challenging.

Chapter 4

Analysis of Architectures

4.1 Service-oriented Architectures for IoT

4.1.1 Service Granularity

A fundamental design consideration in Service-Oriented Architecture (SOA) is the granularity of services, which refers to the scope and size of functionality encapsulated within each service. Service granularity typically ranges from coarse-grained services, which encapsulate broad business functionalities, to fine-grained services, which provide more specialized and narrowly defined operations.

In traditional SOA systems, services are often designed to be relatively coarse-grained in order to reduce communication overhead and improve performance. This design choice is particularly relevant in distributed environments, where excessive service calls can lead to increased latency and network congestion. Coarse-grained services also simplify orchestration by reducing the number of interactions required to complete a given workflow.

However, in IoT systems, the appropriate level of service granularity becomes more complex due to the highly dynamic and heterogeneous nature of the environment. Fine-grained services offer greater flexibility and reusability, allowing system functionalities to be composed dynamically based on changing conditions. For instance, individual sensing, monitoring, and actuation capabilities can be exposed as independent services

and combined to support diverse IoT applications.¹ This composability is essential in large-scale systems where functionalities must be adapted or extended without redeploying the entire system.

Despite these advantages, fine-grained services introduce significant challenges. Increased service decomposition leads to a higher number of interactions between services, which can negatively impact system performance, especially in resource-constrained IoT environments. As observed in service-oriented frameworks for cyber-physical systems, which share many characteristics with IoT environments, the overhead associated with service discovery, inter-service communication, and coordination can become significant as services become increasingly fine-grained.²

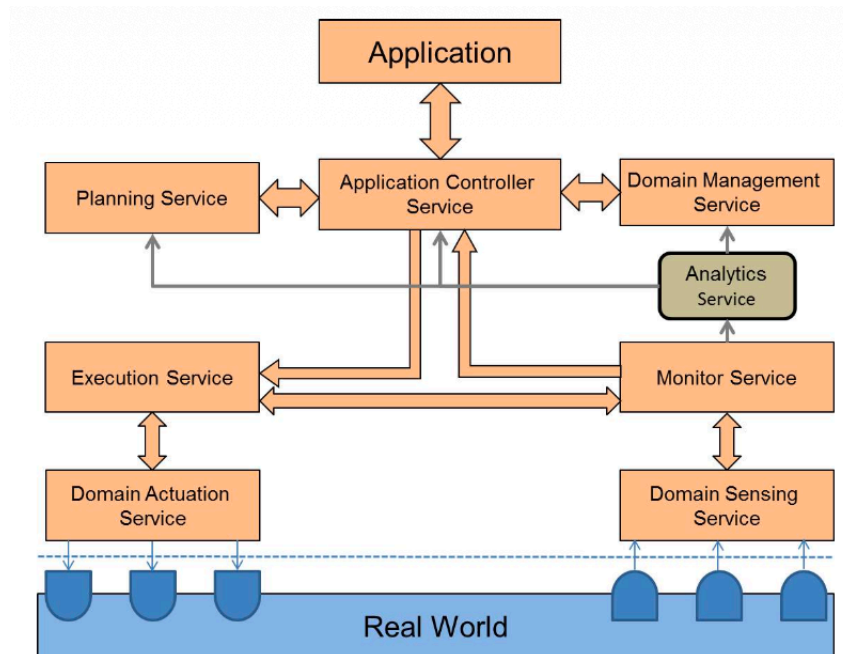


Figure 4.1: Overview of a CPS system

A notable feature of this architecture is the central role of the Application Controller and Execution services, which coordinate interactions across multiple services to achieve system goals. While this orchestration-based approach improves flexibility and supports

¹Sofie Van Hoecke et al. "SAMuS: Service-Oriented Architecture for Multisensor Surveillance in Smart Homes". en. In: *The Scientific World Journal* 2014 (2014), pp. 1–9. ISSN: 2356-6140, 1537-744X. DOI: 10 . 1155 / 2014 / 150696. URL: <http://www.hindawi.com/journals/tswj/2014/150696/> (visited on 03/09/2026).

²SOA-PE: A Service-oriented Architecture for Planning and Execution in Cyber-physical Systems. en. URL: <https://www.ericsson.com/en/reports-and-papers/research-papers/soa-pe-a-service-oriented-architecture-for-planning-and-execution-in-cyber-physical-systems> (visited on 03/09/2026).

dynamic adaptation, it also introduces significant coordination complexity. In particular, continuous interactions between monitoring, execution, and actuation services create tightly coupled runtime dependencies, which may lead to increased communication overhead and latency in large-scale or real-time CPS environments.

Furthermore, the architecture reflects a feedback-driven control loop, where sensing, monitoring, and analytics continuously influence planning and execution. Although this design is well-suited for dynamic and autonomous systems, it reinforces the reliance on centralized orchestration mechanisms, potentially limiting scalability and fault isolation. These challenges highlight important limitations of traditional SOA when applied to highly distributed IoT and CPS systems. Moreover, the proliferation of fine-grained services introduces complex interdependencies, making orchestration more challenging and increasing overall system complexity.

Conversely, coarse-grained services reduce communication overhead and simplify system management, but at the cost of reduced flexibility and reusability. In rapidly evolving IoT scenarios, where new devices and functionalities are continuously integrated, overly coarse services may hinder adaptability and limit the ability to reuse existing components across different applications.

Therefore, determining an appropriate level of service granularity is a critical trade-off in SOA-based IoT systems. Designers must balance the need for flexibility and composability with the constraints of performance and system complexity. This trade-off has motivated the evolution of more fine-grained architectural styles, such as microservices, which aim to further refine service decomposition while addressing the limitations of traditional SOA.

4.1.2 Communications

Communication between services is a fundamental aspect of Service-Oriented Architecture, directly influencing system performance, scalability, and coupling.

In traditional SOA implementations, communication is primarily based on synchronous request-response interactions (like HTTP, RESTful...),³ where services invoke one another through well-defined interfaces. However, synchronous communication intro-

³What is Service-Oriented Architecture (SOA)?.

duces several limitations. Since services depend on immediate responses, failures or delays in one service can propagate through the system, reducing overall reliability. Furthermore, blocking communication increases latency and limits scalability, as system throughput becomes constrained by the slowest service in the interaction chain. In the context of IoT and CPS systems, these limitations become more pronounced. As illustrated in service-oriented CPS frameworks such as SOA-PE, continuous interactions between services—such as execution, monitoring, and actuation—require frequent synchronous communication. This leads to increased coordination overhead and can negatively impact system responsiveness in real-time environments.

To address the limitations of synchronous interaction, SOA systems increasingly incorporate **asynchronous communication models**. In this approach, services communicate through message passing mechanisms, such as message queues or publish-subscribe systems, allowing services to operate independently without blocking.

Asynchronous communication reduces temporal coupling between services, enabling higher scalability and improved fault tolerance. Services can continue operating even if other components are temporarily unavailable, making this model particularly suitable for distributed and dynamic environments.

In IoT systems, asynchronous communication is especially relevant due to the large number of devices and the need to handle intermittent connectivity.

Despite its advantages, asynchronous communication introduces additional complexity in system design. Managing message delivery, ordering, and consistency becomes more challenging, and debugging distributed interactions can be more difficult compared to synchronous models.

4.1.3 Service Composition and Orchestration

Service composition is a fundamental principle of Service-Oriented Architecture (SOA), enabling complex applications to be constructed by combining multiple independent services. Rather than implementing monolithic systems, SOA promotes the reuse of modular services that can be dynamically integrated to fulfill specific business or system requirements. This approach is particularly important in IoT environments, where applications must coordinate heterogeneous components such as sensors, data processors,

and actuators.

Two primary approaches to service coordination are commonly identified: orchestration and choreography. In orchestration, a central controller manages the interactions between services, defining the execution order and logic of the overall workflow. This model provides a high degree of control and predictability, making it suitable for structured processes. In contrast, choreography relies on decentralized interactions, where each service reacts to events and communicates with others without a central coordinator, enabling more flexible and scalable systems.

The SOA-PE architecture exemplifies an orchestration-based approach, where the Application Controller Service centrally coordinates the interactions between planning, execution, monitoring, and domain services. This centralized control enables a structured feedback loop that is essential for cyber-physical systems (CPS), ensuring that sensing, analysis, and actuation processes are tightly integrated. However, this design also introduces potential limitations. The reliance on a central orchestrator can create bottlenecks and increase communication overhead, particularly in large-scale IoT environments where numerous services must interact continuously.

In contrast, the Enablers Framework⁴ adopts a more flexible approach to service composition by introducing a middleware layer that facilitates the integration of heterogeneous services through unified interfaces. This framework allows applications to dynamically discover, access, and combine services without requiring tight coupling between components. By abstracting the underlying communication protocols and enabling interoperability, the framework supports more scalable and adaptable service compositions. Additionally, the use of message queues and distributed components allows for more decoupled interactions between services, reducing dependency on centralized coordination.

The comparison between these approaches highlights a key trade-off in SOA-based IoT systems. Orchestration provides strong control and is well-suited for deterministic workflows, as seen in CPS architectures such as SOA-PE. However, it may limit scalability and flexibility due to centralized coordination. On the other hand, more decentralized

⁴Joao Giao et al. "A Framework for Service-Oriented Architecture (SOA)-Based IoT Application Development". en. In: *Processes* 10.9 (Sept. 2022), p. 1782. ISSN: 2227-9717. DOI: 10.3390/pr10091782. URL: <https://www.mdpi.com/2227-9717/10/9/1782> (visited on 03/09/2026).

composition approaches, supported by middleware frameworks, improve scalability and resilience but introduce additional complexity in managing distributed interactions.

Consequently, modern IoT architectures often adopt hybrid approaches that combine orchestration for high-level control with more loosely coupled service interactions at lower levels. This allows systems to maintain structured workflows while benefiting from the flexibility and scalability required in dynamic and distributed environments.

4.1.4 Scalability and Integration

Interoperability and Integration

One of the primary motivations for adopting Service-Oriented Architecture (SOA) in IoT systems is to address the inherent heterogeneity of devices, communication protocols, and platforms. IoT environments consist of diverse components that must collaborate to provide composite services, making interoperability a fundamental requirement. In this context, SOA enables the exposure of functionalities as services with standardized interfaces, allowing different components to communicate and interact despite differences in implementation.

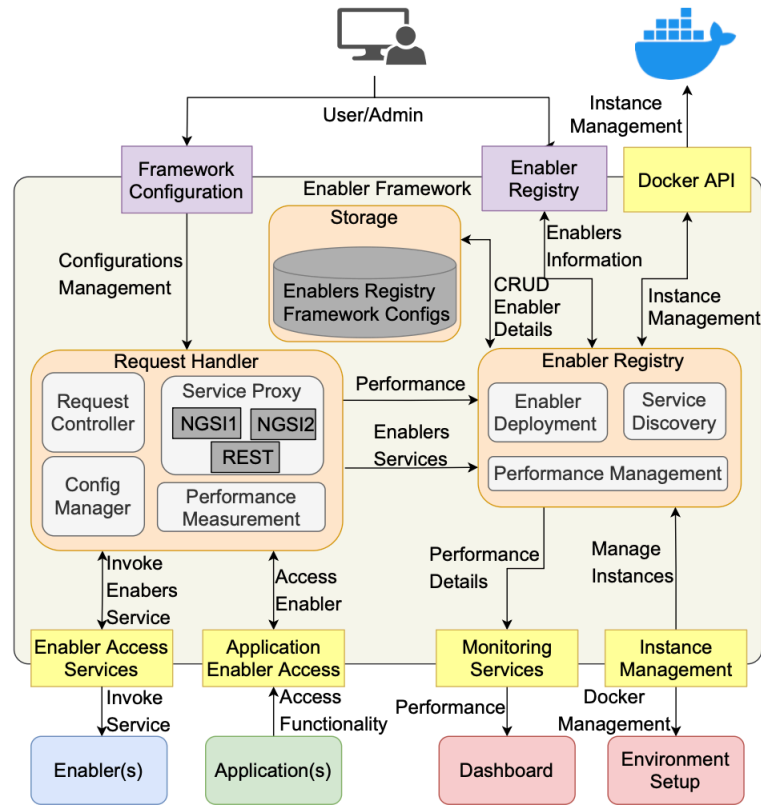


Figure 4.2: The Enabler Framework for SOA

Middleware plays a central role in achieving this integration. As highlighted in SOA-based IoT frameworks, middleware layers abstract the underlying complexity of heterogeneous systems, allowing developers to focus on application logic rather than low-level device-specific details. For instance, the Enablers Framework⁵ introduces a unified interface that facilitates the integration of diverse software modules and communication protocols, enabling applications to dynamically discover and interact with services across the system. This abstraction significantly reduces integration effort and supports reuse of services across multiple applications.

As illustrated in Figure 4.2, the Enablers Framework adopts a middleware-centric design that abstracts the complexity of heterogeneous IoT systems through standardized service interfaces such as REST and NGSI. The framework provides key functionalities including service discovery, deployment, and performance monitoring, enabling dynamic integration of distributed components. In particular, the presence of components such as the Enabler Registry and Service Proxy highlights the importance of centralized coordination for managing service interactions. While this design facilitates interoper-

⁵Giao et al., “A Framework for Service-Oriented Architecture (SOA)-Based IoT Application Development”.

ability and simplifies integration, it also introduces potential scalability challenges, as core components may become bottlenecks under high load. Furthermore, the use of container-based deployment mechanisms, such as Docker, reflects the need for flexible and scalable service management in modern IoT environments.

Scalability Challenges in SOA-based IoT Systems

Despite its strengths in integration, SOA introduces several scalability challenges when applied to large-scale IoT systems. A key issue arises from the reliance on centralized components, such as middleware platforms or orchestration engines, which can become bottlenecks as the number of services and interactions increases. In architectures such as SOA-PE,⁶ where a central controller coordinates planning, execution, and monitoring processes, continuous service interactions lead to significant communication overhead and increased system latency.

Furthermore, as services become more fine-grained, the number of inter-service communications grows rapidly, increasing network traffic and processing overhead. This is particularly problematic in IoT environments, where devices are often resource-constrained and operate under strict latency requirements. While SOA enables functional scalability by allowing new services to be added easily, it does not always guarantee operational scalability, as system performance may degrade with increasing service interactions.

The distributed nature of IoT systems further complicates scalability. Devices are often geographically dispersed and may experience intermittent connectivity, requiring systems to handle asynchronous communication and partial failures. Although middleware solutions can help mitigate some of these issues by managing communication and service interactions, they may also introduce additional overhead and complexity.

Security and Trust Considerations

In addition to scalability and integration, security and trust represent critical challenges in SOA-based IoT systems. The dynamic and distributed nature of IoT environments increases the attack surface, as services interact across multiple devices, platforms, and

⁶SOA-PE.

administrative domains. Ensuring secure communication and controlled access to services is therefore essential.

Traditional SOA implementations often treat security as a separate architectural layer. However, this approach is insufficient in IoT contexts, where security mechanisms must be integrated throughout the entire architecture. As highlighted in secure SOA-based IoT models, security policies, authentication mechanisms, and access control should be enforced at multiple levels, including device, service, and middleware layers, to ensure that only authorized entities can access system resources.

Moreover, trust management becomes a key requirement in distributed IoT systems. Applications must be able to verify the trustworthiness of both the middleware and the devices they interact with before relying on their data or functionality. This involves mechanisms such as device identification, attestation, and policy-based enforcement, which ensure that services behave as expected and are not compromised.⁷

These additional requirements introduce further complexity and overhead, as security checks and trust verification must be continuously performed across the system. Consequently, scalability in SOA-based IoT systems is not solely a matter of performance, but also of maintaining secure and trustworthy interactions across a large and dynamic service ecosystem.

Overall, while SOA provides strong support for interoperability and modular integration, its scalability is constrained by communication overhead, centralized coordination, and the added complexity of security and trust management. These limitations motivate the exploration of more lightweight and decentralized architectural approaches, which are discussed in the following section.

4.2 Microservice Architectures

Microservice architecture has gained significant attention as an effective approach for managing the complexity and scalability demands of modern IoT systems. By decomposing applications into loosely coupled, independently deployable services, it enables

⁷Ahmad-Atamli Reineh, Andrew J. Paverd, and Andrew P. Martin. *Trustworthy and Secure Service-Oriented Architecture for the Internet of Things*. arXiv:1606.01671 [cs]. June 2016. DOI: 10.48550/arXiv.1606.01671. URL: <http://arxiv.org/abs/1606.01671> (visited on 03/09/2026).

a modular structure that aligns naturally with the distributed and heterogeneous characteristics of IoT environments, like Agriculture and Aqua system⁸

One of the most significant advantages of microservices in IoT is adaptability. IoT environments are highly dynamic, with evolving device ecosystems and changing application requirements. Microservices allow individual components to be modified or replaced independently, enabling continuous system evolution and flexible integration of new functionalities.⁹

Scalability is another critical benefit. As IoT deployments grow to thousands or even millions of devices, systems must handle increasing data volumes efficiently. Microservices support horizontal scaling, allowing specific services such as data ingestion and analytics to scale independently based on workload demands, improving both performance and resource utilization.

Furthermore, microservices are well-suited for integration across edge and cloud environments. As IoT systems increasingly adopt edge computing to reduce latency and bandwidth usage, microservices can be distributed across the edge–cloud continuum. This enables real-time data processing closer to the data source while leveraging cloud resources for more complex tasks.¹⁰

From a data processing perspective, microservices enable the decomposition of complex analytics pipelines into smaller, specialized services. This is particularly important in IoT systems, where data is generated as continuous streams that require real-time processing. By structuring the system as a set of independent processing units, microservices allow each service to handle specific stream processing tasks such as filtering, transformation, aggregation, or event detection.¹¹

⁸Thai Minh Truong et al. “The Microservice Based Architecture for IoT Systems in Agriculture and Aquaculture”. en. In: *Proceedings of the 2024 14th International Conference on Information Communication and Management*. Paris France: ACM, Nov. 2024, pp. 22–28. ISBN: 979-8-4007-1747-5. DOI: 10.1145/3711609.3711613. URL: <https://dl.acm.org/doi/10.1145/3711609.3711613> (visited on 03/11/2026).

⁹Abir El Akhdar, Chafik Baidada, and Ali Kartit. “Adaptability of Microservices Architecture in IoT Systems : A Comprehensive Review”. In: *Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security*. NISS '24. New York, NY, USA: Association for Computing Machinery, Aug. 2024, pp. 1–9. ISBN: 979-8-4007-0929-6. DOI: 10.1145/3659677.3659734. URL: <https://dl.acm.org/doi/10.1145/3659677.3659734> (visited on 03/11/2026).

¹⁰Francesco Tusa et al. “Microservices and serverless functions—lifecycle, performance, and resource utilisation of edge based real-time IoT analytics”. en. In: *Future Generation Computer Systems* 155 (June 2024), pp. 204–218. ISSN: 0167739X. DOI: 10.1016/j.future.2024.02.006. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0167739X24000529> (visited on 03/02/2026).

¹¹Samodha Pallewatta, Vassilis Kostakos, and Rajkumar Buyya. “Microservices-based IoT Application Placement within Hetero-

This approach supports the construction of composable stream processing pipelines, where the output of one service becomes the input of another. As a result, data can be processed incrementally and in parallel, enabling efficient real-time analytics on large-scale IoT data streams. Furthermore, microservices allow processing tasks to be distributed across the edge–cloud continuum, with lightweight operations executed at the edge to reduce latency and bandwidth usage, while more complex analytics are handled in the cloud.

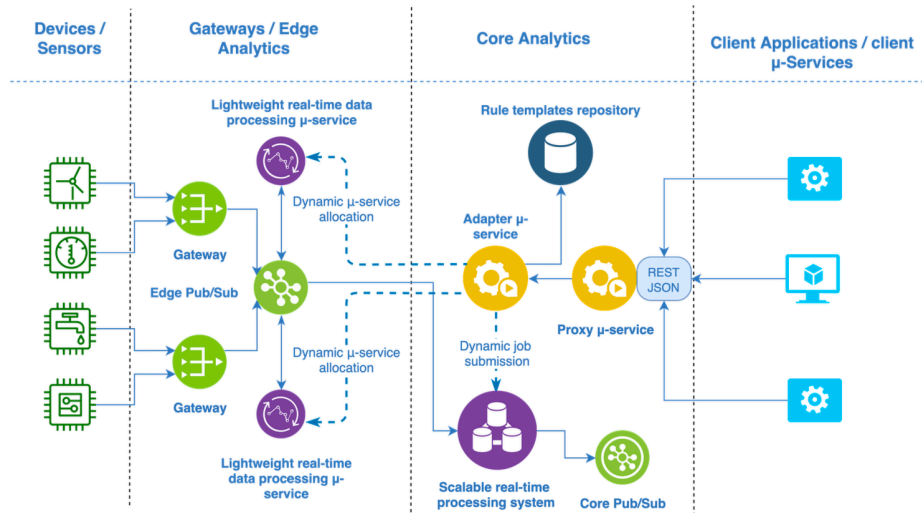


Figure 4.3: Reference Architecture for IoT Stream Processing

The reference architecture in Figure 4.3 illustrates how microservices enable flexible stream processing across IoT layers. Data generated by sensors is first collected through gateways and processed at the edge using lightweight microservices, where initial filtering and real-time analytics reduce latency and bandwidth usage. These processing units can be dynamically allocated and relocated depending on system requirements. The processed data is then forwarded to the core layer, where more scalable and resource-intensive analytics are performed using distributed stream processing systems. A proxy microservice provides a unified REST interface for managing processing tasks, while an adapter microservice handles the deployment and execution of stream processing jobs across heterogeneous environments. This design supports composable data pipelines,

geneous and Resource Constrained Fog Computing Environments”. In: *Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing*. UCC’19. New York, NY, USA: Association for Computing Machinery, Dec. 2019, pp. 71–81. ISBN: 978-1-4503-6894-0. DOI: 10 . 1145 / 3344341 . 3368800. URL: <https://dl.acm.org/doi/10.1145/3344341.3368800> (visited on 03/02/2026).

allowing stream processing tasks to be distributed efficiently between edge and cloud.¹² However, this distributed stream processing model introduces challenges related to data consistency, coordination, and communication overhead between services, which must be carefully managed to maintain system performance.¹³

Moreover, for Microservice and IoT Systems, Security is another critical consideration. Microservices improve resilience through service-level isolation, limiting the impact of failures or attacks. However, the increased number of services and communication interfaces expands the attack surface, requiring robust mechanisms for authentication, authorization, and secure communication.¹⁴

Despite these advantages, microservice-based IoT systems introduce additional complexity. Challenges include service orchestration, monitoring, and inter-service communication overhead. Moreover, network latency and processing overhead can impact performance, particularly in real-time IoT applications, and must be carefully managed.¹⁵ Moreover, microservice-based approaches relying on API-driven communication (e.g., REST) may introduce performance limitations in IoT stream processing scenarios. Frequent synchronous API calls between services can increase latency and reduce system responsiveness, which is critical in real-time IoT applications. In contrast, event-driven communication models are often more suitable, as they enable asynchronous, non-blocking interactions and better support continuous data streams. By leveraging message brokers and publish–subscribe mechanisms, event-driven microservices can improve scalability and reduce communication overhead. Therefore, while API-based interactions remain useful for control and management operations, IoT stream processing systems typically benefit from adopting event-driven architectures for data-intensive workflows.

¹²Pallewatta, Kostakos, and Buyya, “Microservices-based IoT Application Placement within Heterogeneous and Resource Constrained Fog Computing Environments”.

¹³El Akhdar, Baidada, and Kartit, “Adaptability of Microservices Architecture in IoT Systems”.

¹⁴Abir El Akhdar et al. “Exploring the Potential of Microservices in Internet of Things: A Systematic Review of Security and Prospects”. en. In: *Sensors* 24.20 (Oct. 2024). ISSN: 1424-8220. DOI: 10.3390/s24206771. URL: <https://www.mdpi.com/1424-8220/24/20/6771> (visited on 03/02/2026).

¹⁵Tusa et al., “Microservices and serverless functions—lifecycle, performance, and resource utilisation of edge based real-time IoT analytics”.

4.3 Event-Driven Architecture: MQTT Protocol

The Message Queuing Telemetry Transport (MQTT) protocol¹⁶ is one of the most widely used communication protocols for implementing Event-Driven Architecture in IoT systems. Designed for lightweight, low-bandwidth, and resource-constrained environments, MQTT enables efficient and reliable message exchange between distributed devices.

4.3.1 Definition

MQTT follows a publish-subscribe communication model,¹⁷ which aligns naturally with the principles of Event-Driven Architecture.

In this model, devices act as event producers (publishers) by sending messages to specific **topics**, while other components act as event consumers (subscribers) by receiving messages from those topics. A central component, known as the **MQTT broker**, is responsible for managing message distribution between publishers and subscribers.

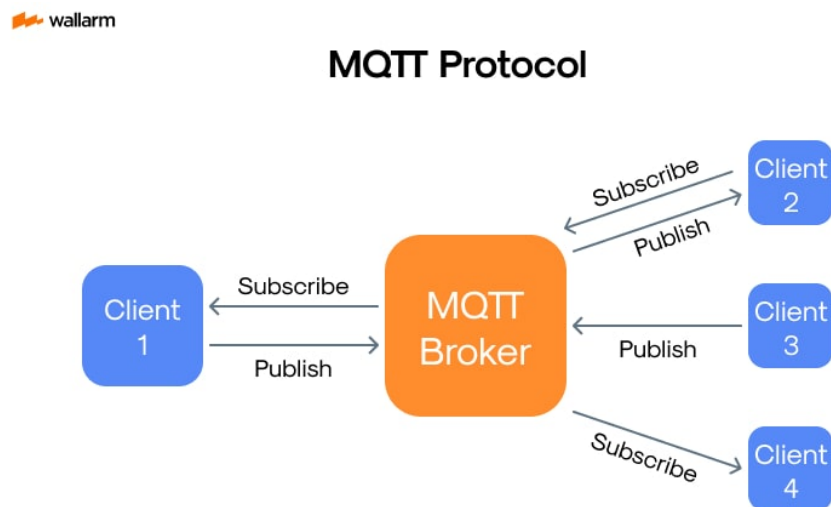


Figure 4.4: Basic MQTT Protocol Overview

This decoupled communication model allows IoT systems to scale efficiently and operate asynchronously. Devices do not need to know the identity or location of other components in the system, which reduces dependencies and simplifies integration. As a

¹⁶MQTT - The Standard for IoT Messaging. URL: <https://mqtt.org/> (visited on 03/02/2026).

¹⁷Publish-Subscribe - Intro to Pub-Sub Messaging. en. URL: <https://www.confluent.io/learn/publish-subscribe/> (visited on 03/02/2026).

result, MQTT is particularly well-suited for large-scale IoT deployments such as smart homes, industrial monitoring systems, and smart cities. As highlighted in recent studies, MQTT enables real-time data ingestion and processing in IoT platforms, where large volumes of sensor data are transmitted from distributed environments to cloud-based systems for analysis and decision-making.¹⁸ This makes MQTT particularly suitable for event-driven systems, where data is continuously generated and must be processed asynchronously.

The protocol defines structured message exchanges using control packets and supports hierarchical topic naming, allowing flexible organization of event streams. Additionally, MQTT provides multiple Quality of Service (QoS) levels to balance reliability and performance, ensuring appropriate delivery guarantees depending on application requirements.¹⁹

Due to its lightweight design, publish-subscribe model, and support for scalable data distribution, MQTT serves as a practical implementation of the broker-based topology in Event-Driven Architecture for IoT systems.

4.3.2 MQTT Architecture

MQTT follows a centralized broker-based architecture that enables asynchronous communication between distributed clients in an IoT system.²⁰ The architecture consists of three primary components: publishers, subscribers, and the broker. **Publishers** are clients that generate and send messages, typically representing IoT devices such as sensors or gateways. These devices publish data to specific topics without needing to know which components will consume the data.

Subscribers are clients that express interest in one or more topics and receive messages that match their subscriptions. These are often applications, analytics services, or monitoring systems that process incoming data.

¹⁸Nitol Saha et al. "Performance evaluation framework of MQTT client libraries for IoT applications in manufacturing". In: *Manufacturing Letters*. 52nd SME North American Manufacturing Research Conference (NAMRC 52) 41 (Oct. 2024), pp. 1237–1245. ISSN: 2213-8463. DOI: 10.1016/j.mfglet.2024.09.150. URL: <https://www.sciencedirect.com/science/article/pii/S221384632400213X> (visited on 03/02/2026).

¹⁹Keila Lima et al. *Evaluation of MQTT Bridge Architectures in a Cross-Organizational Context*. arXiv:2501.14890 [cs]. Jan. 2025. DOI: 10.48550/arXiv.2501.14890. URL: <http://arxiv.org/abs/2501.14890> (visited on 03/02/2026).

²⁰Quincozes, Emilio, and Kazienko, "MQTT Protocol".

Broker is the central component responsible for managing communication. It receives all messages from publishers, filters them based on topic subscriptions, and distributes them to the appropriate subscribers. This design decouples producers and consumers, enabling scalability and flexibility in IoT systems.

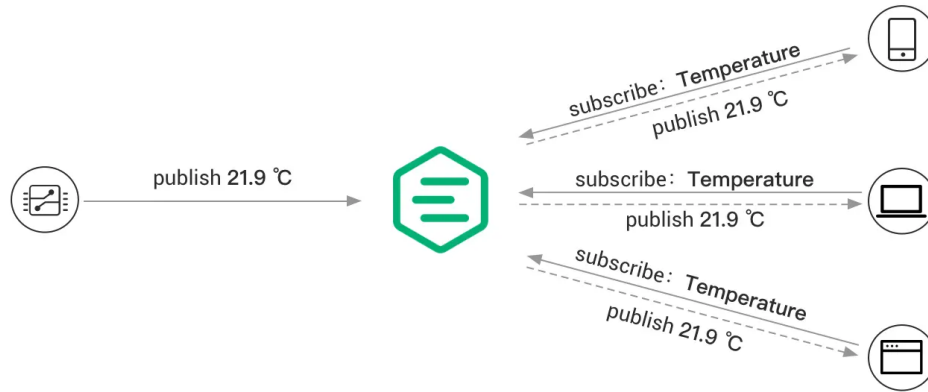


Figure 4.5: An Example of how MQTT works

Communication Workflow The interaction between MQTT components follows a structured sequence of message exchanges between clients and the broker. A client first establishes a connection with the broker by sending a CONNECT message and receives a CONNACK acknowledgment in response. Once connected, a publisher can send messages to a specific topic using a PUBLISH message. Subscribers express their interest in topics by sending a SUBSCRIBE request to the broker and receiving a SUBACK acknowledgment. After successful subscription, the broker forwards all relevant messages published to the subscribed topics. This communication model allows a single published message to be delivered to multiple subscribers efficiently.²¹

²¹Aleesha M and Laseena. *MQTT Protocol for Resource Constrained IoT Applications : A Review*. en. SSRN Scholarly Paper. Rochester, NY, Aug. 2022. DOI: 10.2139/ssrn.4299372. URL: <https://papers.ssrn.com/abstract=4299372> (visited on 03/02/2026).

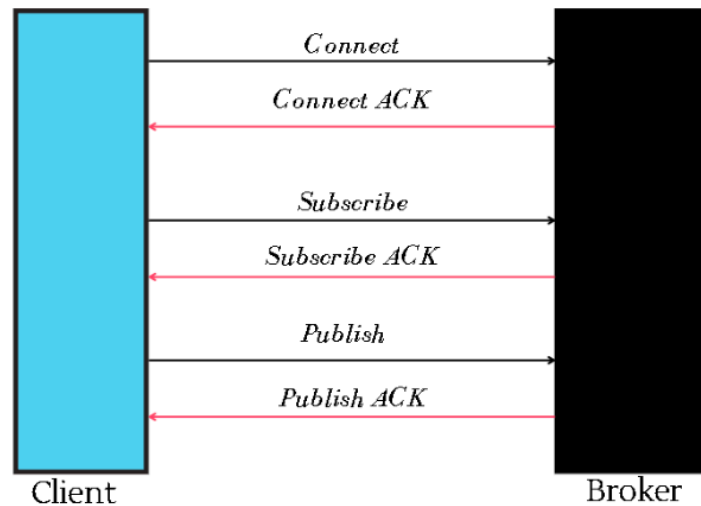


Figure 4.6: Working Demonstration of MQTT Protocol

This asynchronous workflow enables real-time data distribution while maintaining loose coupling between system components.

MQTT Message MQTT communication is based on a set of control packet types that define how clients and brokers interact. Each packet serves a specific role in establishing connections, publishing messages, and maintaining communication.

- **CONNECT (1):** Sent by a client to initiate a connection with the broker.
- **CONNACK (2):** Sent by the broker in response to a CONNECT request, indicating whether the connection was successful.
- **PUBLISH (3):** Used to transmit messages from publishers to the broker, and from the broker to subscribers.
- **PUBACK (4):** Acknowledgment for QoS level 1 message delivery, confirming that a message has been received.
- **PUBREC (5):** Part of the QoS level 2 handshake, indicating that a message has been received.
- **PUBREL (6):** Sent to release the message as part of the QoS level 2 delivery process.

- **PUBCOMP (7):** Final step in QoS level 2 communication, confirming that message delivery is complete.
- **SUBSCRIBE (8):** Sent by a client to request subscription to one or more topics.
- **SUBACK (9):** Sent by the broker to confirm a successful subscription.
- **UNSUBSCRIBE (10):** Sent by a client to cancel a subscription to specific topics.
- **UNSUBACK (11):** Acknowledgment from the broker confirming the unsubscribe request.
- **PINGREQ (12):** Sent by a client to maintain an active connection and check if the broker is reachable.
- **PINGRESP (13):** Sent by the broker in response to a PINGREQ to confirm connectivity.
- **DISCONNECT (14):** Sent by a client to gracefully terminate the connection with the broker.

Quality of Service (QoS) MQTT provides three Quality of Service (QoS) levels that define the guarantee of message delivery between publishers and subscribers.²² These levels allow IoT systems to balance reliability, latency, and network overhead depending on application requirements.

QoS 0 – At most once: This is the simplest level of service, where messages are delivered on a best-effort basis without acknowledgment. There is no guarantee that the message will reach the subscriber, making it suitable for non-critical data such as periodic sensor readings. This level offers the lowest latency and minimal overhead.

QoS 1 – At least once: In this level, the sender ensures that the message is delivered at least once by requiring an acknowledgment (PUBACK). If the acknowledgment is not received, the message is retransmitted. While this guarantees delivery, it may result in duplicate messages, requiring the receiver to handle potential duplicates.

QoS 2 – Exactly once: This is the highest level of service, ensuring that each message is delivered exactly once. It uses a four-step handshake mechanism involving PUBREC,

²²Cisco, *IoT fundamentals: Networking technologies, protocols, and use cases*.

PUBREL, and PUBCOMP messages. This eliminates duplicates but introduces additional communication overhead and latency.

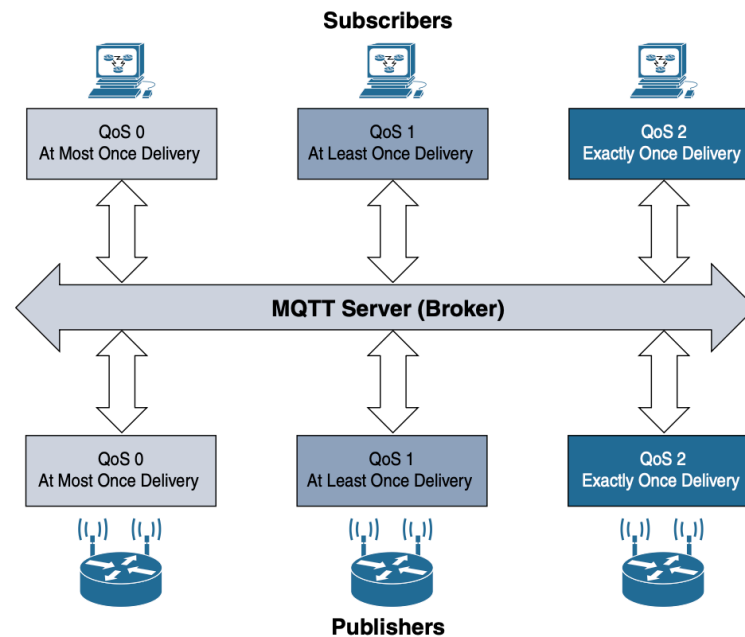


Figure 4.7: MQTT's Quality of Service

These QoS levels provide flexibility in IoT systems, allowing developers to choose the appropriate trade-off between performance and reliability.²³ For example, critical control messages may use QoS 2, while high-frequency telemetry data often uses QoS 0 to minimize resource consumption. From an architectural perspective, QoS mechanisms enhance the reliability of event delivery in MQTT-based Event-Driven Architecture. They ensure that events are delivered according to the required guarantees, even in unreliable network conditions, which are common in IoT environments.²⁴

Overall, MQTT architecture combines a broker-based design, publish–subscribe communication, and flexible QoS mechanisms to support scalable, reliable, and asynchronous event-driven IoT systems.

²³What is MQTT Quality of Service (QoS) 0,1, & 2? – MQTT Essentials: Part 6. en. Section: blog. Feb. 2026. URL: <https://www.hivemq.com/blog/mqtt-essentials-part-6-mqtt-quality-of-service-levels/> (visited on 03/02/2026).

²⁴An Introduction to Low Power IoT. en. URL: <https://www.particle.io/iot-guides-and-resources/low-power-iot/> (visited on 03/02/2026).

4.3.3 Pub/Sub Communications

MQTT is fundamentally based on the publish–subscribe communication model, which differs significantly from traditional request–response paradigms such as HTTP. In this model, communication is mediated by a broker, and interacting entities do not communicate directly with each other.

In MQTT, publishers generate messages and send them to specific topics, while subscribers express interest in these topics and receive relevant messages. The broker acts as an intermediary that **decouples** publishers and subscribers by handling message routing and delivery.

Compared to traditional client-server communication, the publish–subscribe model offers significant advantages in IoT systems.²⁵ In request–response models, clients must directly address servers and wait for responses, which can introduce latency and tight coupling. In contrast, MQTT allows many-to-many communication, where a single message can be distributed to multiple subscribers without additional overhead. This model is particularly effective in IoT environments, where large numbers of devices continuously generate data. For example, a temperature sensor can publish readings to a topic, and multiple services—such as monitoring dashboards, alert systems, and storage platforms—can independently consume the same data stream without affecting the publisher.

From an architectural perspective, MQTT’s publish–subscribe model directly implements the principles of Event-Driven Architecture. Messages correspond to events, topics act as event channels, and subscribers represent event consumers. This enables scalable, loosely coupled, and real-time data processing across distributed IoT systems.

4.3.4 MQTT Features

MQTT provides several built-in features that enhance reliability, flexibility, and efficiency in IoT systems. These features support various communication requirements such as message persistence, fault tolerance, and connection management, making MQTT

²⁵“Event-Driven vs. API-Driven: A Strategic Architectural Paradigm Choice”. en. In: *ResearchGate* (Dec. 2025). DOI: 10.22399/ijcesen.4458. URL: https://www.researchgate.net/publication/398545188_Event-Driven_vs_API-Driven_A_Strategic_Architectural_Paradigm_Choice (visited on 03/02/2026).

suitable for resource-constrained and distributed environments.

Retained Messages

Retained messages allow the MQTT broker to store the last message published on a specific topic and deliver it immediately to new subscribers.²⁶ When a client subscribes to a topic, the broker sends the retained message without waiting for a new publication.

This feature is particularly useful in IoT systems where devices may join the network intermittently. For example, a newly connected device can instantly receive the latest sensor value or system state without requiring the publisher to resend the data. Retained messages improve data availability and reduce latency, ensuring that subscribers always have access to the most recent information.

Last Will and Testament (LWT)

The Last Will and Testament (LWT) is a mechanism that allows a client to notify other components when it disconnects unexpectedly.²⁷ When establishing a connection, a client can register a will message with the broker, which is automatically published if the client loses connection without a proper DISCONNECT.

This feature is essential for fault detection in IoT systems. For instance, if a sensor fails or goes offline, the broker can publish a predefined alert message to notify monitoring systems.

LWT enhances system reliability by enabling automatic detection of failures and supporting reactive event-driven behavior.

²⁶thingsboard. *Retained Messages*. URL: <https://thingsboard.io/docs/mqtt-broker/user-guide/retained-messages/> (visited on 03/02/2026).

²⁷*What is MQTT Last Will and Testament (LWT)? – MQTT Essentials: Part 9*. en. Section: blog. Feb. 2026. URL: <https://www.hivemq.com/blog/mqtt-essentials-part-9-last-will-and-testament/> (visited on 03/02/2026).

MQTT-Packet:	
CONNECT	
contains:	Example
clientId	"client-1"
cleanSession	true
username (optional)	"hans"
password (optional)	"letmein"
lastWillTopic (optional)	"/hans/will"
lastWillQos (optional)	2
lastWillMessage (optional)	"unexpected exit"
lastWillRetain (optional)	false
keepAlive	60

Figure 4.8: Example Last Will Testament Message

Persistent Sessions

Persistent sessions enable MQTT clients to maintain their subscription information and undelivered messages across disconnections. When a client reconnects, the broker can resume the session and deliver any messages that were missed during the offline period.

This feature is particularly important in IoT environments where devices may experience intermittent connectivity. It ensures message continuity and reduces data loss.

Persistent sessions contribute to the robustness of MQTT-based systems by supporting reliable communication even in unstable network conditions.

Keep Alive Mechanism

The keep alive mechanism allows clients and brokers to monitor the status of a connection. Clients periodically send PINGREQ messages to the broker, which responds with PINGRESP messages. If no communication occurs within a specified time interval, the connection is considered lost.

This mechanism helps detect broken connections and ensures timely cleanup of inactive clients. In IoT systems, where devices may operate in unreliable networks, keep alive plays a critical role in maintaining system stability.

Overall, these features enhance the functionality of MQTT beyond basic message transmission. They support reliable and efficient communication in dynamic IoT environments, where devices frequently connect and disconnect, and real-time data availability is essential.

4.3.5 Security in MQTT

Security is a critical concern in MQTT-based IoT systems due to the large number of connected devices, the sensitivity of transmitted data, and the resource constraints of IoT environments. Although MQTT provides efficient and lightweight communication, it does not inherently include strong security mechanisms, making it vulnerable to various attacks.

In its basic form, MQTT relies on the underlying transport protocol, typically TCP, which does not provide encryption.²⁸ As a result, transmitted messages can be intercepted, modified, or replayed by attackers. Therefore, additional security mechanisms must be implemented to ensure confidentiality, integrity, and authentication in MQTT systems.

Authentication

MQTT supports basic authentication using username and password credentials during the connection phase. However, these credentials are transmitted in plain text unless protected by secure transport protocols, making them vulnerable to brute-force and interception attacks.

Recent research proposes enhanced authentication mechanisms, including lightweight two-way authentication schemes, to improve security while maintaining low computational overhead. These approaches are particularly important for resource-constrained IoT devices, where traditional security mechanisms may be too expensive.

Transport Security (TLS/SSL)

To ensure confidentiality and integrity, MQTT commonly relies on Transport Layer Security (TLS). TLS encrypts communication between clients and brokers, protecting data from interception and modification.

²⁸Timothy Murkomen. “Performance, privacy, and security issues of TCP/IP at the application layer: A comprehensive survey”. In: *GSC Advanced Research and Reviews* 18.3 (Mar. 2024), pp. 234–264. ISSN: 25824597. DOI: 10.30574/gscarr.2024.18.3.0106. URL: <https://gsconlinepress.com/journals/gscarr/content/performance-privacy-and-security-issues-tcpip-application-layer-comprehensive-survey> (visited on 03/02/2026).

However, while TLS significantly enhances security, it introduces additional computational overhead and latency. This creates a trade-off between security and performance, especially in constrained IoT environments.²⁹ :contentReference[oaicite:1]index=1

Authorization

Authorization mechanisms control access to topics and define what actions a client is allowed to perform. Without proper authorization, clients may publish or subscribe to unauthorized topics, leading to data leakage or system compromise.

Effective topic-level access control is therefore essential in MQTT systems to ensure secure and controlled communication between components.

Intrusion Detection and Advanced Security Mechanisms

Beyond traditional security mechanisms, recent research has explored the use of machine learning-based intrusion detection systems (IDS) to secure MQTT environments. These systems analyze MQTT traffic patterns to detect abnormal or malicious behavior.

For example, feature engineering combined with machine learning models has been shown to achieve high detection accuracy (over 96%) in identifying attacks such as DoS, brute-force, and malformed packet attacks.³⁰ :contentReference[oaicite:2]index=2

Such approaches are particularly effective in IoT systems, where large volumes of data make manual monitoring impractical. By selecting key features (e.g., QoS level, message length, and timing characteristics), IDS models can efficiently distinguish between normal and malicious traffic.

Security Threats and Challenges

MQTT systems are vulnerable to multiple types of attacks, including:

- **Denial-of-Service (DoS)** attacks that overwhelm brokers with excessive messages
- **Man-in-the-Middle (MitM)** attacks that intercept and manipulate communication

²⁹Abdulelah Al Hanif and Mohammad Ilyas. “Effective Feature Engineering Framework for Securing MQTT Protocol in IoT Environments”. In: *Sensors (Basel, Switzerland)* 24.6 (Mar. 2024), p. 1782. ISSN: 1424-8220. DOI: 10.3390/s24061782. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10975182/> (visited on 03/02/2026).

³⁰Al Hanif and Ilyas, “Effective Feature Engineering Framework for Securing MQTT Protocol in IoT Environments”.

- **Brute-force attacks** targeting authentication credentials
- **Eavesdropping** due to lack of encryption

These threats are exacerbated by the lightweight design of MQTT, which prioritizes efficiency over security. As a result, securing MQTT requires additional mechanisms beyond the protocol itself.

Furthermore, IoT devices often have limited computational resources, making it difficult to implement strong security mechanisms without affecting performance. This creates a fundamental trade-off between security, efficiency, and scalability.

Overall, while MQTT enables efficient communication in IoT systems, its security limitations necessitate the integration of advanced protection mechanisms such as TLS, access control, and intelligent intrusion detection systems to ensure robust and reliable operation.

Security Enhancements and Countermeasures

Several countermeasures can be applied to address the security limitations of MQTT.³¹ To ensure message integrity, cryptographic hashing can be used, allowing brokers to detect and reject tampered messages. Topic validation and restrictions on wildcard usage help prevent topic hijacking and malicious injections.

Access control can be strengthened through role-based access control (RBAC) and Access Control Lists (ACLs), ensuring that only authorized clients can publish or subscribe to specific topics. Additionally, the use of TLS/SSL encryption protects data confidentiality by securing communication between clients and brokers.

Emerging approaches also explore hardware-based security mechanisms, such as device identity verification and tamper-resistant cryptographic modules, particularly for resource-constrained IoT devices.

Overall, these enhancements improve the security, reliability, and trustworthiness of MQTT-based IoT systems.

³¹OWASP MQTT Guide | OWASP Foundation. en. URL: <https://owasp.org/www-project-mqtt-guide/> (visited on 03/02/2026).

4.3.6 Performance Analysis

4.3.7 Performance and Scalability Analysis

MQTT is widely adopted in IoT systems due to its lightweight design and low communication overhead, making it suitable for resource-constrained devices with limited processing power and bandwidth.³²

From a performance perspective, MQTT is optimized for transmitting small, frequent messages, which is typical in IoT applications such as sensor data streaming. Its minimal header size and efficient publish–subscribe model reduce network traffic and latency, enabling real-time communication in distributed environments.

Empirical studies of real-world MQTT deployments reveal that QoS level 0 is predominantly used, as it minimizes communication overhead and latency. This indicates that many IoT systems prioritize performance over strict delivery guarantees, especially in large-scale environments.³³

In terms of scalability, MQTT’s broker-based architecture enables efficient message distribution to multiple subscribers. However, this centralization can also introduce bottlenecks, particularly in high-throughput scenarios where a single broker must handle a large volume of messages.

Additionally, real-world traffic analysis shows that MQTT workloads are characterized by numerous low-throughput connections rather than a few high-intensity streams. This suggests that scalability challenges are more related to managing large numbers of devices than handling high data rates.³⁴

Despite its advantages, MQTT has limitations. It is not well-suited for large payload transmission and relies on TCP for reliability, which may introduce additional overhead. Furthermore, the need for external security mechanisms can impact performance, especially in constrained environments.

Overall, MQTT achieves a balance between efficiency and scalability, making it highly effective for IoT systems, although careful system design is required to address

³²M and Laseena, *MQTT Protocol for Resource Constrained IoT Applications*.

³³Corrado Innamorati et al. “The pulse of MQTT in the wild: A large-scale traffic analysis”. en. In: *Computer Networks* 274 (Jan. 2026), p. 111845. ISSN: 13891286. DOI: 10.1016/j.comnet.2025.111845. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1389128625008114> (visited on 03/02/2026).

³⁴Innamorati et al., “The pulse of MQTT in the wild”.

its limitations in large-scale deployments.

4.3.8 Conclusion

Overall, MQTT offers a lightweight and scalable solution for IoT communication, with performance influenced by system design and deployment scale.

4.4 Advanced Protocols: Beyond MQTT

4.4.1 MQTT 5.0

MQTT 5.0 is the latest version of the MQTT protocol, designed to enhance functionality, scalability, and error handling while maintaining the lightweight nature of earlier versions. It introduces several improvements that address limitations present in MQTT 3.1.1, making it more suitable for complex and large-scale IoT systems.³⁵

One of the key enhancements in MQTT 5.0 is improved error reporting. Unlike previous versions, which provided limited feedback, MQTT 5.0 introduces reason codes and diagnostic information that enable better debugging and system monitoring.

Another significant feature is the introduction of user properties, which allow additional metadata to be included in messages. This enhances flexibility by enabling application-specific information to be transmitted alongside payload data.

MQTT 5.0 also improves flow control through mechanisms such as message expiry intervals and session expiry. These features allow messages to be automatically discarded after a specified time and enable better management of client sessions, reducing unnecessary resource usage.

In addition, topic aliasing is introduced to reduce the size of transmitted messages by replacing long topic names with shorter identifiers. This optimization is particularly beneficial in bandwidth-constrained environments.

The protocol also enhances scalability through shared subscriptions, which allow multiple subscribers to share the processing of messages from a single topic. This enables load balancing and improves system performance in high-throughput scenarios.

³⁵MQTT - The Standard for IoT Messaging.

From an architectural perspective, MQTT 5.0 strengthens the publish–subscribe model by adding greater control over message delivery, session management, and system observability. These enhancements make MQTT 5.0 more suitable for enterprise-scale and industrial IoT applications.

Table 4.1: Comparison of MQTT 3.1.1 and MQTT 5.0

Feature	MQTT 3.1.1	MQTT 5.0
Error Reporting	Limited, minimal feedback on failures	Enhanced with reason codes and detailed diagnostics
Message Metadata	No support for additional metadata	Supports user properties for custom metadata
Session Management	Basic clean session flag	Advanced session expiry control
Message Expiry	Not supported	Supports message expiry intervals
Flow Control	Limited control over message flow	Improved flow control with receive maximum and quotas
Topic Handling	Full topic names required	Topic aliasing reduces message size
Subscription Model	Standard subscriptions only	Shared subscriptions for load balancing
Scalability	Suitable for small to medium systems	Improved scalability for large-scale IoT systems
Error Transparency	Low visibility into errors	High transparency with detailed feedback
Complexity	Simple and lightweight	More complex but more flexible

Compared to MQTT 3.1.1, MQTT 5.0 provides more granular control over commu-

nication behavior and significantly improves system transparency and scalability.

4.4.2 CoAP: Constrained Application Protocol

The Constrained Application Protocol (CoAP) is a lightweight application-layer protocol designed for **resource-constrained IoT devices** and low-power networks. Unlike MQTT, which follows a publish–subscribe model, CoAP is based on a request–response paradigm similar to HTTP, enabling direct communication between clients and servers.

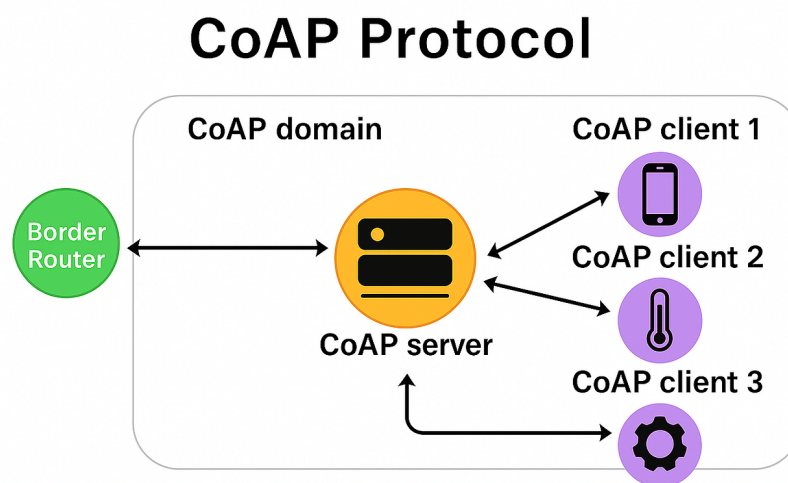


Figure 4.9: High-level Overview of CoAP

CoAP operates over UDP, which reduces communication overhead and latency compared to TCP-based protocols such as MQTT.³⁶ This makes it particularly suitable for environments where bandwidth and energy consumption are critical constraints. Additionally, CoAP supports asynchronous message exchanges and confirmable messages, allowing reliable communication despite using an unreliable transport protocol.

³⁶Cisco, *IoT fundamentals: Networking technologies, protocols, and use cases*.

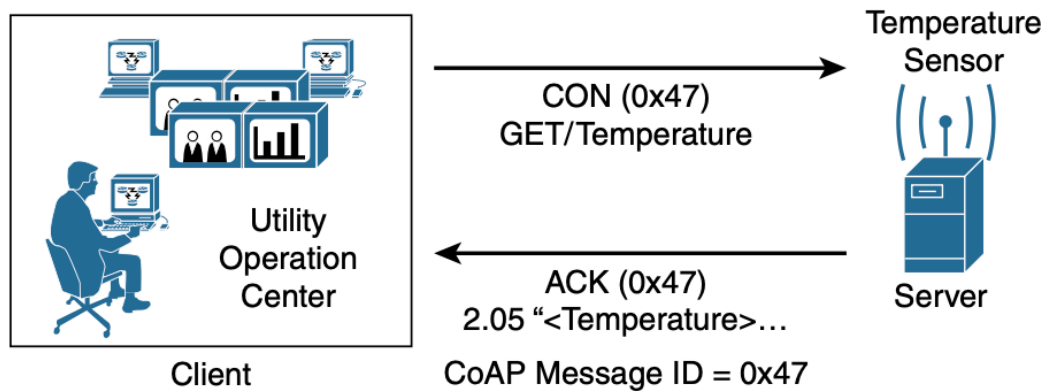


Figure 4.10: Example CoAP session

A key feature of CoAP is its RESTful design, which allows IoT devices to expose resources that can be accessed using standard methods such as GET, POST, PUT, and DELETE.³⁷ This enables seamless integration with web technologies and simplifies interoperability between IoT systems and traditional web services.

From an architectural perspective, CoAP differs significantly from MQTT. While MQTT relies on a centralized broker to mediate communication, CoAP enables direct device-to-device or client-to-server interactions. This reduces dependency on a central component but may limit scalability in large distributed systems.

In terms of performance, CoAP is highly efficient for simple request–response interactions and is well-suited for scenarios such as device control and configuration. However, it lacks the inherent many-to-many communication capabilities provided by MQTT’s publish–subscribe model, making it less suitable for event-driven applications that require data distribution to multiple consumers.

³⁷EMQX Team. *CoAP Protocol: Features, Use Cases, Pros & Cons for IoT*. en. URL: <https://www.emqx.com/en/blog/coap-protocol> (visited on 03/02/2026).

Factor	CoAP	MQTT
Main transport protocol	UDP	TCP
Typical messaging	Request/response	Publish/subscribe
Effectiveness in LLNs	Excellent	Low/fair (Implementations pairing UDP with MQTT are better for LLNs.)
Security	DTLS	SSL/TLS
Communication model	One-to-one	many-to-many
Strengths	Lightweight and fast, with low overhead, and suitable for constrained networks; uses a RESTful model that is easy to code to; easy to parse and process for constrained devices; support for multicasting; asynchronous and synchronous messages	TCP and multiple QoS options provide robust communications; simple management and scalability using a broker architecture
Weaknesses	Not as reliable as TCP-based MQTT, so the application must ensure reliability.	Higher overhead for constrained devices and networks; TCP connections can drain low-power devices; no multicasting support

Figure 4.11: MQTT vs CoAP

Overall, CoAP provides a lightweight and efficient alternative to MQTT for IoT systems that require direct, low-overhead communication, while MQTT remains more suitable for scalable, event-driven architectures.

4.4.3 DDS: Data Distribution Service

The Data Distribution Service (DDS) is a high-performance, data-centric publish–subscribe protocol designed for real-time and mission-critical systems.³⁸ Unlike MQTT, which relies on a centralized broker, DDS operates in a fully decentralized, broker-less architecture, where participants communicate directly with each other.

DDS follows a data-centric communication model, where data is organized into topics and shared across distributed systems using a global data space. Publishers and subscribers interact through this shared space without requiring direct knowledge of each other, enabling efficient and scalable data exchange.

³⁸What is DDS?. URL: <https://www.dds-foundation.org/what-is-dds-3/> (visited on 03/11/2026).

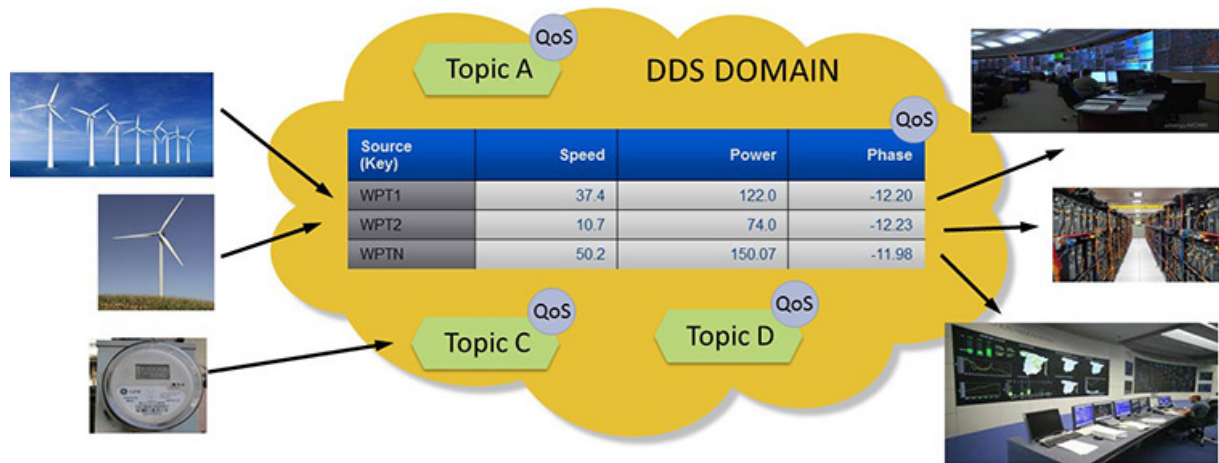


Figure 4.12: High-level Overview of DDS Systems

One of the defining features of DDS is its support for real-time Quality of Service (QoS) policies. These policies allow fine-grained control over communication behavior, including latency, reliability, durability, and resource usage. This makes DDS particularly suitable for applications that require deterministic performance, such as autonomous vehicles, robotics, and industrial control systems.

From an architectural perspective, DDS eliminates the single point of failure associated with broker-based systems like MQTT. By enabling peer-to-peer communication, DDS improves fault tolerance and reduces communication latency. However, this decentralized design also introduces increased complexity in system configuration and management.

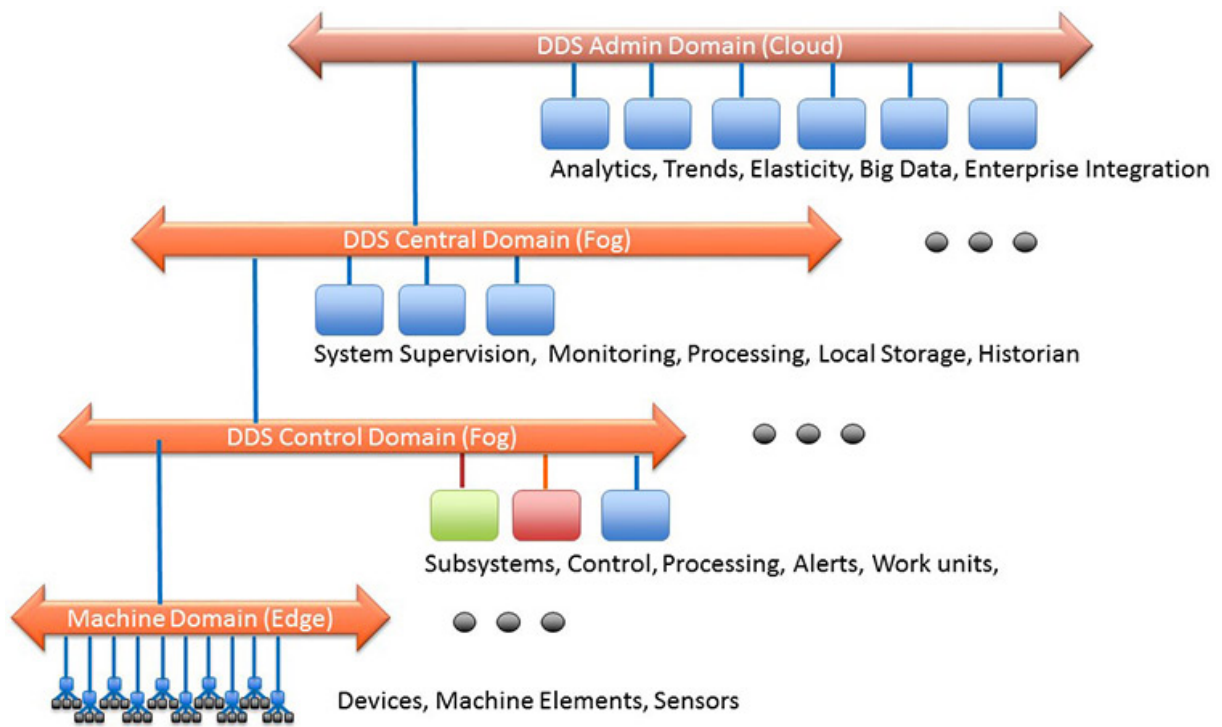


Figure 4.13: A Reference Architecture for DDS

In comparison to MQTT, DDS provides stronger guarantees for real-time communication and system reliability, but at the cost of higher computational and network overhead. While MQTT is optimized for lightweight and large-scale IoT deployments, DDS is better suited for environments where strict timing constraints and high reliability are essential.

Overall, DDS represents a more advanced and robust communication paradigm that extends beyond traditional IoT messaging protocols, offering real-time, scalable, and fault-tolerant communication for complex distributed systems.

Chapter 5

Industrial Case Studies

5.1 Case Study 1: MQTT-Based Cold Chain Monitoring System

5.1.1 Industrial Problem

Cooling systems play a critical role in the food, pharmaceutical, and chemical industries, where maintaining controlled environmental conditions is essential to ensure product quality and safety. Traditional monitoring approaches often rely on manual inspection by on-site personnel, which can be time-consuming, costly, and prone to human error. As industrial systems evolve toward automation, there is an increasing demand for solutions that improve operational efficiency, reliability, and data accuracy.¹

Temperature is one of the most critical parameters in food preservation, as it directly influences physical, chemical, and microbiological properties, ultimately affecting product quality and shelf life.² Despite the availability of various cold storage technologies, maintaining consistent temperature conditions throughout storage and transportation remains a significant challenge.³ Consequently, continuous monitoring and traceability across the cold chain are essential to prevent spoilage, ensure compliance with safety

¹H. Karaoglu, T. Kabak, and I. Kırbaş. “—”. In: *Turkish Journal of Health Science and Life* 7.1 (2024), pp. 33–45.

²A. Böbrek, İ. Kırbaş, and K. Özcan. “Patient and infant monitoring and control system (HBİKS)”. in: *Mehmet Akif Ersoy University Journal of Graduate School of Natural and Applied Sciences* 8.Supplementary Issue 1 (2017), pp. 252–261. DOI: 10.29048/makufebd.341310.

³*Law on veterinary services, plant health, food and feed (law no: 5996)*. 2010.

standards, and support quality assurance processes.

The Internet of Things (IoT) has emerged as a key enabler in industrial automation and supply chain management by facilitating real-time data acquisition and remote system control.⁴ However, IoT environments generate continuous streams of sensor data, requiring efficient communication protocols that minimize bandwidth usage and computational overhead. Lightweight messaging protocols such as MQTT have been widely adopted to address these constraints.⁵

In this context, the present study applies MQTT to enable real-time monitoring of cold storage environments. Sensor data are transmitted to a centralized server and visualized through a web-based interface, allowing users to remotely monitor environmental conditions and respond rapidly to deviations. This approach enhances system responsiveness, reduces operational risks, and improves overall cold chain management efficiency.

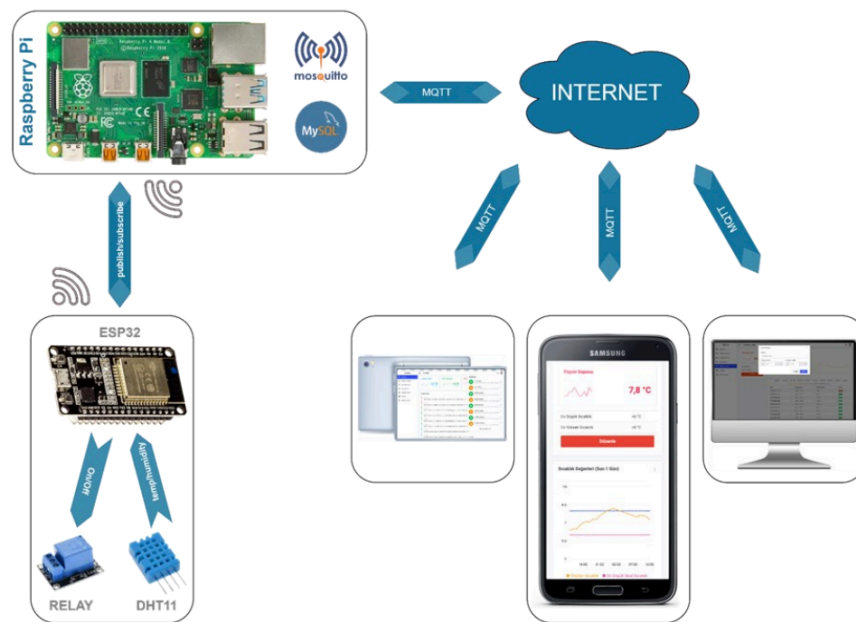


Figure 5.1: Reference Hardware Setup

⁴B. Keleş and G. Ova. “Use of information technologies in food supply chain management”. In: *Adnan Menderes University Journal of Faculty of Agriculture* 17.1 (2020), pp. 137–143. DOI: 10.25308/aduziraat.695732.

⁵N. Naik. “Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP”. in: *IEEE international systems engineering symposium (ISSE)*. Vienna, Austria, 2017, pp. 1–7; S. Saritha and V. Saravathi. “A study on application layer protocols used in IoT”. in: *International conference on circuits, controls, and communications (CCUBE)*. Bengaluru, India, 2017, pp. 155–159.

5.1.2 MQTT-Based System Architecture

The proposed system architecture is based on a publish–subscribe communication model enabled by MQTT. ESP32-based sensor nodes equipped with environmental sensors are deployed within cold storage units to collect real-time temperature and humidity data. These nodes act as MQTT clients and publish sensor readings to a centralized MQTT broker.

The broker serves as an intermediary that routes messages to subscribed clients based on predefined topics.⁶ This design decouples data producers from consumers, enabling scalable, asynchronous communication and simplifying system integration.

MQTT provides three Quality of Service (QoS) levels to support different reliability requirements. QoS 0 delivers messages at most once without acknowledgment, QoS 1 ensures at-least-once delivery through retransmission, and QoS 2 guarantees exactly-once delivery via a two-phase handshake process.⁷ In industrial monitoring scenarios, QoS 1 is often preferred as it balances reliability and communication overhead.

Compared to alternative application-layer protocols such as CoAP, HTTP, AMQP, and DDS, MQTT is widely adopted due to its lightweight design, minimal header size, and low bandwidth consumption.⁸ These characteristics make it particularly suitable for resource-constrained IoT devices and wireless sensor networks.⁹

In addition to fixed storage monitoring, the architecture can be extended to support logistics tracking. During transportation, product data can be collected using RFID, NFC, or Bluetooth technologies and transmitted to a Central Monitoring System (CMS) via GSM/GPRS connectivity.¹⁰ This extension enables end-to-end visibility and traceability across the entire cold chain.

⁶A. Alkan and İ. Kırbaş. “An investigation of master’s theses written on the internet of things (IoT) in turkey: a meta-analysis study”. In: *Journal of Engineering Sciences and Design* 10.1 (2022), pp. 318–327. DOI: 10.21923/jesd.955660.

⁷M. A. Ebleme. “Performance analysis of internet of things application layer communication protocols”. MA thesis. Sakarya University, 2019.

⁸Naik, “Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP”.

⁹Saritha and Sarasvathi, “A study on application layer protocols used in IoT”.

¹⁰T. Evgen. *RFID and internet of things based supply chain information management*. 2017; M. R. Cansız. *IoT data protocols MQTT, CoAP, AMQP and HTTP comparison*. 2020.

5.1.3 Implementation Overview

The system was implemented in the storage facilities of the Dairy Products and Technologies Application and Research Center at Burdur Mehmet Akif Ersoy University. An IoT-based monitoring solution was deployed to continuously track environmental conditions across multiple storage rooms.

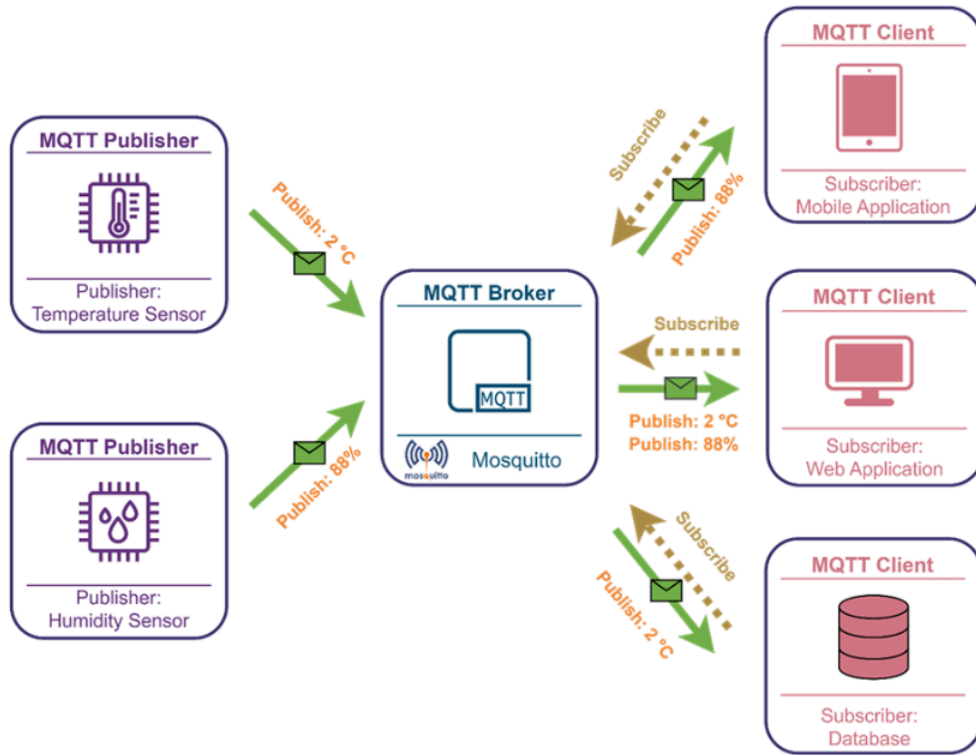


Figure 5.2: IOT System Architecture

Each storage unit was equipped with an ESP32 microcontroller integrated with a DHT11 temperature and humidity sensor. The ESP32 modules collected environmental data in real time and transmitted measurements wirelessly via MQTT. A Raspberry Pi was configured as the central server, hosting the MQTT broker, database services, and backend application logic.

The collected data were stored in a MySQL database to ensure persistence and support historical analysis. A web-based application provided real-time visualization, enabling users to monitor environmental conditions, query historical records, and manage system configurations efficiently.

The DHT11 sensor is a low-cost digital sensor capable of measuring temperature (0–50°C with $\pm 2^\circ\text{C}$ accuracy) and humidity (20–80% with $\pm 5\%$ accuracy). Its digital output simplifies integration with microcontrollers, making it suitable for indoor monitoring applications where moderate accuracy is sufficient.¹¹

The ESP32 microcontroller is a low-power system-on-chip featuring integrated Wi-Fi and Bluetooth connectivity. Its support for multiple communication interfaces and efficient power consumption makes it highly suitable for IoT deployments.¹²

The Raspberry Pi serves as a centralized processing and storage unit. Although it lacks real-time processing capabilities comparable to microcontrollers, it is well-suited for server-side tasks such as data aggregation, database management, and web hosting.¹³

In practice, the system was deployed across three cold storage rooms and one incubation room. When temperature values exceeded predefined thresholds, automated alerts were triggered and delivered via email, SMS, or push notifications. This event-driven alerting mechanism enabled rapid response to environmental anomalies and improved operational efficiency.

5.1.4 Architectural Evaluation

The system was evaluated from an architectural perspective, focusing on key quality attributes including responsiveness, reliability, modifiability, and traceability.

First, the system demonstrates effective real-time data processing through an event-driven architecture. Sensor nodes continuously publish data, which are immediately propagated through the MQTT broker to subscribed components. This eliminates the need for polling and reduces communication latency, making the system suitable for time-sensitive monitoring scenarios.

Second, reliability is enhanced through the use of MQTT QoS 1, which ensures at-least-once message delivery.¹⁴ This reduces the likelihood of data loss in unstable network conditions. Additionally, persistent storage in a MySQL database enables historical analysis and supports audit requirements, thereby improving traceability and data

¹¹*DHT11 SIP packaged temperature and humidity sensor*. 2020. URL: <http://www.aosong.com/en/products-21.html>.

¹²*ESP32: a feature-rich MCU with integrated wi-fi and bluetooth connectivity*. 2024. URL: <https://www.espressif.com/en/products/socs/esp32>.

¹³*Raspberry pi*. 2024. URL: <https://www.raspberrypi.com/>.

¹⁴Ebleme, “Performance analysis of internet of things application layer communication protocols”.

integrity.

Third, the system supports reactive behavior through an event-triggered alerting mechanism. When sensor readings exceed predefined thresholds, notifications are generated automatically. This reduces dependence on manual monitoring and significantly improves response time in critical situations.

Modifiability is achieved through a layered architecture that separates sensing, communication, and application logic. Configuration parameters such as storage units and temperature thresholds can be modified without affecting lower-level components, demonstrating strong separation of concerns and adaptability.

However, the architecture also exhibits certain limitations. The centralized deployment of the MQTT broker, database, and backend services on a single Raspberry Pi introduces a single point of failure. This reduces fault tolerance and may impact system availability. Furthermore, the lack of broker clustering or distributed processing limits scalability in large-scale industrial environments.

5.1.5 Architectural Limitations

Despite its effectiveness, the system presents several architectural constraints. The centralized design simplifies deployment but introduces reliability risks due to the absence of redundancy. As the number of connected devices increases, the single broker instance may become a performance bottleneck.

Additionally, while MQTT ensures efficient communication, the current implementation does not incorporate advanced security mechanisms such as SSL/TLS encryption or authentication frameworks. Data integrity guarantees are also limited to QoS-level assurances, without additional validation or fault-tolerant mechanisms.

These limitations suggest that the current architecture is most suitable for small-to-medium-scale deployments. Future improvements may include distributed broker architectures, enhanced security mechanisms, and stronger data consistency models.

5.1.6 Conclusion

This case study demonstrates the effectiveness of an MQTT-based IoT system for real-time monitoring of cold storage environments. By integrating wireless sensors, a cen-

tralized server, and a web-based application, the system enables continuous monitoring, automated data logging, and real-time alerting.

Compared to traditional manual monitoring methods, the proposed solution significantly improves reliability, reduces human error, and enhances cold chain traceability. With further enhancements in scalability, security, and predictive analytics, the system has strong potential for broader industrial adoption and long-term operational optimization.

5.2 Case Study 2: Secure MQTT 5.0-Based OTA Architecture

5.2.1 Industrial Problem

Over-the-air (OTA) updating enables remote firmware and key updates in IoT systems and is widely adopted in platforms such as AWS IoT. In many existing broker-based architectures, the MQTT broker acts not only as a message relay but also as the OTA update server. In such systems, application managers often delegate firmware signing authority to the broker for convenience.

While this approach simplifies system management, it introduces significant security and privacy risks, particularly when the broker is operated by a third party or is potentially untrusted. A compromised or curious broker may gain access to sensitive data such as firmware images, cryptographic keys, or update metadata. Given that many IoT deployments rely on external cloud-based MQTT brokers, ensuring confidentiality and integrity of OTA updates becomes a critical challenge.

To address these concerns, the referenced work proposes a novel MQTT 5.0-based OTA architecture that establishes an end-to-end (E2E) secure communication channel between the application manager and IoT devices. This design enables full managerial control over the update process while significantly improving security and privacy guarantees.¹⁵

¹⁵Hung-Yu Chien and Nian-Zu Wang. “A novel MQTT 5.0-based over-the-air updating architecture facilitating stronger security”. In: ().

5.2.2 Background: MQTT 5.0 Enhancements

MQTT is a lightweight publish–subscribe protocol consisting of three main entities: publishers, subscribers, and a broker. Messages are organized into topics, and the broker forwards published messages to all subscribed clients.

The MQTT 5.0 standard introduces several enhancements that improve flexibility and security:

Enhanced Authentication: MQTT 5.0 provides an extensible authentication framework that allows custom authentication mechanisms. It introduces fields such as *auth_id* and *auth_data*, along with an AUTH control packet to support multi-step authentication exchanges.¹⁶

User Properties: User properties enable the transmission of custom metadata as UTF-8 key–value pairs within MQTT messages. This feature allows additional contextual information (e.g., client identity or certificates) to be embedded directly in communication flows.¹⁷

Request–Response Model: MQTT 5.0 introduces the *ResponseTopic* property, enabling request–response communication patterns. Combined with correlation data, this feature simplifies synchronous interactions over a fundamentally asynchronous protocol.¹⁸

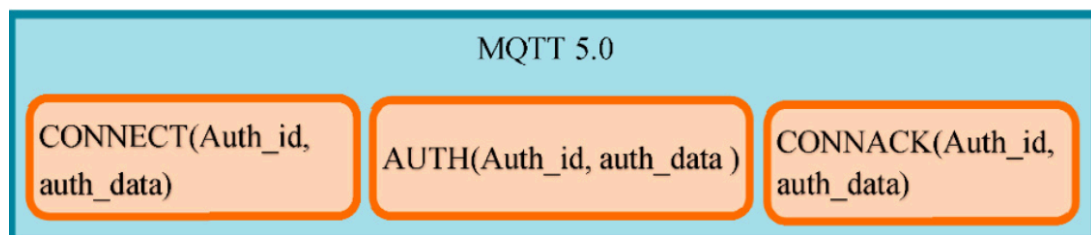


Figure 5.3: MQTT 5.0

These features provide the foundation for implementing more advanced and secure communication models in IoT systems.

¹⁶Enhanced authentication—MQTT 5 essentials part 11. 2022. URL: <https://www.hivemq.com/blog/mqtt5-essentials-part11-enhanced-authentication/>.

¹⁷User properties—MQTT 5 essentials part 6. 2022. URL: <https://www.hivemq.com/blog/mqtt5-essentials-part6-user-properties/>.

¹⁸Understanding and using MQTT v5 request response. 2022. URL: <http://www.steves-internet-guide.com/mqttv5-request-response/>.

5.2.3 Traditional OTA Architecture

In conventional MQTT-based OTA systems, such as those integrated with AWS IoT and FreeRTOS, the cloud platform manages both communication and update processes.

The OTA workflow typically consists of the following steps:

- The application manager prepares and uploads firmware images to the cloud.
- The platform optionally signs firmware on behalf of the manager.
- An OTA job is created and distributed via MQTT topics.
- IoT devices receive update notifications and download firmware using MQTT and HTTP.

While efficient, this architecture places significant trust in the broker and cloud infrastructure. Since firmware and keys may be processed or stored by the broker, confidentiality and integrity cannot be fully guaranteed in untrusted environments.¹⁹

5.2.4 Proposed MQTT 5.0 OTA Model

The proposed architecture introduces an end-to-end (E2E) secure OTA model that separates broker-level communication from application-level security. The system consists of three phases:

Outer-Channel Establishment

In this phase, both the application manager and IoT devices authenticate with the MQTT broker using certificates. A TLS-secured connection is established between each client and the broker, ensuring secure transmission over the network. However, the broker can still access message contents in this channel.

Inner-Channel Establishment

To prevent broker-level data exposure, an additional E2E channel is established between the manager and each IoT device. Using MQTT topics and the request–response mecha-

¹⁹Chien and Wang, “A novel MQTT 5.0-based over-the-air updating architecture facilitating stronger security”.

nism, both parties exchange certificates and perform a Diffie–Hellman key exchange to derive a shared secret (E2E key).

All sensitive data are encrypted using this shared key, ensuring that only the communicating endpoints can access the message contents, even though messages still pass through the broker.

P1.Cert, S1.Cert	P1.Cert denotes Publisher P1’s certificate; S1.Cert denotes S1’s certificate. P1.Cert has the public key as g^a ; S1.Cert has its public key as g^b . Here we eliminate the specification of the underlying fields, and any secure fields like Elliptic Curve Cryptographies could be used.
$E2E_{key}$	The End-to-End session keys between publisher-subscriber E2E session key. Here, $E2E_{key} = g^{ab}$.
$Enc_{key}[], Dec_{key}[]$	Encryption/Decryption using the key key .

Figure 5.4: The notations.

OTA Update Phase

During the update phase, the application manager encrypts firmware images, certificates, and private keys using the E2E key before publishing them to device-specific topics. Devices decrypt and process updates locally, ensuring confidentiality and integrity of update data.

This design ensures that:

- The broker cannot access sensitive OTA data
- Firmware authenticity is preserved
- End-to-end confidentiality is maintained

5.2.5 Topic Design and Communication Workflow

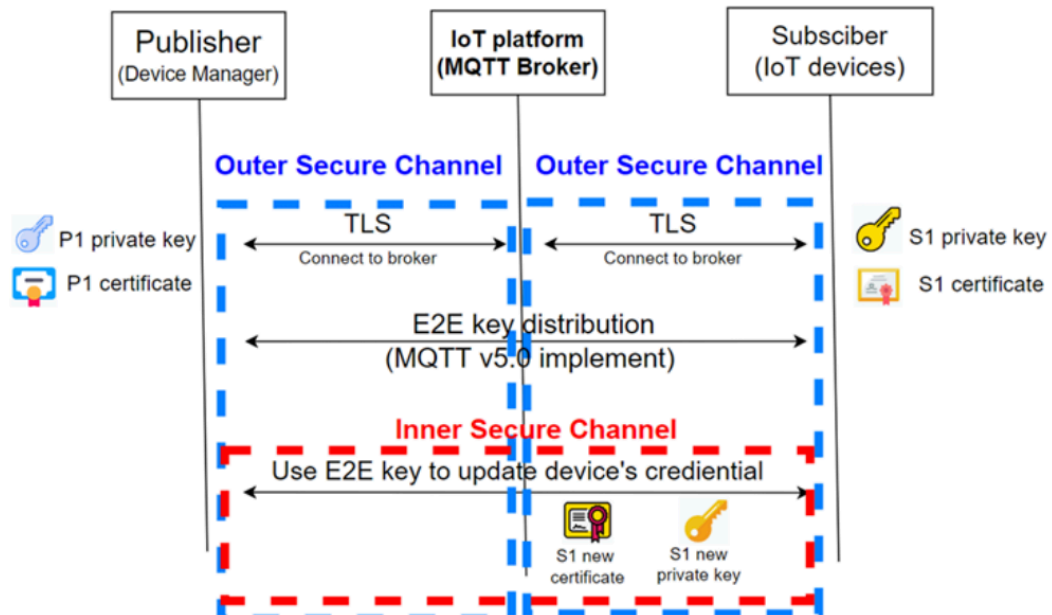


Figure 5.5: The two channels of the OTA model

The system uses a structured topic hierarchy to coordinate communication between the manager and devices

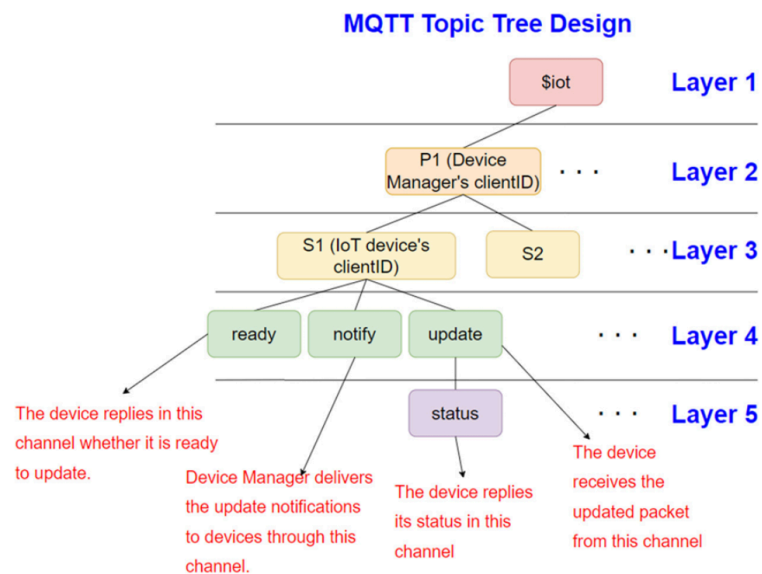


Figure 5.6: Topic Tree Design

The manager subscribes to wildcard topics such as:

`$iot/P1/+/ready`

Each device subscribes to dedicated topics such as:

`$iot/P1/S1/notify`

Through these topics:

- The manager distributes certificates and update instructions
- Devices respond using specified *ResponseTopic* fields
- Both parties derive a shared E2E key using Diffie–Hellman exchange

Encrypted update data are then transmitted through:

`$iot/P1/S1/update`

This structured topic design enables scalable and secure communication across multiple devices while preserving MQTT compatibility.

The inner-channel establishment phase

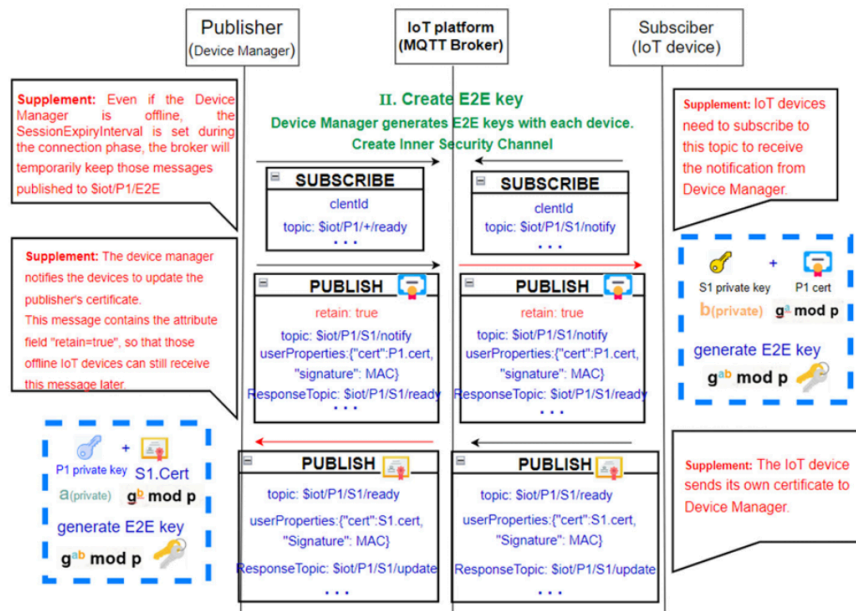


Figure 5.7: The message interactions during the End2End channel building

The OTA update phase

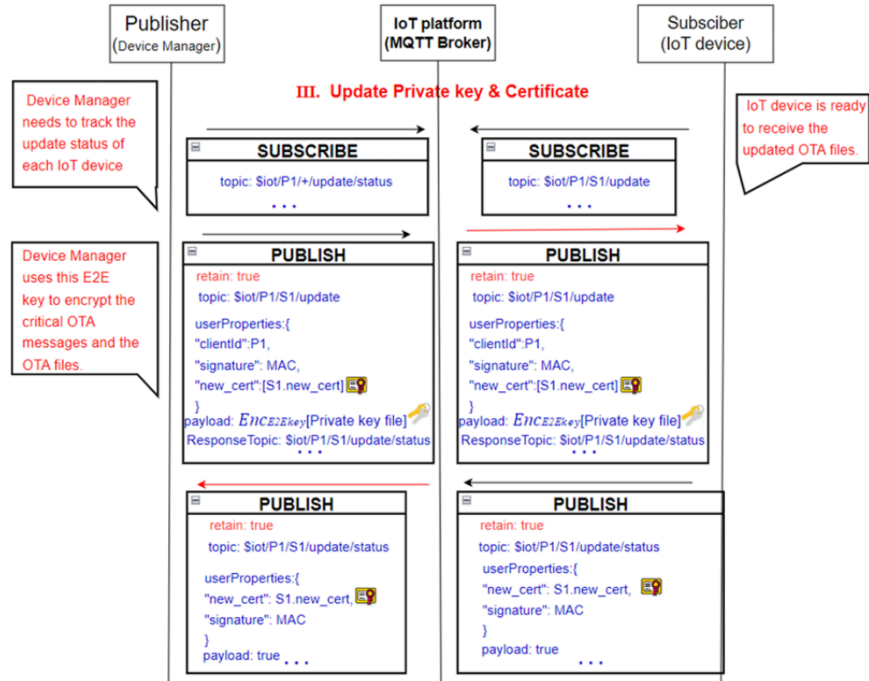


Figure 5.8: The message flows of the OTA update phase

5.2.6 Security Analysis

The proposed architecture adopts a two-layer security model:

Outer Layer (TLS): Provides authentication and secure communication between clients and the broker. It ensures protection against external attackers during data transmission.

Inner Layer (E2E Encryption): Protects sensitive OTA data using encryption based on a shared E2E key derived via Diffie–Hellman key exchange. This prevents data leakage to the broker and ensures confidentiality even in untrusted environments.

The system guarantees:

- **Confidentiality:** Only endpoints can decrypt OTA data
- **Integrity:** Cryptographic protection prevents tampering
- **Authentication:** Certificates validate communicating parties

The security of the model relies on established cryptographic primitives such as

ECDH and ECDSA, and the computational hardness of the Diffie–Hellman problem.²⁰

5.2.7 Performance Evaluation

The system performance is evaluated across three phases:

Outer-channel phase: Equivalent to standard TLS-based MQTT communication, introducing no additional overhead.

Inner-channel phase: Requires approximately six MQTT message exchanges per device. The primary overhead comes from certificate transmission (2–4 KB) and a single Diffie–Hellman computation, which remains feasible for resource-constrained IoT devices.

OTA update phase: Also involves around six MQTT interactions, including encrypted firmware delivery and status reporting. Since most messages are small, the system maintains efficient communication.

Compared to traditional cloud-based OTA solutions, the proposed architecture reduces control overhead and improves energy efficiency, which is critical for IoT environments.²¹

5.2.8 Architectural Evaluation

From an architectural perspective, the proposed model demonstrates several strengths. The introduction of an E2E communication layer enhances security without modifying the underlying MQTT infrastructure, preserving compatibility with existing systems. The separation of outer and inner channels reflects a layered security design that improves modularity and resilience.

Furthermore, the use of MQTT 5.0 features such as user properties and response topics enables flexible communication patterns while maintaining the benefits of publish–subscribe systems.

However, the architecture introduces additional complexity in key management and communication workflows. The requirement for multiple message exchanges and cryptographic operations may increase implementation difficulty and latency in large-scale

²⁰Chien and Wang, “A novel MQTT 5.0-based over-the-air updating architecture facilitating stronger security”.

²¹Chien and Wang, “A novel MQTT 5.0-based over-the-air updating architecture facilitating stronger security”.

deployments.

Overall, the architecture represents a significant improvement over traditional broker-centric OTA systems by addressing critical security limitations while maintaining scalability and efficiency.

5.2.9 Conclusion

This case study demonstrates how MQTT 5.0 can be extended to support secure OTA updates through end-to-end encryption mechanisms. By introducing a dual-channel architecture and leveraging modern MQTT features, the proposed system eliminates reliance on trusted brokers for sensitive operations.

The approach significantly enhances privacy and security while maintaining compatibility with existing IoT platforms. With further optimization in key management and scalability, this architecture provides a strong foundation for secure firmware updates in large-scale IoT deployments

Chapter 6

Future Directions

6.1 Fog Computing

Fog computing has emerged as an extension of the cloud computing paradigm that aims to push computation, storage, and networking capabilities closer to the edge of the network, thereby bridging the gap between resource-constrained IoT devices and centralized cloud infrastructures.¹ Rather than relying solely on distant data centers, fog architectures introduce intermediate nodes—such as gateways, edge servers, and network devices—that are capable of performing localized processing. This architectural shift is particularly motivated by the growing demands of IoT applications that require low-latency responses, real-time analytics, and efficient bandwidth utilization.

In contrast to traditional cloud-centric models, where raw sensor data is continuously transmitted to remote servers for processing, fog computing enables data to be filtered, aggregated, and analyzed closer to its source. This localized processing reduces end-to-end latency and alleviates network congestion, which is especially important in scenarios involving high-frequency data streams or mission-critical operations.² Applications such as autonomous driving systems, industrial control systems, and remote health monitoring benefit significantly from this paradigm, as they require rapid decision-making and cannot tolerate delays introduced by long-distance communication.³

¹Flavio et al. Bonomi. “Fog computing and its role in the Internet of Things”. In: *MCC workshop*. 2012; Shanhe et al. Yi. “A survey of fog computing”. In: *Workshop on Mobile Big Data* (2015).

²Bonomi, “Fog computing and its role in the Internet of Things”; Mahadev Satyanarayanan. “The emergence of edge computing”. In: *Computer* (2017).

³Shi, “Edge computing: Vision and challenges”; Blessen et al. Varghese. “Challenges and opportunities in edge computing”. In: *IEEE Cloud Computing* (2016).

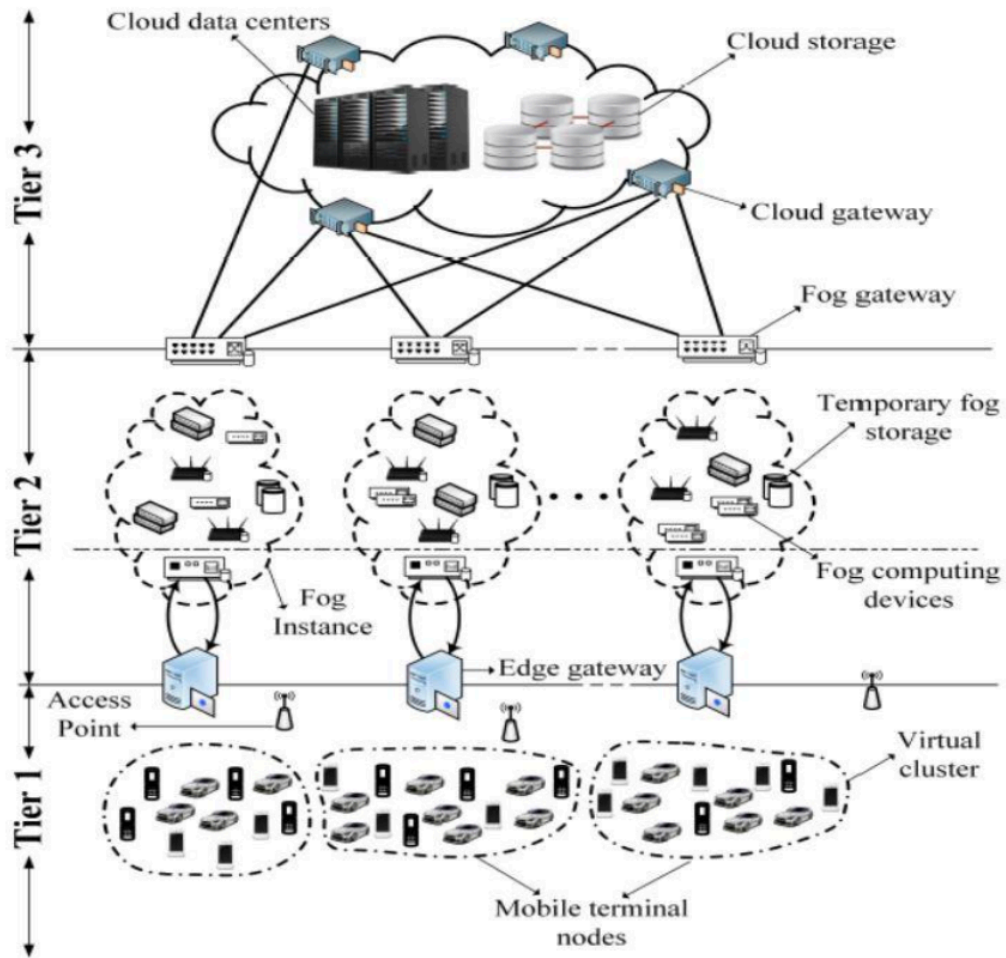


Figure 6.1: Fog Architecture

Another important advantage of fog computing lies in its ability to improve system scalability and flexibility. By distributing computational workloads across multiple fog nodes, the architecture avoids over-reliance on centralized cloud resources and reduces the likelihood of performance bottlenecks.⁴ This decentralized approach also allows systems to dynamically adapt to varying workloads and network conditions. Furthermore, processing data closer to where it is generated can enhance privacy and data protection, as sensitive information does not need to be transmitted to external cloud environments.⁵

Despite these advantages, fog computing introduces a number of challenges that complicate its adoption in large-scale IoT deployments. One of the primary concerns is the increased architectural complexity resulting from the distribution of computation across

⁴Yi, "A survey of fog computing".

⁵Ivan Stojmenovic and Sheng Wen. "Fog computing: A cloud to the ground support for smart things and machine-to-machine networks". In: *Telecommunication Networks* (2014).

heterogeneous and geographically dispersed nodes. Managing these resources efficiently requires sophisticated orchestration mechanisms capable of handling task placement, load balancing, and resource allocation in dynamic environments.⁶ Additionally, ensuring consistency and reliability across distributed nodes remains a non-trivial problem, particularly in the presence of network failures or intermittent connectivity. Security is another critical concern, as the expanded attack surface introduced by multiple fog nodes increases the risk of vulnerabilities and unauthorized access.⁷ Addressing these challenges is essential for realizing the full potential of fog computing in future IoT systems.

6.2 Serverless Architecture

Serverless architecture, often referred to as Function-as-a-Service (FaaS), represents a significant evolution in cloud computing that abstracts away infrastructure management and allows developers to focus solely on application logic.⁸ In this model, applications are decomposed into small, stateless functions that are executed in response to specific events. The underlying cloud platform is responsible for provisioning resources, scaling execution, and managing runtime environments, thereby reducing operational complexity and enabling rapid application development.

This paradigm is particularly well-suited to IoT systems, which are inherently event-driven in nature. IoT applications typically involve continuous streams of data generated by sensors and devices, where processing is triggered by events such as threshold violations, state changes, or external inputs. Serverless computing aligns naturally with this model by allowing functions to be invoked on demand, facilitating efficient and scalable data processing pipelines.⁹ As a result, IoT developers can build highly responsive systems without the need to provision and manage dedicated servers.

⁶Varghese, “Challenges and opportunities in edge computing”; Kirak et al. Hong. “Mobile fog: A programming model for large-scale applications on the Internet of Things”. In: 2013.

⁷Shi, “Edge computing: Vision and challenges”.

⁸Ioana et al. Baldini. “Serverless computing: Current trends and open problems”. In: *Research Advances in Cloud Computing* (2017); Eric et al. Jonas. “Cloud programming simplified: A Berkeley view on serverless computing”. In: *arXiv preprint arXiv:1902.03383* (2019).

⁹Marco et al. Carvalho. “Serverless computing: A survey”. In: *Computer Communications* (2020); Adam Hall and Umakishore Ramachandran. “Serverless computing for internet of things: a survey”. In: *IEEE Internet of Things Journal* (2019).

A key advantage of serverless architecture is its elasticity. Resources are allocated dynamically based on workload, allowing systems to scale seamlessly from handling a few events to processing millions of requests. This pay-per-use model also improves cost efficiency, as users are charged only for the actual execution time of functions rather than for idle resources.¹⁰ Moreover, serverless platforms provide built-in integration with cloud services, enabling rapid development of data processing, analytics, and integration workflows.¹¹

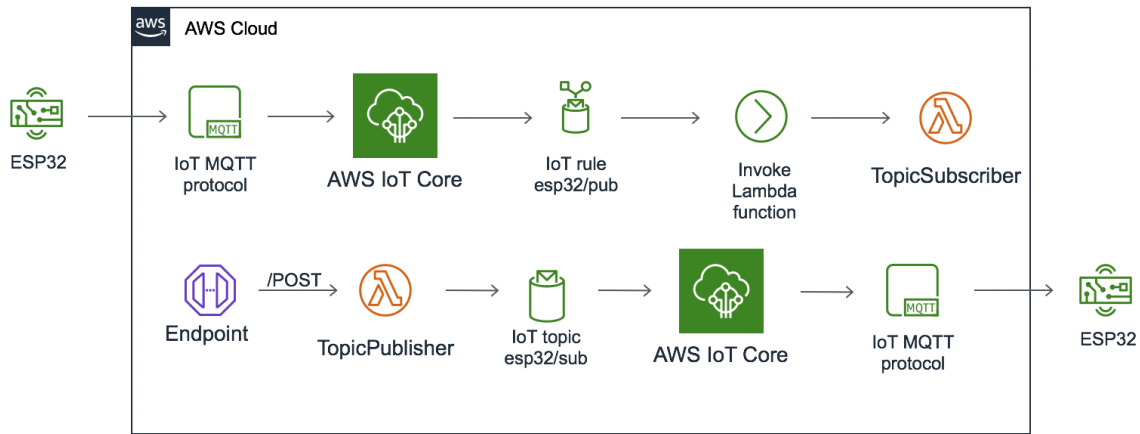


Figure 6.2: Amazon’s Serverless IOT System with ESP32

However, the adoption of serverless computing in IoT systems is not without limitations. One notable issue is the presence of cold start latency, where functions experience delays during initialization, potentially impacting time-sensitive applications.¹² Additionally, serverless platforms impose constraints on execution duration and resource usage, making them less suitable for long-running or computationally intensive tasks.¹³ The stateless nature of serverless functions further complicates the management of application state, requiring external storage systems and careful architectural design.¹⁴

Recent research efforts have explored the integration of serverless computing with edge and fog environments, aiming to combine the benefits of low-latency processing

¹⁰Jonas, “Cloud programming simplified: A Berkeley view on serverless computing”.

¹¹Baldini, “Serverless computing: Current trends and open problems”.

¹²Liang et al. Wang. “Peeking behind the curtains of serverless platforms”. In: *Usenix etc.* 2018.

¹³Gareth McGrath and Paul Brenner. “Serverless computing: Design, implementation, and performance”. In: *IEEE Distributed Systems Online* (2017).

¹⁴Baldini, “Serverless computing: Current trends and open problems”.

with the scalability of cloud-based execution.¹⁵ Such hybrid architectures enable functions to be deployed closer to IoT devices while still leveraging centralized cloud resources when necessary. This approach represents a promising direction for future IoT systems, as it seeks to balance performance, scalability, and resource efficiency in increasingly complex and distributed environments.

¹⁵Jonas, “Cloud programming simplified: A Berkeley view on serverless computing”; Blesson Varghese and Rajkumar Buyya. “Next generation cloud computing: New trends and research directions”. In: *Future Generation Computer Systems* (2018).

Chapter 7

Conclusion

This report has examined the role of software architecture in addressing the challenges of modern IoT systems, including scalability, heterogeneity, real-time processing, and resource constraints. Traditional approaches such as N-tier and Service-Oriented Architecture provide a strong foundation for system organization and modularity, but are often limited in highly distributed and dynamic environments.

More recent architectural styles, including microservices and event-driven architectures, offer improved flexibility, scalability, and responsiveness by enabling decentralized and asynchronous interactions. Communication protocols such as MQTT, CoAP, and DDS further support these architectures by providing efficient and adaptable data exchange mechanisms tailored to different IoT requirements.

Additionally, emerging paradigms such as fog computing and serverless architecture demonstrate how computation can be distributed across edge and cloud environments to optimize performance, latency, and resource utilization.

Overall, no single architectural approach is sufficient to address all IoT challenges. Instead, modern IoT systems increasingly adopt hybrid architectures that combine multiple paradigms. Such approaches enable systems to balance trade-offs between performance, scalability, complexity, and reliability, making them better suited for the evolving demands of large-scale and real-time IoT applications.

Chapter 8

Ethics Report: AI Uses

This report was developed with the assistance of AI-based tools (Overleaf AI) to support the writing and organization process. AI was primarily used to help refine language, improve clarity, and generate draft explanations for certain technical concepts. Specifically, AI assistance was utilized in drafting first version of sections related to N-tier Architecture. The AI was also used to suggest structural improvements and formatting in Overleaf. However, all the content of the report was prepared and written by all the teammates of the group. No Generative AI tools were made to create content and ideas in the seminar. No AI tools were used to generate citations without verification, and all referenced materials were manually checked.

References

- [1] *A Review on Service Oriented Architectures for Internet of Things (IoT)*. en-US. URL: <https://ieeexplore.ieee.org/document/8553767> (visited on 03/09/2026).
- [2] Salma H. Abdelwahed et al. “Survey of IoT multi-protocol gateways: Architectures, protocols and cybersecurity”. In: *Internet of Things* 33 (Sept. 2025), p. 101703. ISSN: 2542-6605. DOI: 10.1016/j.iot.2025.101703. URL: <https://www.sciencedirect.com/science/article/pii/S2542660525002173> (visited on 02/25/2026).
- [3] Abir El Akhdar et al. “Exploring the Potential of Microservices in Internet of Things: A Systematic Review of Security and Prospects”. en. In: *Sensors* 24.20 (Oct. 2024). ISSN: 1424-8220. DOI: 10.3390/s24206771. URL: <https://www.mdpi.com/1424-8220/24/20/6771> (visited on 03/02/2026).
- [4] Shanhe Li et al. “Edge-oriented computing paradigms: a survey on architecture design and system management”. In: *ACM Computing Surveys* (2018).
- [5] Abdulelah Al Hanif and Mohammad Ilyas. “Effective Feature Engineering Framework for Securing MQTT Protocol in IoT Environments”. In: *Sensors (Basel, Switzerland)* 24.6 (Mar. 2024), p. 1782. ISSN: 1424-8220. DOI: 10.3390/s24061782. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10975182/> (visited on 03/02/2026).
- [6] Nawaz Ali et al. “Edge-cloud continuum driven industry 4.0”. In: *Procedia Computer Science*. 6th International Conference on Industry 4.0 and Smart Manufacturing 253 (Jan. 2025), pp. 2586–2594. ISSN: 1877-0509. DOI: 10.1016/j.

- procs.2025.01.318. URL: <https://www.sciencedirect.com/science/article/pii/S1877050925003266> (visited on 03/11/2026).
- [7] A. Alkan and İ. Kırbaş. “An investigation of master’s theses written on the internet of things (IoT) in turkey: a meta-analysis study”. In: *Journal of Engineering Sciences and Design* 10.1 (2022), pp. 318–327. DOI: 10.21923/jesd.955660.
 - [8] Nouf Saeed Alotaibi et al. “Secure Enhancement for MQTT Protocol Using Distributed Machine Learning Framework”. In: *Sensors (Basel, Switzerland)* 24.5 (Mar. 2024), p. 1638. ISSN: 1424-8220. DOI: 10.3390/s24051638. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10935133/> (visited on 03/02/2026).
 - [9] Amazon. *Amazon robotics*. 2023.
 - [10] *An Introduction to Low Power IoT*. en. URL: <https://www.particle.io/iot-guides-and-resources/low-power-iot/> (visited on 03/02/2026).
 - [11] Tajkia Nuri Ananna and Munshi Saifuzzaman. “Introduction to IoT”. In: vol. 1169. arXiv:2312.06689 [cs]. 2024, pp. 1–49. DOI: 10.1007/978-981-97-5624-7_1. URL: <http://arxiv.org/abs/2312.06689> (visited on 02/17/2026).
 - [12] Luigi Atzori, Antonio Iera, and Giacomo Morabito. “The internet of things: a survey”. In: *Computer Networks* (2010).
 - [13] Luigi Atzori, Antonio Iera, and Giacomo Morabito. “The internet of things: a survey”. In: *Computer Networks* (2010).
 - [14] *AWS IoT Connected Home Solutions*. en-US. URL: <https://aws.amazon.com/iot/solutions/connected-home/> (visited on 03/11/2026).
 - [15] Ioana et al. Baldini. “Serverless computing: Current trends and open problems”. In: *Research Advances in Cloud Computing* (2017).
 - [16] Pallavi S. Bangare and Kishor P. Patil. “Enhancing MQTT security for internet of things: Lightweight two-way authorization and authentication with advanced security measures”. In: *Measurement: Sensors* 33 (June 2024), p. 101212. ISSN: 2665-9174. DOI: 10.1016/j.measen.2024.101212. URL: <https://www.sciencedirect.com/science/article/pii/S2665917424001880> (visited on 03/02/2026).

- [17] Pallavi Sunil Bangare and Kishor P. Patil. “Security Issues and Challenges in Internet of Things (IoT) System”. In: *2022 2nd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE)*. Apr. 2022, pp. 91–94. DOI: 10.1109/ICACITE53722.2022.9823709. URL: <https://ieeexplore.ieee.org/document/9823709> (visited on 02/14/2026).
- [18] Gunjan Beniwal and Anita Singhrova. ““A systematic literature review on IoT gateways””. In: *Journal of King Saud University - Computer and Information Sciences* 34.10, Part B (Nov. 2022), pp. 9541–9563. ISSN: 1319-1578. DOI: 10.1016/j.jksuci.2021.11.007. URL: <https://www.sciencedirect.com/science/article/pii/S1319157821003219> (visited on 02/25/2026).
- [19] Luca Bixio et al. “A Flexible IoT Stream Processing Architecture Based on Microservices”. en. In: *Information* 11.12 (Dec. 2020). ISSN: 2078-2489. DOI: 10.3390/info11120565. URL: <https://www.mdpi.com/2078-2489/11/12/565> (visited on 03/11/2026).
- [20] Gedare Bloom et al. “Design patterns for the industrial Internet of Things”. en. In: *2018 14th IEEE International Workshop on Factory Communication Systems (WFCS)*. Imperia, Italy: IEEE, June 2018, pp. 1–10. ISBN: 978-1-5386-1066-4. DOI: 10.1109/WFCS.2018.8402353. URL: <https://ieeexplore.ieee.org/document/8402353/> (visited on 02/14/2026).
- [21] Flavio et al. Bonomi. “Fog computing and its role in the Internet of Things”. In: *MCC workshop*. 2012.
- [22] *Building event-driven architectures with IoT sensor data | AWS Architecture Blog*. Section: Amazon Kinesis. Nov. 2022. URL: <https://aws.amazon.com/blogs/architecture/building-event-driven-architectures-with-iot-sensor-data/> (visited on 03/02/2026).
- [23] A. Böbrek, İ. Kırbaş, and K. Özcan. “Patient and infant monitoring and control system (HBİKS)”. In: *Mehmet Akif Ersoy University Journal of Graduate School of Natural and Applied Sciences* 8.Supplementary Issue 1 (2017), pp. 252–261. DOI: 10.29048/makufebed.341310.

- [24] M. R. Cansız. *IoT data protocols MQTT, CoAP, AMQP and HTTP comparison*. 2020.
- [25] Marco et al. Carvalho. “Serverless computing: A survey”. In: *Computer Communications* (2020).
- [26] *Challenges and benefits of the microservice architectural style*. en. URL: <https://developer.ibm.com/articles/challenges-and-benefits-of-the-microservice-architectural-style-part-1/> (visited on 03/11/2026).
- [27] Hung-Yu Chien and Nian-Zu Wang. “A novel MQTT 5.0-based over-the-air updating architecture facilitating stronger security”. In: ().
- [28] Cisco. *IoT fundamentals: Networking technologies, protocols, and use cases*. 2017.
- [29] claytonsiemens77. *N-tier Architecture Style - Azure Architecture Center*. en-us. URL: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier> (visited on 03/09/2026).
- [30] Stuart Corner. *Cisco IoT World Forum Reference Model for interoperability*. en-AU. July 2015. URL: <https://www.iotaustralia.org.au/2015/07/22/iotnewsglobal/cisco-iot-world-forum-reference-model-promises-interoperability/> (visited on 02/14/2026).
- [31] Resul Das and Muhammad Muhammad Inuwa. “A review on fog computing: Issues, characteristics, challenges, and potential applications”. In: *Telematics and Informatics Reports* 10 (June 2023), p. 100049. ISSN: 2772-5030. DOI: 10.1016/j.teler.2023.100049. URL: <https://www.sciencedirect.com/science/article/pii/S2772503023000099> (visited on 03/11/2026).
- [32] Abdulkadir Dauda, Olivier Flauzac, and Florent Nolot. “A Survey on IoT Application Architectures”. In: *Sensors (Basel, Switzerland)* 24.16 (Aug. 2024), p. 5320. ISSN: 1424-8220. DOI: 10.3390/s24165320. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11360728/> (visited on 03/11/2026).

- [33] Martina De Sanctis, Henry Muccini, and Karthik Vaidhyanathan. “A User-driven Adaptation Approach for Microservice-based IoT Applications”. en. In: *Proceedings of the 11th International Conference on the Internet of Things*. St.Gallen Switzerland: ACM, Nov. 2021, pp. 48–56. ISBN: 978-1-4503-8566-4. DOI: 10 . 1145 / 3494322 . 3494329. URL: <https://dl.acm.org/doi/10.1145/3494322.3494329> (visited on 03/02/2026).
- [34] *DHT11 SIP packaged temperature and humidity sensor*. 2020. URL: <http://www.aosong.com/en/products-21.html>.
- [35] M. A. Ebleme. “Performance analysis of internet of things application layer communication protocols”. MA thesis. Sakarya University, 2019.
- [36] Abir El Akhdar, Chafik Baidada, and Ali Kartit. “Adaptability of Microservices Architecture in IoT Systems : A Comprehensive Review”. In: *Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security*. NISS '24. New York, NY, USA: Association for Computing Machinery, Aug. 2024, pp. 1–9. ISBN: 979-8-4007-0929-6. DOI: 10 . 1145 / 3659677 . 3659734. URL: <https://dl.acm.org/doi/10.1145/3659677.3659734> (visited on 03/11/2026).
- [37] Abir El Akhdar, Chafik Baidada, and Ali Kartit. “Adaptability of Microservices Architecture in IoT Systems : A Comprehensive Review”. In: *Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security*. NISS '24. New York, NY, USA: Association for Computing Machinery, Aug. 2024, pp. 1–9. ISBN: 979-8-4007-0929-6. DOI: 10 . 1145 / 3659677 . 3659734. URL: <https://doi.org/10.1145/3659677.3659734> (visited on 03/11/2026).
- [38] *Enhanced authentication—MQTT 5 essentials part 11*. 2022. URL: <https://www.hivemq.com/blog/mqtt5-essentials-part11-enhanced-authentication/>.
- [39] *ESB middleware - Google Search*. URL: <https://www.oracle.com/technical-resources/articles/middleware/soa-ind-soa-esb.html> (visited on 03/09/2026).

- [40] *ESP32: a feature-rich MCU with integrated wi-fi and bluetooth connectivity*. 2024. URL: <https://www.espressif.com/en/products/socs/esp32>.
- [41] *EVENT-DRIVEN ARCHITECTURE BROKER TOPOLOGY | GUARDIAN - An Opinionated Inductive Guidance Framework for Digital Transformation*. URL: <https://guardian-framework.github.io/framework/event-driven-architecture-broker-topology.html> (visited on 03/02/2026).
- [42] “Event-Driven vs. API-Driven: A Strategic Architectural Paradigm Choice”. en. In: *ResearchGate* (Dec. 2025). DOI: 10.22399/ijcesen.4458. URL: https://www.researchgate.net/publication/398545188_Event-Driven_vs_API-Driven_A_Strategic_Architectural_Paradigm_Choice (visited on 03/02/2026).
- [43] T. Evgen. *RFID and internet of things based supply chain information management*. 2017.
- [44] Mark Richards Ford Neal. *Fundamentals of Software Architecture*. en. ISBN: 978-1-4920-4344-7. URL: <https://www.oreilly.com/library/view/fundamentals-of-software/9781492043447/> (visited on 02/17/2026).
- [45] Erich Gamma, ed. *Design patterns: elements of reusable object-oriented software*. Addison-Wesley professional computing series. Reading, Mass: Addison-Wesley, 1995. ISBN: 978-0-201-63361-0.
- [46] Joao Giao et al. “A Framework for Service-Oriented Architecture (SOA)-Based IoT Application Development”. en. In: *Processes* 10.9 (Sept. 2022), p. 1782. ISSN: 2227-9717. DOI: 10.3390/pr10091782. URL: <https://www.mdpi.com/2227-9717/10/9/1782> (visited on 03/09/2026).
- [47] João Giao et al. “A framework for service-oriented architecture (SOA)-based IoT application development”. In: *Processes* 10.9 (2022), p. 1782. DOI: 10.3390/pr10091782. URL: <https://doi.org/10.3390/pr10091782>.
- [48] Jayavardhana Gubbi et al. *Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions*. arXiv:1207.0203 [cs]. July 2012. DOI: 10.48550/arXiv.1207.0203. URL: <http://arxiv.org/abs/1207.0203> (visited on 02/25/2026).

- [49] Jayavardhana et al. Gubbi. “Internet of Things (IoT): A vision, architectural elements, and future directions”. In: *Future Generation Computer Systems* (2013).
- [50] Harshit Gupta and Ajay Kumar Bharti. *Fog Computing& IoT: Overview, Architecture and Applications*. arXiv:2304.08302 [cs]. Apr. 2023. DOI: 10.48550/arXiv.2304.08302. URL: <http://arxiv.org/abs/2304.08302> (visited on 03/11/2026).
- [51] Adam Hall and Umakishore Ramachandran. “Serverless computing for internet of things: a survey”. In: *IEEE Internet of Things Journal* (2019).
- [52] Moneer Helu, Thomas Hedberg, and Allison Barnard Feeney. “Reference architecture to integrate heterogeneous manufacturing systems for the digital thread”. In: *CIRP journal of manufacturing science and technology* 19 (2017), 10.1016/j.cirpj.2017.04.002. ISSN: 1755-5817. DOI: 10.1016/j.cirpj.2017.04.002. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC6459417/> (visited on 02/14/2026).
- [53] Kirak et al. Hong. “Mobile fog: A programming model for large-scale applications on the Internet of Things”. In: 2013.
- [54] “IEEE Recommended Practice for Architectural Description for Software-Intensive Systems”. In: *IEEE Std 1471-2000* (Oct. 2000), pp. 1–30. DOI: 10.1109/IEEESTD.2000.91944. URL: <https://ieeexplore.ieee.org/document/875998> (visited on 02/14/2026).
- [55] IEEE Standards Association. *IEEE 2413-2019: Standard for an architectural framework for the internet of things (IoT)*. Tech. rep. 2019.
- [56] Corrado Innamorati et al. “The pulse of MQTT in the wild: A large-scale traffic analysis”. en. In: *Computer Networks* 274 (Jan. 2026), p. 111845. ISSN: 13891286. DOI: 10.1016/j.comnet.2025.111845. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1389128625008114> (visited on 03/02/2026).
- [57] *Internet of Things - an overview | ScienceDirect Topics*. URL: https://www.sciencedirect.com/topics/computer-science/internet-of-things?__cf_chl_tk=rZj4oj48zwWVInYRqFpak7u9_NXyAMloJXc1vhk.I7A-1773386612-1.0.1.1-xSppxPogSd5G0eTkVEMJG8PoI3pKNZrM4XDSr8AQke0 (visited on 02/17/2026).

- [58] ISO/IEC JTC 1/SC 41. *ISO/IEC 30141:2024 internet of things (IoT) — reference architecture*. Tech. rep. International Organization for Standardization, 2024.
- [59] ITU-T. *Recommendation Y.4228: Requirements and framework of industrial IoT (IIoT) infrastructure for smart manufacturing*. Tech. rep. 2024.
- [60] Aldona Jarašūnienė, Kristina Čižiūnienė, and Audrius Čereška. “Research on Impact of IoT on Warehouse Management”. In: *Sensors (Basel, Switzerland)* 23.4 (Feb. 2023), p. 2213. ISSN: 1424-8220. DOI: 10 . 3390 / s23042213. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9967450/> (visited on 02/14/2026).
- [61] Eric et al. Jonas. “Cloud programming simplified: A Berkeley view on serverless computing”. In: *arXiv preprint arXiv:1902.03383* (2019).
- [62] H. Karaoglu, T. Kabak, and I. Kirbas. “—”. In: *Turkish Journal of Health Science and Life* 7.1 (2024), pp. 33–45.
- [63] B. Keleş and G. Ova. “Use of information technologies in food supply chain management”. In: *Adnan Menderes University Journal of Faculty of Agriculture* 17.1 (2020), pp. 137–143. DOI: 10.25308/aduziraat.695732.
- [64] Martin Kleppmann. *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O’Reilly Media, 2017.
- [65] Olivera Kotevska, Jordan Johnson, and A. Gilad Kusne. “Analyzing Data Privacy for Edge Systems”. en. In: *NIST* (July 2022). Last Modified: 2025-08-29T11:34-04:00. URL: <https://www.nist.gov/publications/analyzing-data-privacy-edge-systems> (visited on 03/11/2026).
- [66] *Law on veterinary services, plant health, food and feed (law no: 5996)*. 2010.
- [67] Apostolos Lazidis, Konstantinos Tsakos, and Euripides G. M. Petrakis. “Publish–Subscribe approaches for the IoT and the cloud: Functional and performance evaluation of open-source systems”. In: *Internet of Things* 19 (Aug. 2022), p. 100538. ISSN: 2542-6605. DOI: 10.1016/j.iot.2022.100538. URL: <https://www.sciencedirect.com/science/article/pii/S2542660522000403> (visited on 02/14/2026).

- [68] Apostolos Lazidis, Konstantinos Tsakos, and Euripides G. M. Petrakis. “Publish–Subscribe approaches for the IoT and the cloud: Functional and performance evaluation of open-source systems”. In: *Internet of Things* 19 (Aug. 2022), p. 100538. ISSN: 2542-6605. DOI: 10.1016/j.iot.2022.100538. URL: <https://www.sciencedirect.com/science/article/pii/S2542660522000403> (visited on 03/02/2026).
- [69] Keila Lima et al. *Evaluation of MQTT Bridge Architectures in a Cross-Organizational Context*. arXiv:2501.14890 [cs]. Jan. 2025. DOI: 10.48550/arXiv.2501.14890. URL: <http://arxiv.org/abs/2501.14890> (visited on 03/02/2026).
- [70] Aleesha M and Laseena. *MQTT Protocol for Resource Constrained IoT Applications : A Review*. en. SSRN Scholarly Paper. Rochester, NY, Aug. 2022. DOI: 10.2139/ssrn.4299372. URL: <https://papers.ssrn.com/abstract=4299372> (visited on 03/02/2026).
- [71] Parushi Malhotra et al. “Internet of things: Evolution, concerns and security challenges”. In: *Sensors* 21.5 (2021), p. 1809. DOI: 10.3390/s21051809.
- [72] *Manage Your Smart Home With Google Home | Google Home*. en. URL: <https://home.google.com/welcome/> (visited on 03/11/2026).
- [73] Gareth McGrath and Paul Brenner. “Serverless computing: Design, implementation, and performance”. In: *IEEE Distributed Systems Online* (2017).
- [74] Microsoft. *Azure IoT gateway documentation*. 2023.
- [75] Sugyan Kumar Mishra and Anirban Sarkar. “Service-oriented architecture for Internet of Things: A semantic approach”. In: *Journal of King Saud University - Computer and Information Sciences* 34.10, Part A (Nov. 2022), pp. 8765–8776. ISSN: 1319-1578. DOI: 10.1016/j.jksuci.2021.09.024. URL: <https://www.sciencedirect.com/science/article/pii/S1319157821002755> (visited on 02/14/2026).
- [76] *MQTT - The Standard for IoT Messaging*. URL: <https://mqtt.org/> (visited on 03/02/2026).

- [77] N. Naik. “Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP”. In: *IEEE international systems engineering symposium (ISSE)*. Vienna, Austria, 2017, pp. 1–7.
- [78] Naghmeh Niknejad et al. “Understanding Service-Oriented Architecture (SOA): A systematic literature review and directions for further investigation”. In: *Information Systems* 91 (July 2020), p. 101491. ISSN: 0306-4379. DOI: 10.1016/j.is.2020.101491. URL: <https://www.sciencedirect.com/science/article/pii/S0306437920300028> (visited on 02/14/2026).
- [79] NIST. *NIST special publication 800-183: Networks of things*. 2016.
- [80] OASIS. *MQTT version 3.1.1*. 2014.
- [81] Ruiqi Ouyang et al. “A Microservice and Serverless Architecture for Secure IoT System”. In: *Sensors (Basel, Switzerland)* 23.10 (May 2023), p. 4868. ISSN: 1424-8220. DOI: 10.3390/s23104868. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10220873/> (visited on 02/14/2026).
- [82] *OWASP MQTT Guide | OWASP Foundation*. en. URL: <https://owasp.org/www-project-mqtt-guide/> (visited on 03/02/2026).
- [83] Samodha Pallewatta, Vassilis Kostakos, and Rajkumar Buyya. “Microservices-based IoT Application Placement within Heterogeneous and Resource Constrained Fog Computing Environments”. In: *Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing*. UCC’19. New York, NY, USA: Association for Computing Machinery, Dec. 2019, pp. 71–81. ISBN: 978-1-4503-6894-0. DOI: 10.1145/3344341.3368800. URL: <https://dl.acm.org/doi/10.1145/3344341.3368800> (visited on 03/02/2026).
- [84] *Publish-Subscribe - Intro to Pub-Sub Messaging*. en. URL: <https://www.confluent.io/learn/publish-subscribe/> (visited on 03/02/2026).
- [85] Silvio Quincozes, Tubino Emilio, and Juliano Kazienko. “MQTT Protocol: Fundamentals, Tools and Future Directions”. In: *IEEE Latin America Transactions* 17.09 (Sept. 2019), pp. 1439–1448. ISSN: 1548-0992. DOI: 10.1109/TLA.2019.8931137. URL: <https://ieeexplore.ieee.org/abstract/document/8931137> (visited on 02/14/2026).

- [86] *Raspberry pi*. 2024. URL: <https://www.raspberrypi.com/>.
- [87] Ahmad-Atamli Reineh, Andrew J. Paverd, and Andrew P. Martin. *Trustworthy and Secure Service-Oriented Architecture for the Internet of Things*. arXiv:1606.01671 [cs]. June 2016. DOI: 10.48550/arXiv.1606.01671. URL: <http://arxiv.org/abs/1606.01671> (visited on 03/09/2026).
- [88] Mark Richards and Neal Ford. *Fundamentals of software architecture: An engineering approach*. O'Reilly Media, 2020. ISBN: 978-1-4920-4345-4.
- [89] Chris Richardson. *Microservices patterns: with examples in Java*. Shelter Island, New York: Manning Publications, 2019. ISBN: 978-1-61729-454-9.
- [90] Rodrigo Roman, Javier Lopez, and Masahiro Mambo. "Mobile edge computing, fog et al.: a survey and analysis of security threats and challenges". In: *Future Generation Computer Systems* 78 (2018), pp. 680–698. DOI: 10.1016/j.future.2016.11.009.
- [91] Md. Ruquzzaman et al. "A systematic review of recent advances in IoT-based sensor networks for warehouse management". In: *Results in Engineering* 29 (Mar. 2026), p. 109195. ISSN: 2590-1230. DOI: 10.1016/j.rineng.2026.109195. URL: <https://www.sciencedirect.com/science/article/pii/S259012302600238> (visited on 02/14/2026).
- [92] Nitol Saha et al. "Performance evaluation framework of MQTT client libraries for IoT applications in manufacturing". In: *Manufacturing Letters*. 52nd SME North American Manufacturing Research Conference (NAMRC 52) 41 (Oct. 2024), pp. 1237–1245. ISSN: 2213-8463. DOI: 10.1016/j.mfglet.2024.09.150. URL: <https://www.sciencedirect.com/science/article/pii/S221384632400213X> (visited on 03/02/2026).
- [93] Yatin Sapra. *What Role Do IoT Gateways Play In The Overall IoT Ecosystem?* en-US. Sept. 2023. URL: <https://www.hashstudioz.com/blog/what-role-do-iot-gateways-play-in-the-overall-iot-ecosystem/> (visited on 02/25/2026).

- [94] S. Saritha and V. Sarasvathi. “A study on application layer protocols used in IoT”. In: *International conference on circuits, controls, and communications (CCUBE)*. Bengaluru, India, 2017, pp. 155–159.
- [95] Mahadev Satyanarayanan. “The emergence of edge computing”. In: *Computer* (2017).
- [96] Weisong et al. Shi. “Edge computing: Vision and challenges”. In: *IEEE Internet of Things Journal* (2016).
- [97] Ahmad Showail et al. “An internet of secure and private things: A service-oriented architecture”. In: *Computers & Security* 120 (Sept. 2022), p. 102776. ISSN: 0167-4048. DOI: 10.1016/j.cose.2022.102776. URL: <https://www.sciencedirect.com/science/article/pii/S0167404822001717> (visited on 03/09/2026).
- [98] *Simplify the development of your IoT solutions with IoT architectures*. en. URL: <https://developer.ibm.com/articles/iot-lp201-iot-architectures/> (visited on 02/25/2026).
- [99] *SOA-PE: A Service-oriented Architecture for Planning and Execution in Cyber-physical Systems*. en. URL: <https://www.ericsson.com/en/reports-and-papers/research-papers/soa-pe-a-service-oriented-architecture-for-planning-and-execution-in-cyber-physical-systems> (visited on 03/09/2026).
- [100] Rute C. Sofia. “Latest trends on supporting an energy-efficient and resilient IoT-Edge-Cloud continuum”. In: *IEEE/ACM Distributed Systems Journal* (2024).
- [101] Prakhar Srivastav. *A Docker Tutorial for Beginners*. en-us. URL: <https://docker-curriculum.com/> (visited on 03/11/2026).
- [102] Ivan Stojmenovic and Sheng Wen. “Fog computing: A cloud to the ground support for smart things and machine-to-machine networks”. In: *Telecommunication Networks* (2014).
- [103] *subscribe | ACM Computing Surveys*. URL: <https://dl.acm.org/doi/10.1145/857076.857078> (visited on 03/02/2026).

- [104] EMQX Team. *CoAP Protocol: Features, Use Cases, Pros & Cons for IoT*. en. URL: <https://www.emqx.com/en/blog/coap-protocol> (visited on 03/02/2026).
- [105] NorDutch Technologies. *10 Most Common Smart Home Issues (and How to Fix Them)*. en-US. May 2024. URL: [//nordutch.com/10-most-common-smart-home-issues-and-how-to-fix-them/](https://nordutch.com/10-most-common-smart-home-issues-and-how-to-fix-them/) (visited on 03/11/2026).
- [106] thingsboard. *Retained Messages*. URL: <https://thingsboard.io/docs/mqtt-broker/user-guide/retained-messages/> (visited on 03/02/2026).
- [107] *Tiered versus tierless IoT stacks | Proceedings of the 10th International Conference on the Internet of Things*. en. URL: <https://dl.acm.org/doi/10.1145/3410992.3411002> (visited on 02/14/2026).
- [108] Timothy Murkomen. “Performance, privacy, and security issues of TCP/IP at the application layer: A comprehensive survey”. In: *GSC Advanced Research and Reviews* 18.3 (Mar. 2024), pp. 234–264. ISSN: 25824597. DOI: 10.30574/gscarr.2024.18.3.0106. URL: <https://gsconlinepress.com/journals/gscarr/content/performance-privacy-and-security-issues-tcpip-application-layer-comprehensive-survey> (visited on 03/02/2026).
- [109] Thai Minh Truong et al. “The Microservice Based Architecture for IoT Systems in Agriculture and Aquaculture”. en. In: *Proceedings of the 2024 14th International Conference on Information Communication and Management*. Paris France: ACM, Nov. 2024, pp. 22–28. ISBN: 979-8-4007-1747-5. DOI: 10.1145/3711609.3711613. URL: <https://dl.acm.org/doi/10.1145/3711609.3711613> (visited on 03/11/2026).
- [110] Thai Minh Truong et al. “The Microservice Based Architecture for IoT Systems in Agriculture and Aquaculture”. In: *Proceedings of the 2024 14th International Conference on Information Communication and Management*. ICICM ’24. New York, NY, USA: Association for Computing Machinery, Feb. 2025, pp. 22–28. ISBN: 979-8-4007-1747-5. DOI: 10.1145/3711609.3711613. URL: <https://dl.acm.org/doi/10.1145/3711609.3711613> (visited on 03/02/2026).

- [111] Thai Minh Truong et al. “The Microservice Based Architecture for IoT Systems in Agriculture and Aquaculture”. In: *Proceedings of the 2024 14th International Conference on Information Communication and Management*. ICICM '24. New York, NY, USA: Association for Computing Machinery, Feb. 2025, pp. 22–28. ISBN: 979-8-4007-1747-5. DOI: 10.1145/3711609.3711613. URL: <https://dl.acm.org/doi/10.1145/3711609.3711613> (visited on 03/11/2026).
- [112] Francesco Tusa et al. “Microservices and serverless functions—lifecycle, performance, and resource utilisation of edge based real-time IoT analytics”. en. In: *Future Generation Computer Systems* 155 (June 2024), pp. 204–218. ISSN: 0167739X. DOI: 10.1016/j.future.2024.02.006. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0167739X24000529> (visited on 03/02/2026).
- [113] *Understanding and using MQTT v5 request response*. 2022. URL: <http://www.steves-internet-guide.com/mqttv5-request-response/>.
- [114] Unknown. “Research on impact of IoT on warehouse management”. In: *Journal/Conference TBD* (2023).
- [115] Unknown. “A systematic review of recent advances in IoT-based sensor networks for warehouse management”. In: *Journal/Conference TBD* (2025).
- [116] *Use an event driven architecture in your IoT devices - Internet of Things (IoT) Lens*. URL: <https://docs.aws.amazon.com/wellarchitected/latest/iot-lens/use-an-event-driven-architecture-in-your-iot-devices.html> (visited on 02/14/2026).
- [117] *User properties—MQTT 5 essentials part 6*. 2022. URL: <https://www.hivemq.com/blog/mqtt5-essentials-part6-user-properties/>.
- [118] Sofie Van Hoecke et al. “SAMuS: Service-Oriented Architecture for Multisensor Surveillance in Smart Homes”. en. In: *The Scientific World Journal* 2014 (2014), pp. 1–9. ISSN: 2356-6140, 1537-744X. DOI: 10.1155/2014/150696. URL: <http://www.hindawi.com/journals/tswj/2014/150696/> (visited on 03/09/2026).

- [119] Blesson Varghese and Rajkumar Buyya. “Next generation cloud computing: New trends and research directions”. In: *Future Generation Computer Systems* (2018).
- [120] Blesson et al. Varghese. “Challenges and opportunities in edge computing”. In: *IEEE Cloud Computing* (2016).
- [121] Jeffrey Voas. *Networks of 'Things'*. en. Tech. rep. NIST Special Publication (SP) 800-183. National Institute of Standards and Technology, July 2016. DOI: 10 . 6028/NIST.SP.800-183. URL: <https://csrc.nist.gov/pubs/sp/800/183/final> (visited on 02/25/2026).
- [122] Jiafu Wan et al. “A comprehensive study of internet of things”. In: *IEEE access : practical innovations, open solutions* 4 (2016), pp. 363–378. DOI: 10 . 1109/ACCESS.2016.2523501.
- [123] Liang et al. Wang. “Peeking behind the curtains of serverless platforms”. In: *Usenix atc*. 2018.
- [124] *What is an Edge Device | Glossary*. en-US. URL: <https://www.hpe.com/us/en/what-is/edge-device.html> (visited on 03/11/2026).
- [125] *What is DDS?* URL: <https://www.dds-foundation.org/what-is-dds-3/> (visited on 03/11/2026).
- [126] *What Is Event-Driven Architecture? | IBM*. en. Oct. 2021. URL: <https://www.ibm.com/think/topics/event-driven-architecture> (visited on 03/09/2026).
- [127] *What is MQTT Last Will and Testament (LWT)? – MQTT Essentials: Part 9*. en. Section: blog. Feb. 2026. URL: <https://www.hivemq.com/blog/mqtt-essentials-part-9-last-will-and-testament/> (visited on 03/02/2026).
- [128] *What is MQTT Quality of Service (QoS) 0,1, & 2? – MQTT Essentials: Part 6*. en. Section: blog. Feb. 2026. URL: <https://www.hivemq.com/blog/mqtt-essentials-part-6-mqtt-quality-of-service-levels/> (visited on 03/02/2026).
- [129] *What is Service-Oriented Architecture (SOA)? | IBM*. en. Oct. 2021. URL: <https://www.ibm.com/think/topics/soa> (visited on 03/09/2026).

- [130] Peter R. Wurman, Raffaello D'Andrea, and Mick Mountz. "Coordinating hundreds of cooperative, autonomous vehicles in warehouses". In: *AI Magazine* (2008).
- [131] Xingwen Xiao, Chushu Gao, and Justus Bogner. "On the Effectiveness of Microservices Tactics and Patterns to Reduce Energy Consumption: An Experimental Study on Trade-Offs". In: *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*. arXiv:2501.14402 [cs]. Mar. 2025, pp. 164–175. DOI: 10.1109/ICSA65012.2025.00025. URL: <http://arxiv.org/abs/2501.14402> (visited on 02/25/2026).
- [132] Shanhe et al. Yi. "A survey of fog computing". In: *Workshop on Mobile Big Data* (2015).
- [133] Andrea Zanella et al. "Internet of things for smart cities". In: *IEEE Internet of Things Journal* 1.1 (2014), pp. 22–32. DOI: 10.1109/JIOT.2014.2306328.